

Material adapted from Lee & Seshia “Introduction to Embedded Systems”

Embedded Systems:

Module 3 – Hybrid and Timed Automata

ECE 522/622
UMass Amherst
Spring 2026
Prof. Daniel Holcomb

2/24/26

lecture 7

discuss:

- PS1
- Lab 1 due 5pm on Weds 3/4
- Exam on 3/12. Email me by Thursday 2/26 if you have a conflict (e.g. three exams on same day)

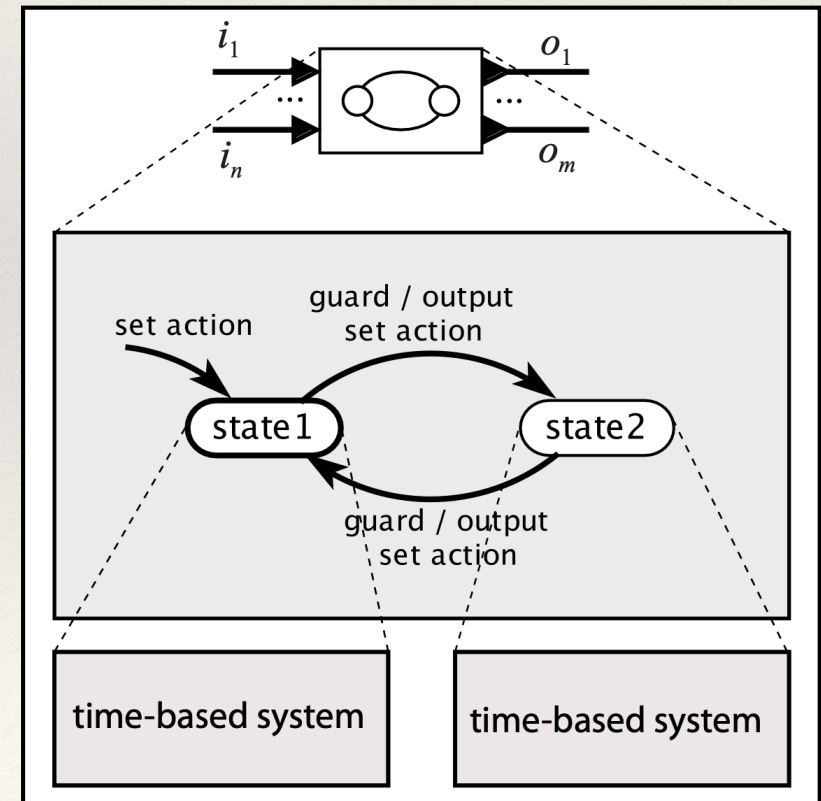
Roadmap

- ~~Introduction~~
- ~~Basic Reachability Analysis (ch 3, 5)~~
- ~~Discrete Dynamics and FSMs (ch 3, 5)~~
- Hybrid Systems and Timed Automata (ch 4)
- Compositions of discrete systems (ch 5)
- Invariants, Temporal Logic (ch 13)
- Equivalence, Refinement/Abstraction, Simulation Relations (ch 14)
- Model Checking (ch 15)
- Models of Computation (ch 6)
- Scheduling (ch 12)
- Sensors, Sampling, ADC (ch 2, 7)
- I/O and Memory

- ❖ Hybrid systems are discrete and continuous dynamics within a single model
- ❖ Hybrid systems are the topic of upcoming labs:
 - ❖ Lab 2 (timed automata)
 - ❖ Lab 3 (general hybrid systems)
- ❖ Hybrid systems are covered in Chapter 4 in textbook
 - ❖ We'll deviate a bit from the book in how we define hybrid systems, to match up with the lab tools

Main Idea of Hybrid Systems

- ❖ **Discrete systems** (module 2) change state by instantaneous jumps
 - ❖ Jumps governed by updates (D-FSM) or possible updates (ND-FSM)
 - ❖ No concept of time. State is unchanging between jumps.
- ❖ **Continuous systems** have state variables that change in time
 - ❖ Evolution modeled with differential equations
 - ❖ $\dot{x}(t) = 3$
 - ❖ $\dot{x}(t) = 10 + \cos(2\pi t)$
 - ❖ ...
- ❖ **Hybrid systems** have both
 - ❖ Variables evolve according to equations (continuous)
 - ❖ Instantaneously jump to different equations (discrete)



- ❖ The equations describing evolution of (continuous) variables can be different for each mode of operation
 - ❖ The “modes” are usually called “**locations**” in hybrid systems. We will use the 2 terms interchangeably
- ❖ Location transitions:
 - ❖ Governed by guards evaluated over state vars and inputs
 - ❖ Guard conditions are **evaluated continuously**
 - ❖ Can set outputs and update variables on transitions
- ❖ Can have “**location invariants**” that restrict allowed values of variables in each mode/location

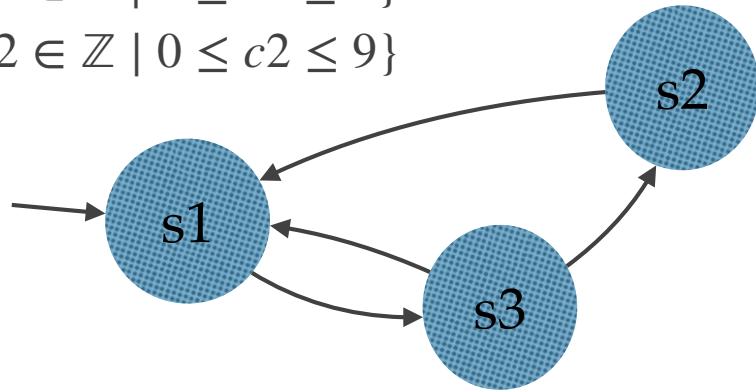
(example for board)

- ❖ How many states does each system have?

variables: $c1, c2$

$\{c1 \in \mathbb{Z} \mid 0 \leq c1 \leq 9\}$

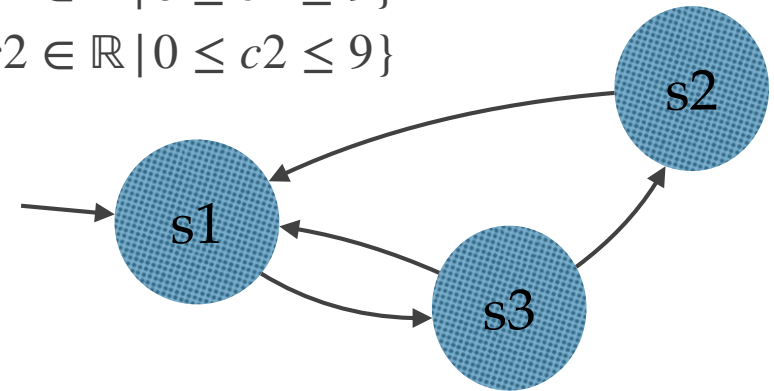
$\{c2 \in \mathbb{Z} \mid 0 \leq c2 \leq 9\}$



variables: $c1, c2$

$\{c1 \in \mathbb{R} \mid 0 \leq c1 \leq 9\}$

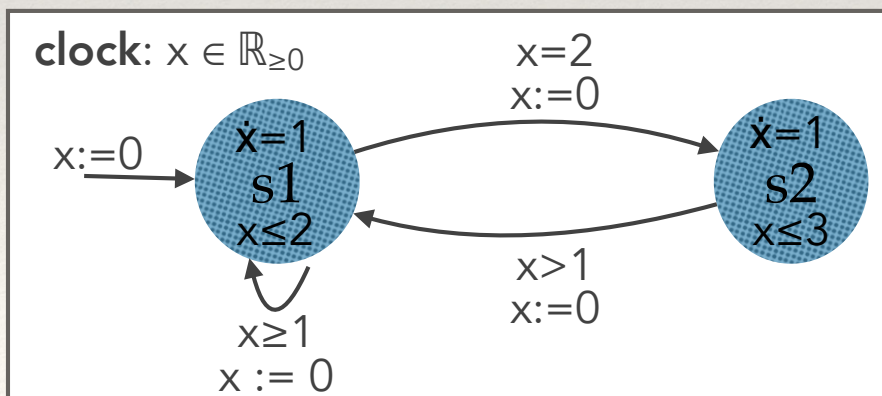
$\{c2 \in \mathbb{R} \mid 0 \leq c2 \leq 9\}$



Hybrid Systems vs E-FSM

- ❖ States of hybrid systems are combination of mode/location and valuation of variables (similar to extended FSM)
 - ❖ Num. states in E-FSM: $num_modes * num_var_valuations^{num_vars}$
- ❖ Hybrid systems have infinite state
 - ❖ Variables are real-valued, so $num_var_valuations$ is infinite
 - ❖ Reachability analysis problematic because it is not possible to visit all reachable states explicitly
- ❖ Checking reachability is **impossible** in **general hybrid systems**
 - ❖ But is **possible** for **restricted classes of hybrid systems**
 - ❖ Timed automata are a restricted class of hybrid system, for which it is possible to check reachability

- ❖ TA are special case of Hybrid systems. Useful for modeling many systems, but still relatively simple. Can check reachability despite infinite state.
- ❖ Defining property of TA: **Continuous variables have constant derivative in all locs**
Unlike book we will require that derivatives are always 1 ("single-rate TA")
 - ❖ The variables with derivative 1 are called "clocks"
 - ❖ We will assume that variables are only compared to integer values (i.e. in guards)
- ❖ When guards are true, mode transitions are **possible but not mandatory**
 - ❖ Because of this, T.A. are usually non-deterministic
 - ❖ **Location invariants** restrict allowed clock values in each loc and can force jumps

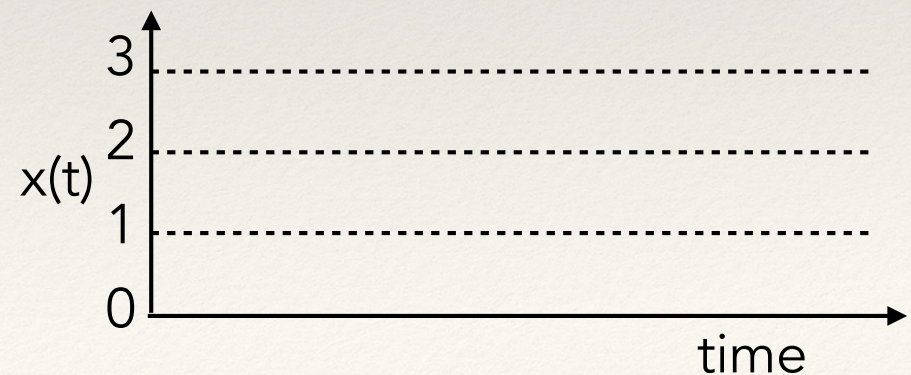
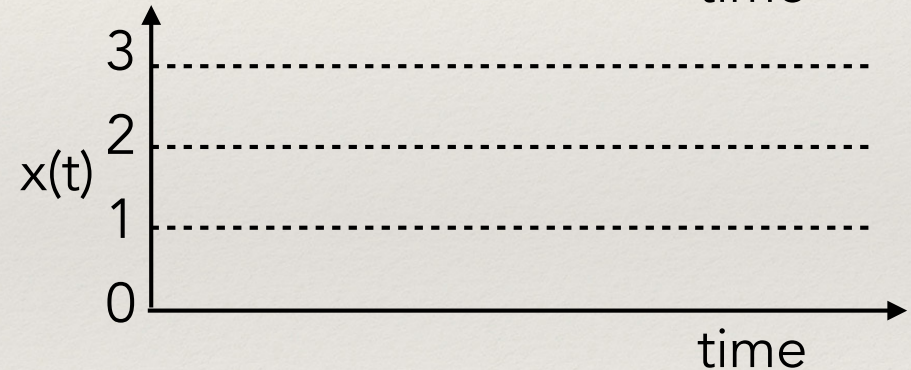
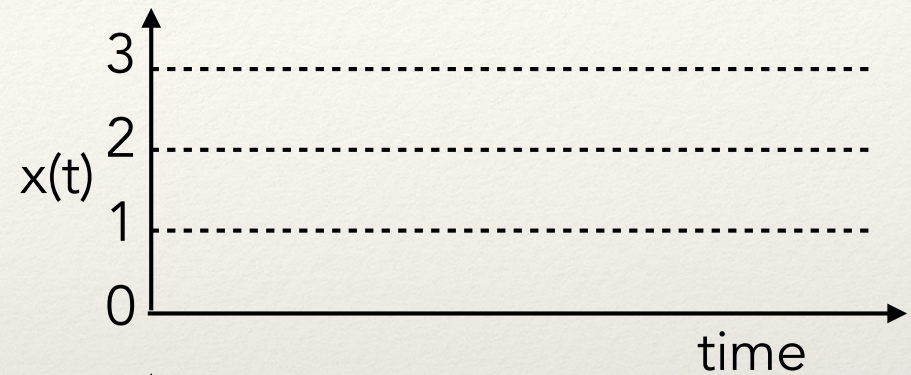
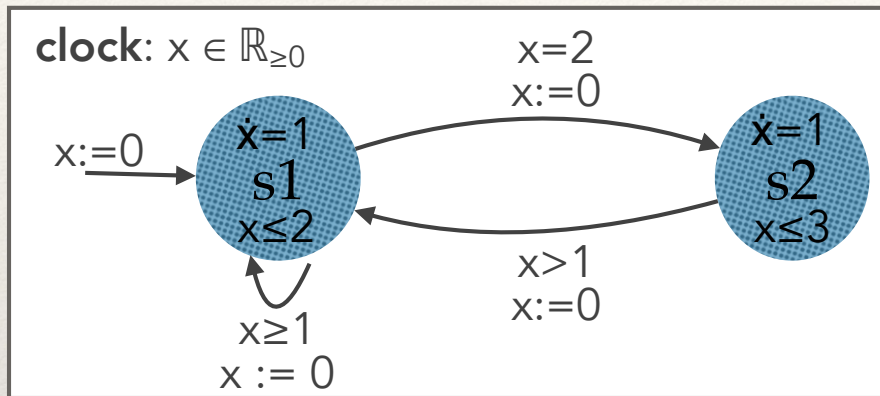


Questions:

1. How many states?
2. Give an example execution with mode changes
3. Give examples of states (loc. and x val) that are unreachable (use intuition, no algorithm yet)

(example to fill in on board)

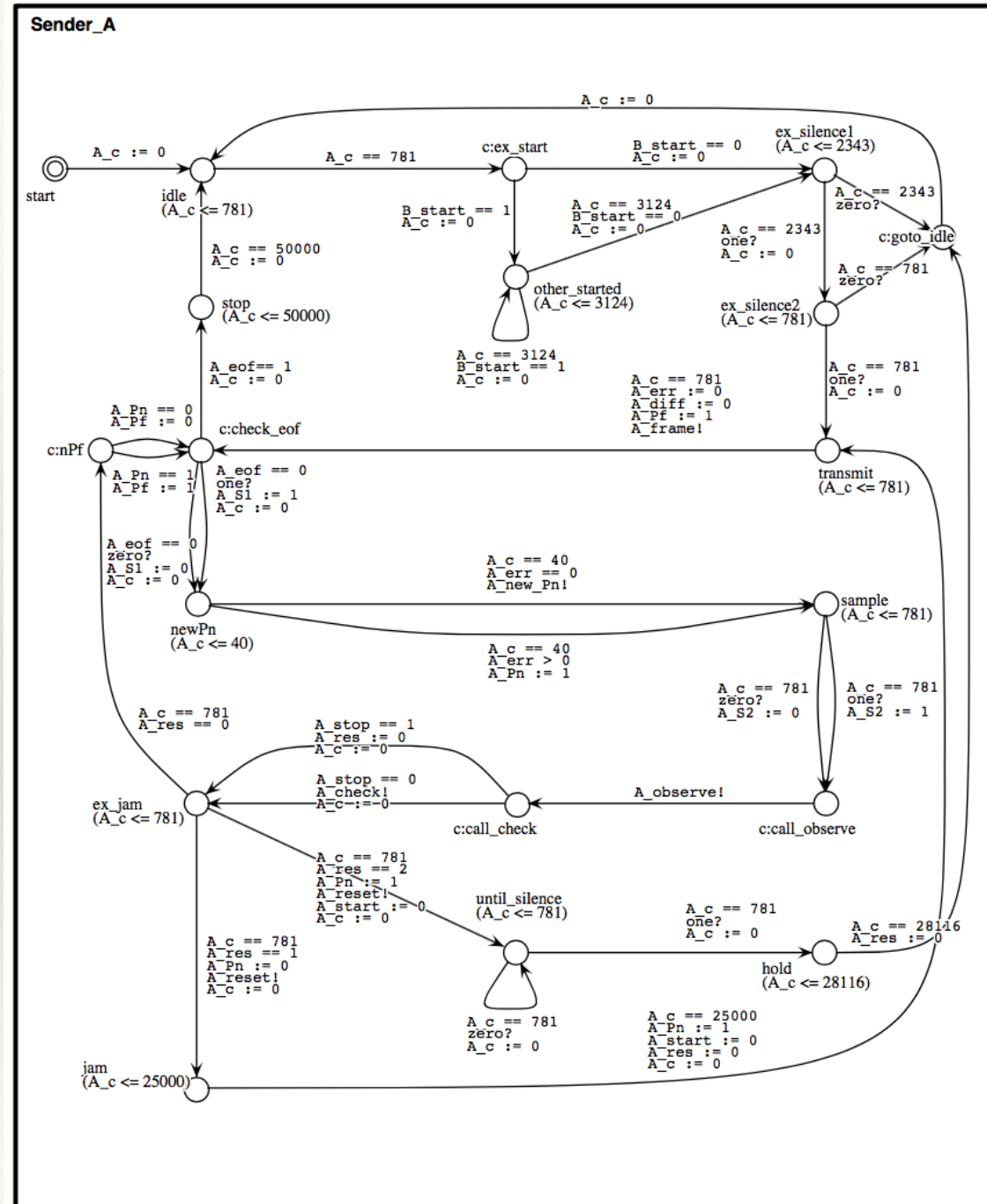
- ❖ Example executions of the TA model on previous slide
- ❖ Note where the jumps occur (discrete) and where time is elapsing (continuous)



- ❖ **Philips Audio Protocol with Bus Collision:** This is an extended variant of Philips audio control protocol. New components (and variables) are introduced, and existing components are modified to deal with bus collisions. Correctness is originally proved by hand, and then by model-checking using Uppaal in [BGKLLPW96]
- ❖ **TDMA Protocol Start-Up Mechanism:** In [LP97], a formal verification of the start-up algorithm of a TDMA protocol is reported. It was proved using Uppaal that four communicating stations become synchronized and operational within bounded time from arbitrary initial state, given a clock-drift corresponding to 1/1000. Furthermore, an upper time-bound for the start-up to complete was derived.
- ❖ **Gearbox Controller:** UPPAAL is applied to design and analyse a prototype gearbox controller for vehicles by Mecel AB [LPW98]. The gearbox controller is a component in the real-time distributed system that controls a modern car. The gear-requests are delivered over a communication network to the gearbox controller that implements the gear change by actuating the clutch, engine and gearbox. The symbolic simulator of Uppaal is applied to validate the system behavior. The correctness of the gear-box controller design is established by automatic proofs of 46 properties derived from requirements specified by Mecel AB.

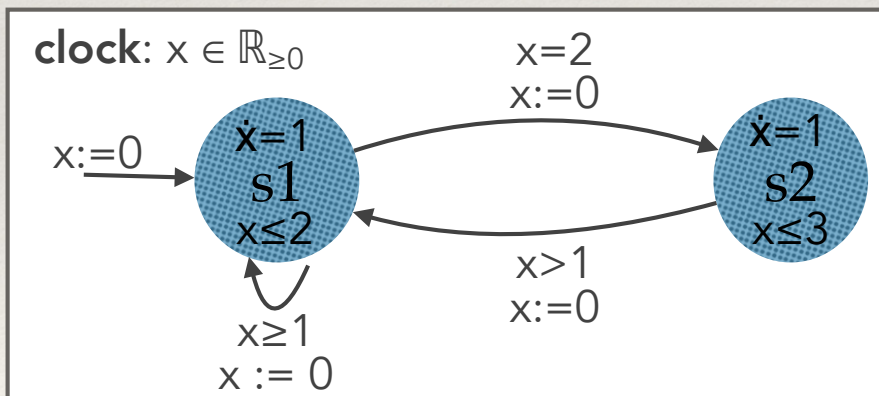
- ❖ TA verification algorithms (i.e. reachability) can scale up to handle real systems
- ❖ Models of industrial systems are large and complicated, so we stick to simple models in class

Figure from: K. Havelund, A. Skou, K. G. Larsen and K. Lund, "Formal modeling and analysis of an audio/video protocol: an industrial case study using UPPAAL," Proceedings Real-Time Systems Symposium, 1997, pp. 2-13.



A Simple Timed Automaton

- ❖ How to check reachability of Timed Automata:
 - ❖ **Partition infinite state space into a finite set of clock regions** such that points within region are “equivalent”
 - ❖ Then, check reachability using regions
- ❖ If guards and location invariants are integers, splitting continuous space at/between all relevant integers creates sufficiently precise regions
- ❖ The UPPAAL tool will perform this analysis for you in lab 2, but you will still need to understand how it works



Questions:

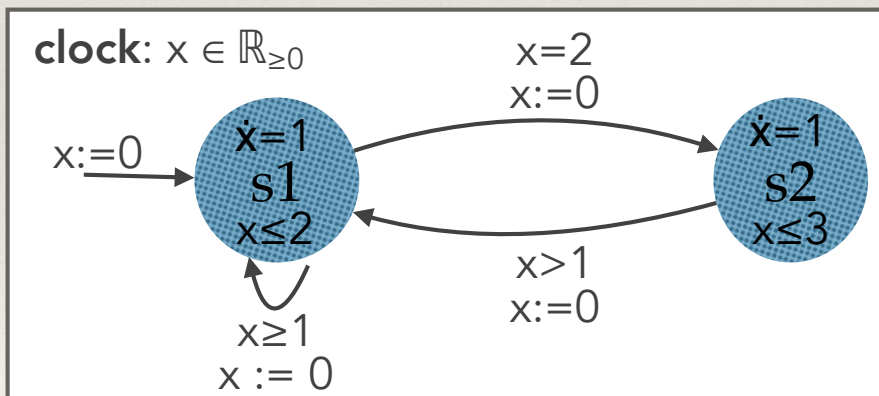
1. Can system be in loc. s1 when clock $x > 2$?
 2. Can system be in loc. s2 when clock $x = 3$?
- (Answer by reachability using clock regions)



we have these 8 regions of interest for clock x because 3 is the largest value compared to clock in **any** location

A Simple Timed Automaton

- ❖ Reachability must account for all possible transitions between clock regions.
There are two types of transitions between clock regions:
 1. **Jump** transitions according to the guards shown in the graph
 - ❖ i.e. jump from (loc s1, $x=1$) to (loc s1, $x=0$)
 2. Transitions that occur due to **time elapsing** within a location
 - ❖ i.e. **flow** from (loc s1, $x=1$) to (loc s1, $1 < x < 2$)



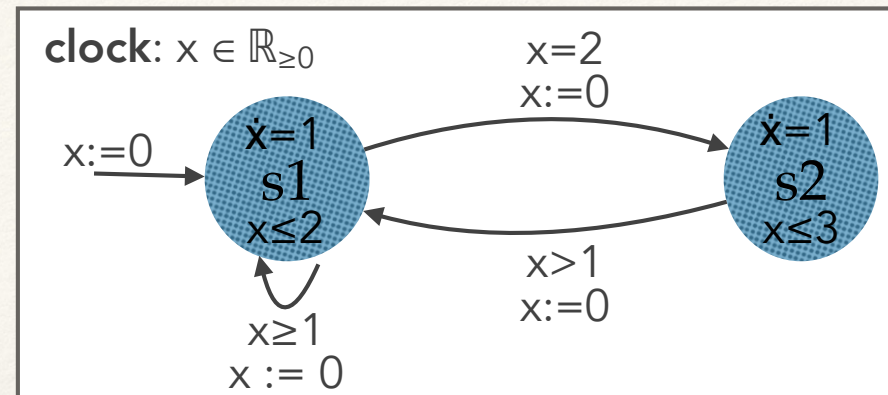
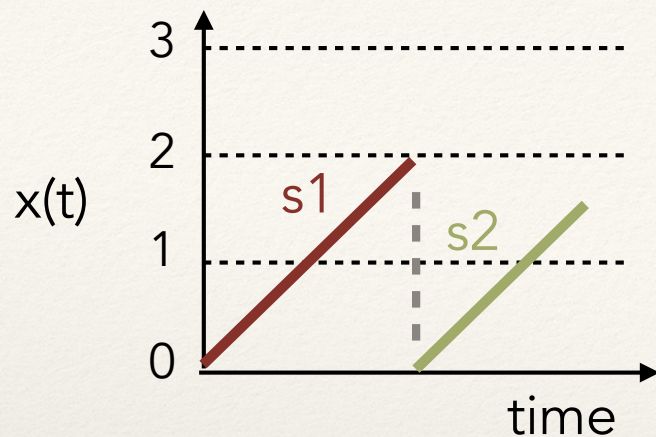
Questions:

1. Can system be in loc. s1 when clock $x > 2$?
 2. Can system be in loc. s2 when clock $x = 3$?
- (Answer by reachability using clock regions)

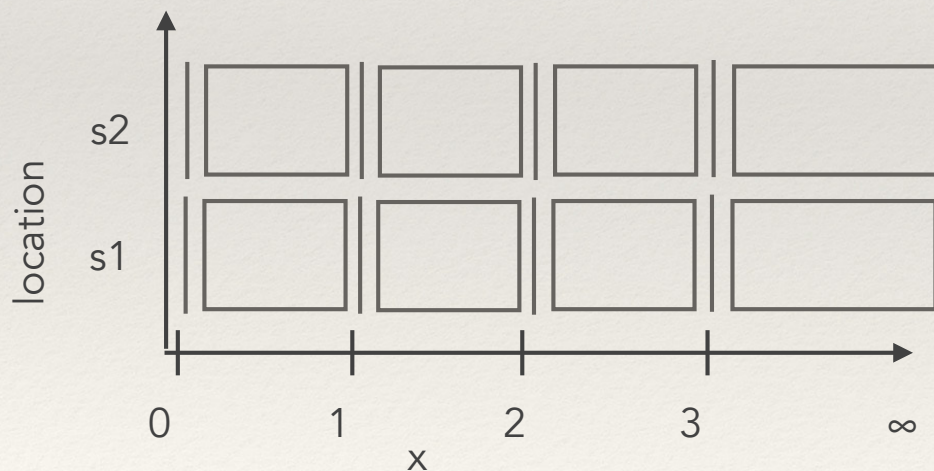


we have these 8 regions of interest for clock x because 3 is the largest value compared to clock in **any** location

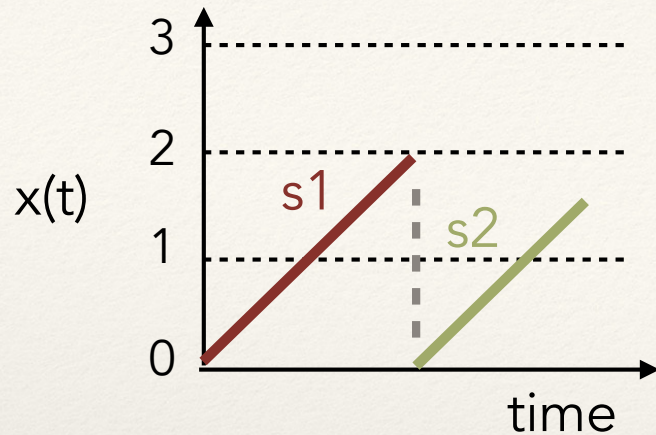
(example on board to fill in)



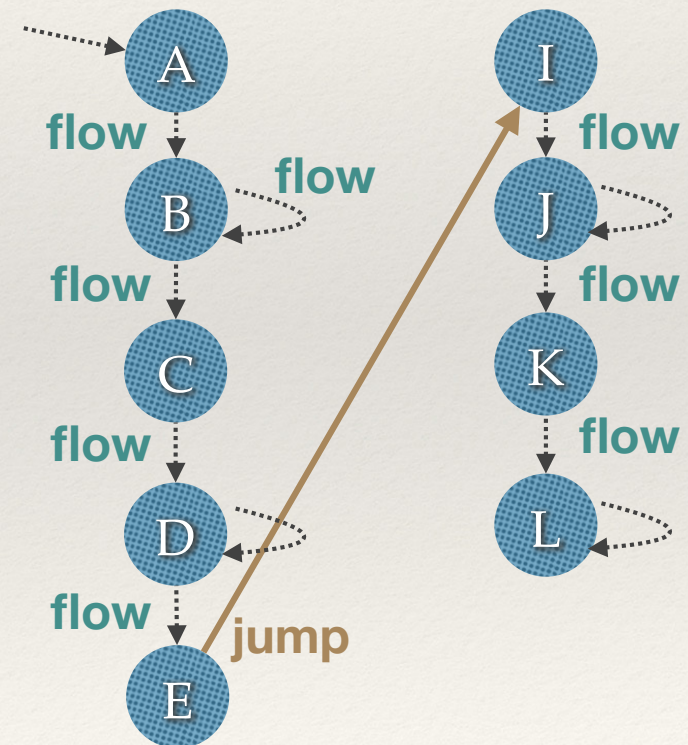
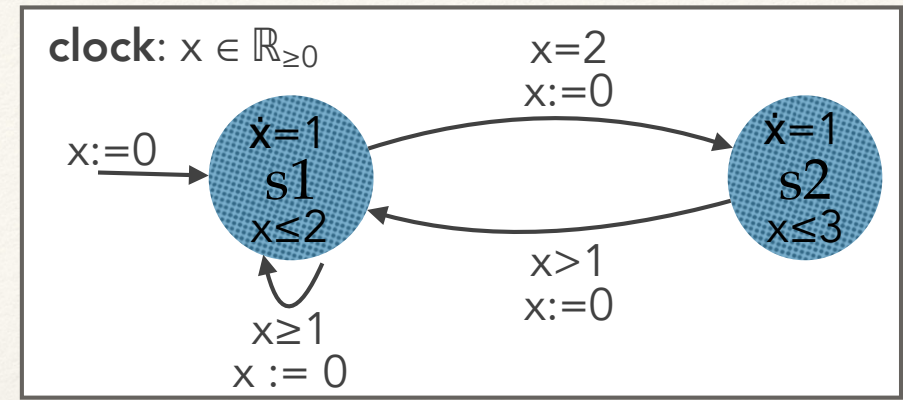
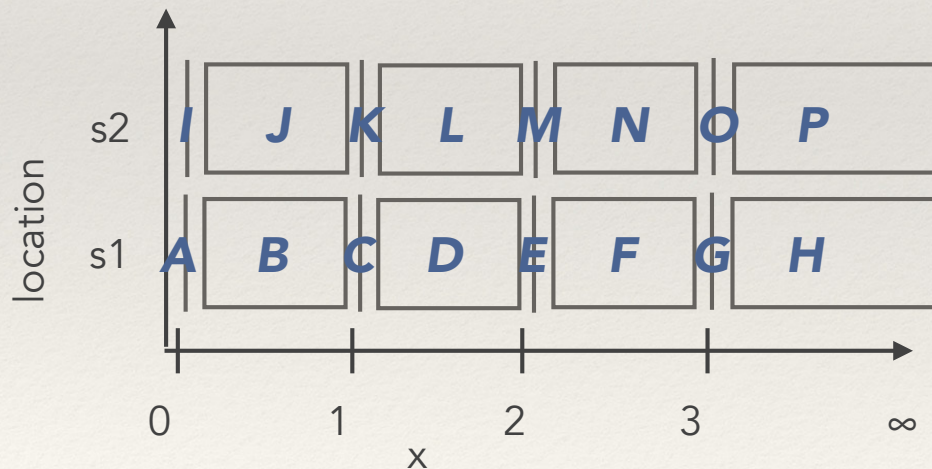
Show the behavior in the plot above as jump and flow transitions between the corresponding clock regions below



(example on board to fill in)

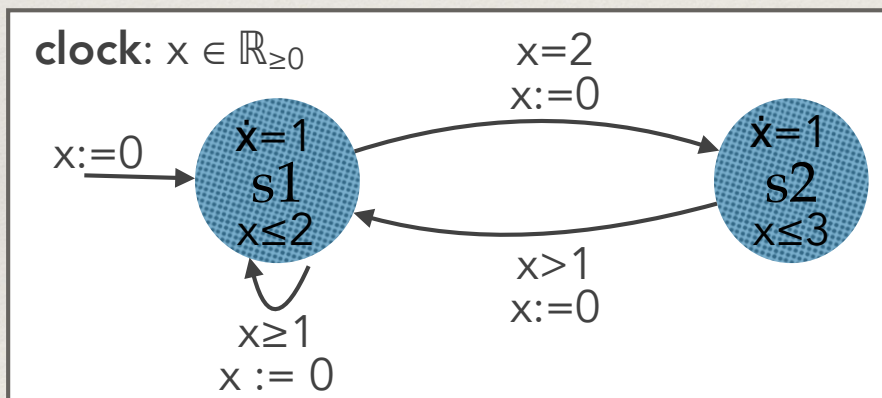
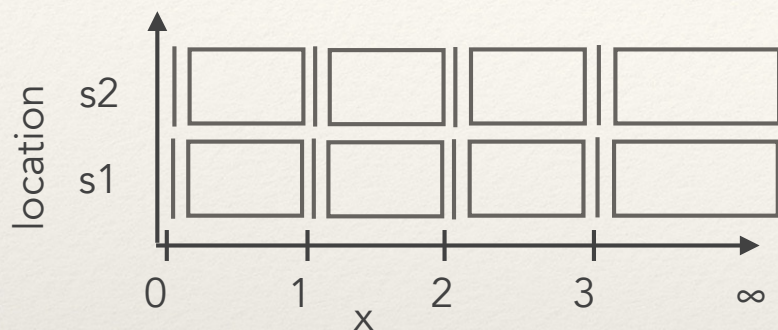


Show the behavior in the plot above as jump and flow transitions between the corresponding clock regions below



(example on board to fill in)

Repeat process from prior slide, but now considering all possible jumps and flows, instead of just behavior given in plot



current state

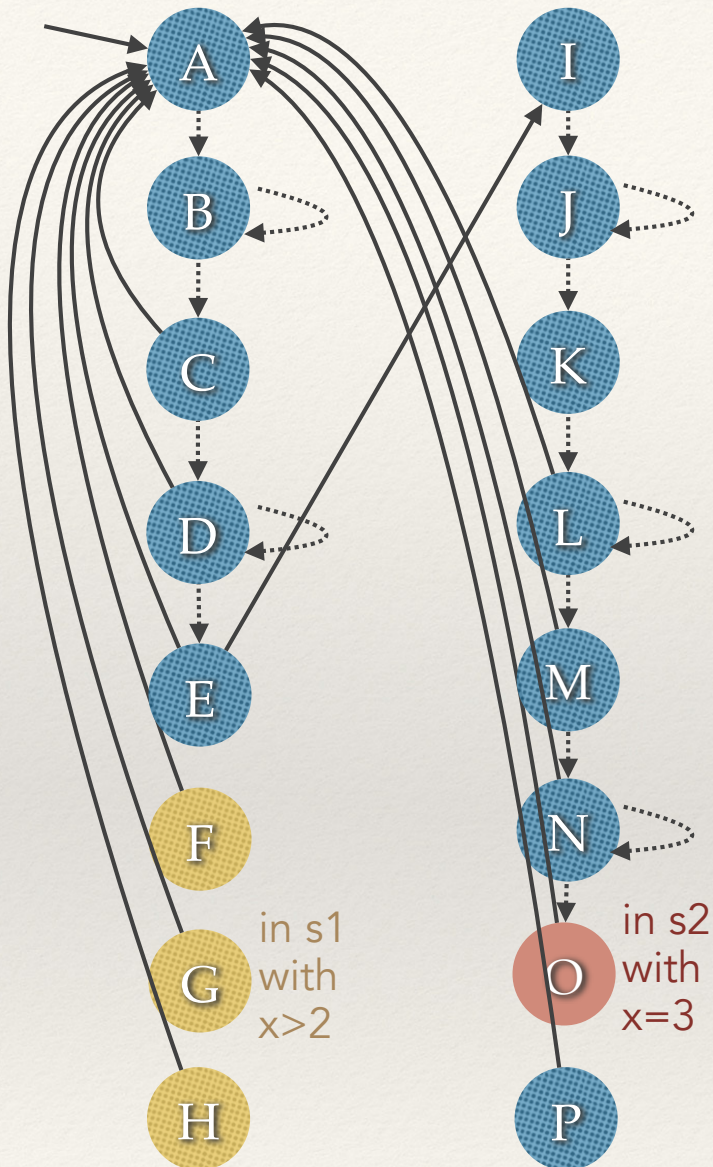
next state

loc.	variable range	region "name" in graph	can time elapse in region?	Jumps enabled?	other reachable regions when time elapses?
s1	$x=0$	A			
s1	$0 < x < 1$	B			
s1	$x=1$	C			
s1	$1 < x < 2$	D			
s1	$x=2$	E			
s1	$2 < x < 3$	F			
s1	$x=3$	G			
s1	$3 < x < \infty$	H			
s2	$x=0$	I			
s2	$0 < x < 1$	J			
s2	$x=1$	K			
s2	$1 < x < 2$	L			
s2	$x=2$	M			
s2	$2 < x < 3$	N			
s2	$x=3$	O			
s2	$3 < x < \infty$	P			

Questions:

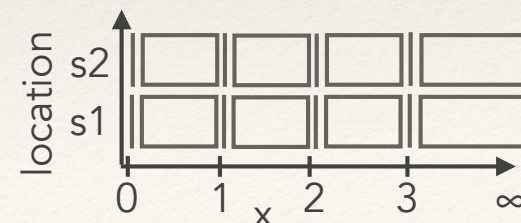
1. Can system be in loc. s1 when clock $x > 2$?
 2. Can system be in loc. s2 when clock $x = 3$?
- (Answer by reachability using clock regions)

(region transitions of example)



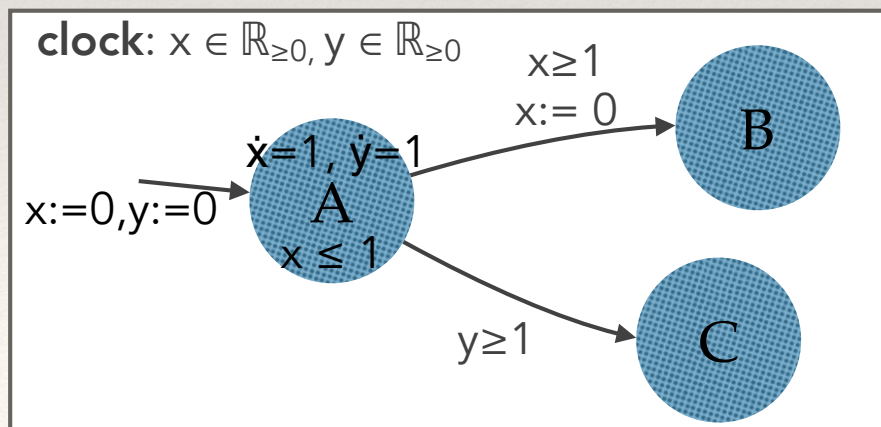
- ❖ dotted edges are time elapsing in TA
- ❖ solid edges are guarded (jump) transitions in TA

loc.	current state			next state	
	variable range	region "name" in graph	can time elapse in region?	Jumps enabled?	other reachable regions when time elapses?
s1	$x=0$	A	-	{}	B
s1	$0 < x < 1$	B	yes	{}	C
s1	$x=1$	C	-	{A}	D
s1	$1 < x < 2$	D	yes	{A}	E
s1	$x=2$	E	-	{A, I}	-
s1	$2 < x < 3$	F	-	{A}	violates loc. invariant
s1	$x=3$	G	-	{A}	violates loc. invariant
s1	$3 < x < \infty$	H	-	{A}	violates loc. invariant
s2	$x=0$	I	-	{}	J
s2	$0 < x < 1$	J	yes	{}	K
s2	$x=1$	K	-	{}	L
s2	$1 < x < 2$	L	yes	{A}	M
s2	$x=2$	M	-	{A}	N
s2	$2 < x < 3$	N	yes	{A}	O
s2	$x=3$	O	-	{A}	-
s2	$3 < x < \infty$	P	-	{A}	violates loc. invariant



Regions with two clocks

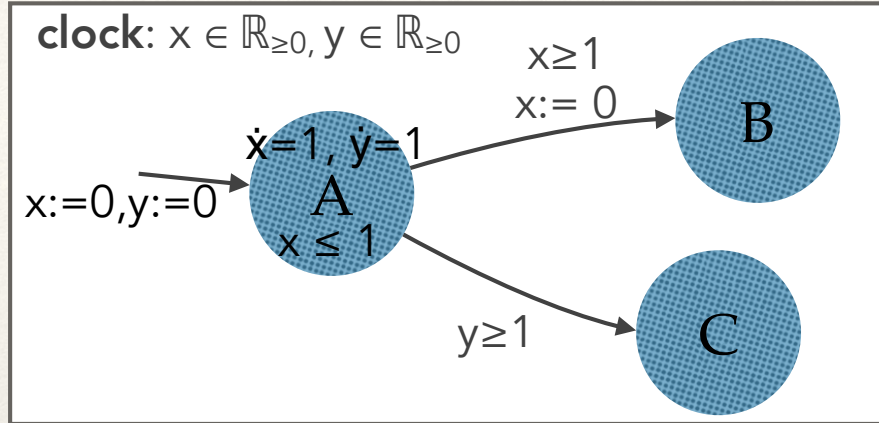
- ❖ If both clocks have same constant rate, and guards and invariants check integer bounds, then it suffices to partition space at integers **and along diagonals**
- ❖ Partitioning on diagonals is required because **ordering** of fractional part of clock determines the successor region when time elapses
 - ❖ Consider location A when x and y are both less than 1
 - ❖ What is next jump that will be enabled when time elapses?
 - ❖ Answer depends on whether $x > y$, $y > x$, or $x = y$
 - ❖ Make regions precise enough to avoid this ambiguity
- ❖ Partitioning space in advance is unnecessary. Done symbolically in UPPAAL.



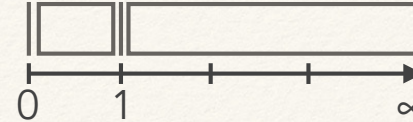
Questions

1. What is state space of this TA?
2. Draw its clock regions
3. How does number of regions change if more clock variables are added?

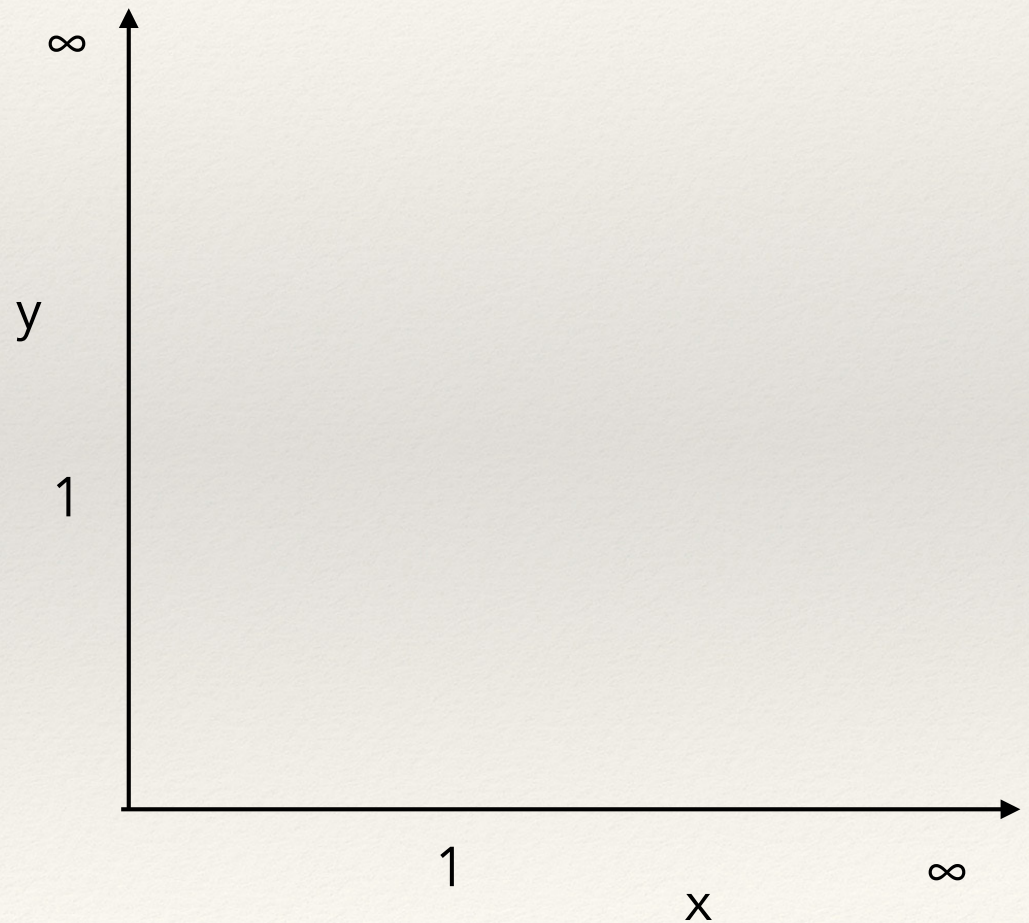
(example on board to fill in)



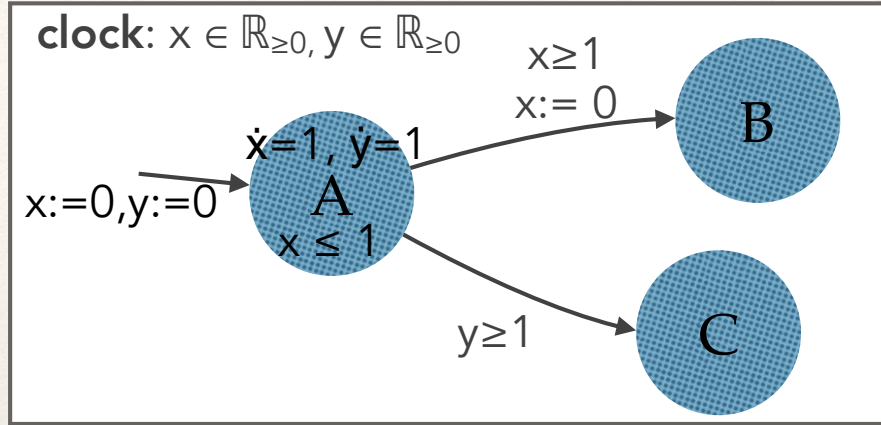
4 regions of interest for clock x because 1 is the largest value compared to it



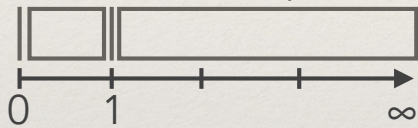
4 regions of interest for clock y because 1 is the largest value compared to it



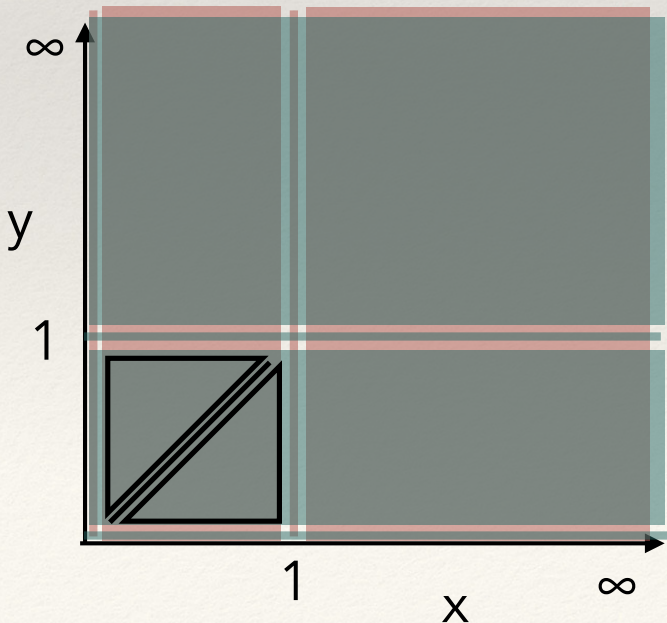
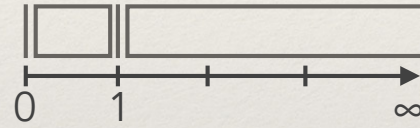
Representing Regions with Constraints



4 regions of interest for clock x because 1 is the largest value compared to it



4 regions of interest for clock y because 1 is the largest value compared to it



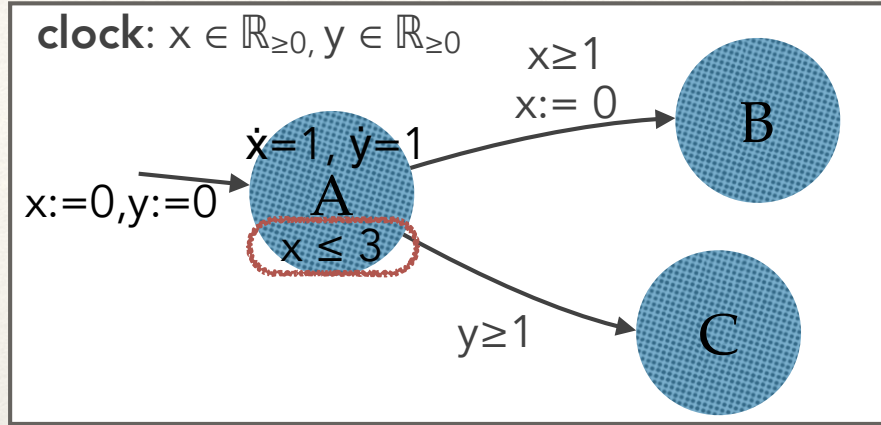
these cases are the triangles and diagonal line

3 constraints define each region

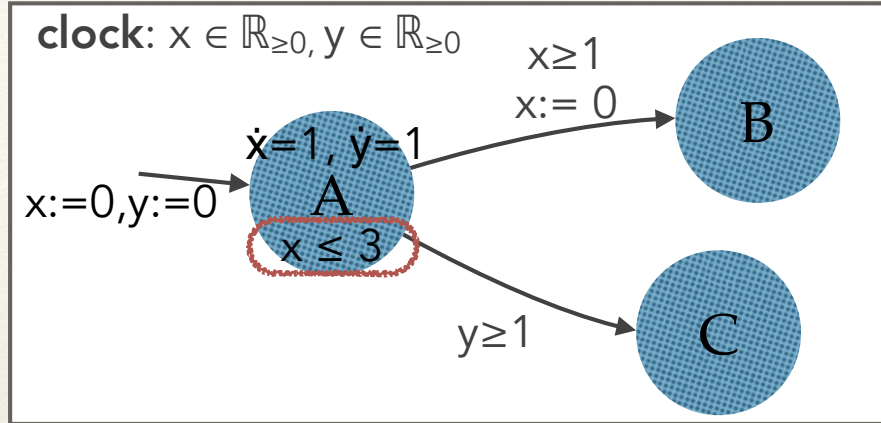
x	y	$y-x$
0	0	-
0	(0,1)	-
0	1	-
0	(1,∞)	-
(0,1)	0	-
(0,1)	(0,1)	0
(0,1)	(0,1)	(0,1)
(0,1)	(0,1)	(-1,0)
(0,1)	1	-
(0,1)	(1,∞)	-
1	0	-
1	(0,1)	-
1	1	-
1	(1,∞)	-
(1,∞)	0	-
(1,∞)	(0,1)	-
(1,∞)	1	-
(1,∞)	(1,∞)	-

"-" means either there is only one possible value or that there is no constraint that needs to be imposed

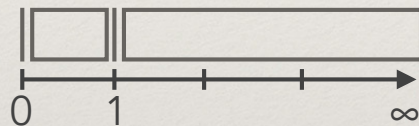
Slightly modified example



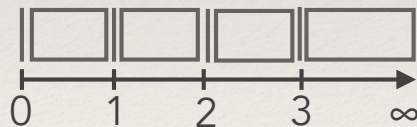
Regions for previous slide



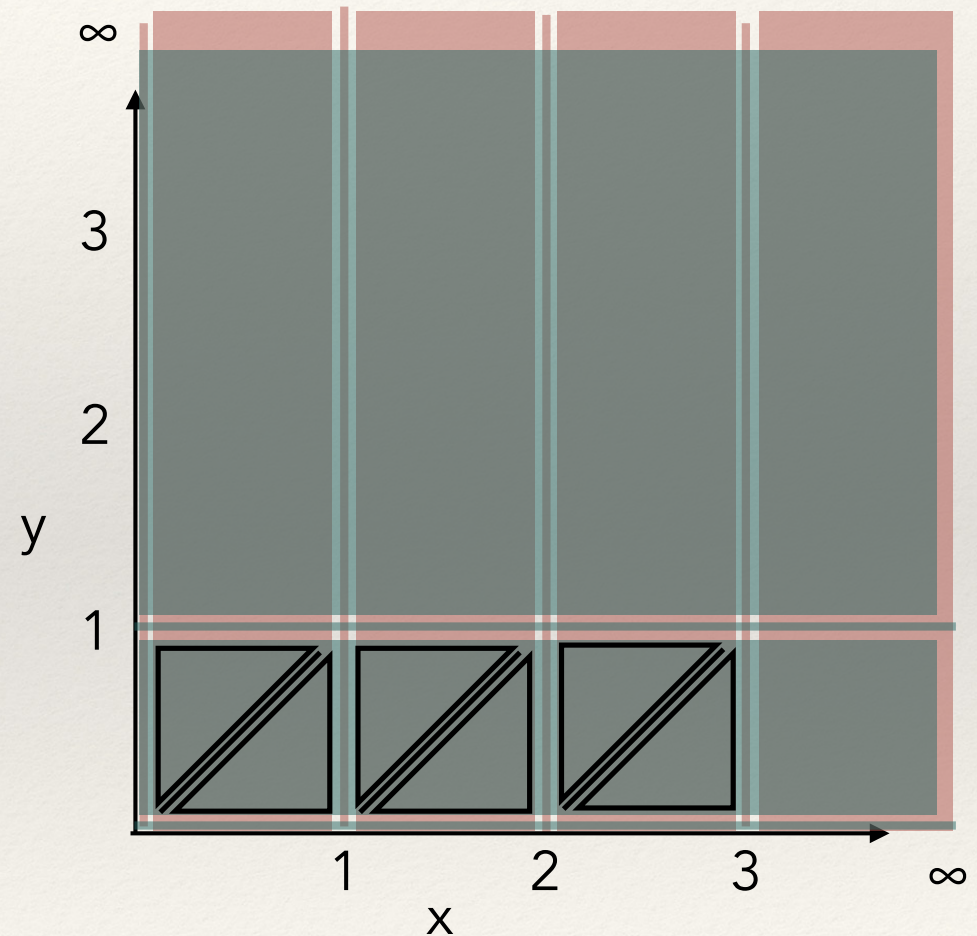
4 regions of interest for clock y because 1 is the largest value compared to it



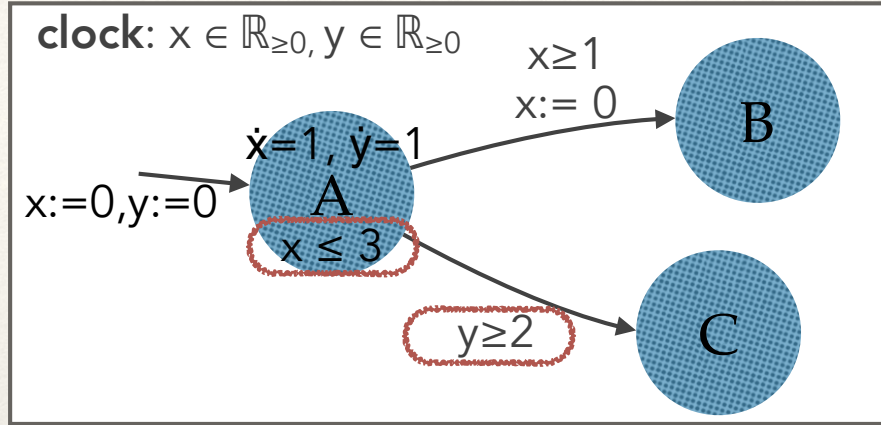
8 regions of interest for clock x because 3 is the largest value compared to it



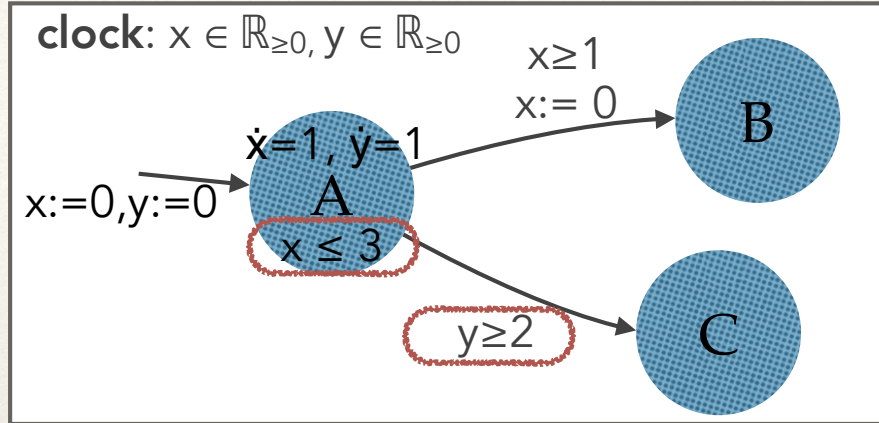
For every clock region as shown below, there is exactly one other region that the system can reach when time elapses



Another slightly modified example

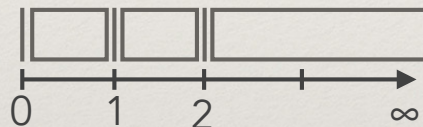


Regions for previous slide

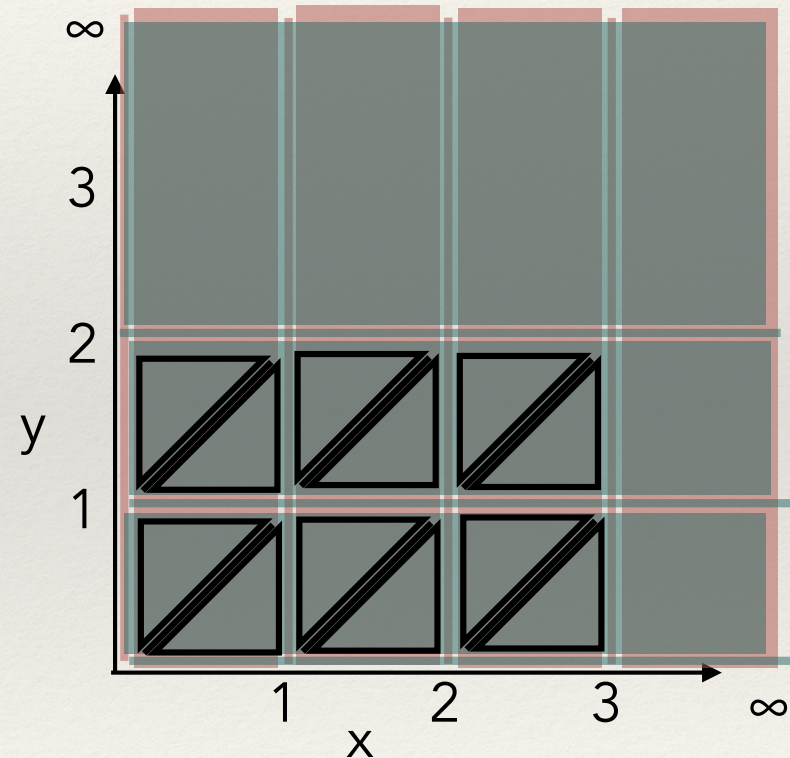


For every clock region as shown below, there is exactly one other region that the system can reach when time elapses

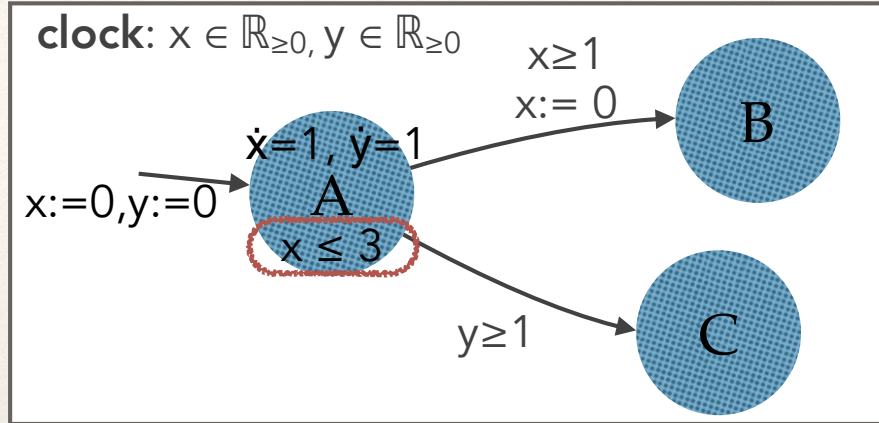
6 regions of interest for clock y because 2 is the largest value compared to it



8 regions of interest for clock x because 3 is the largest value compared to it

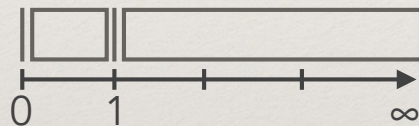


Reducing Number of Regions

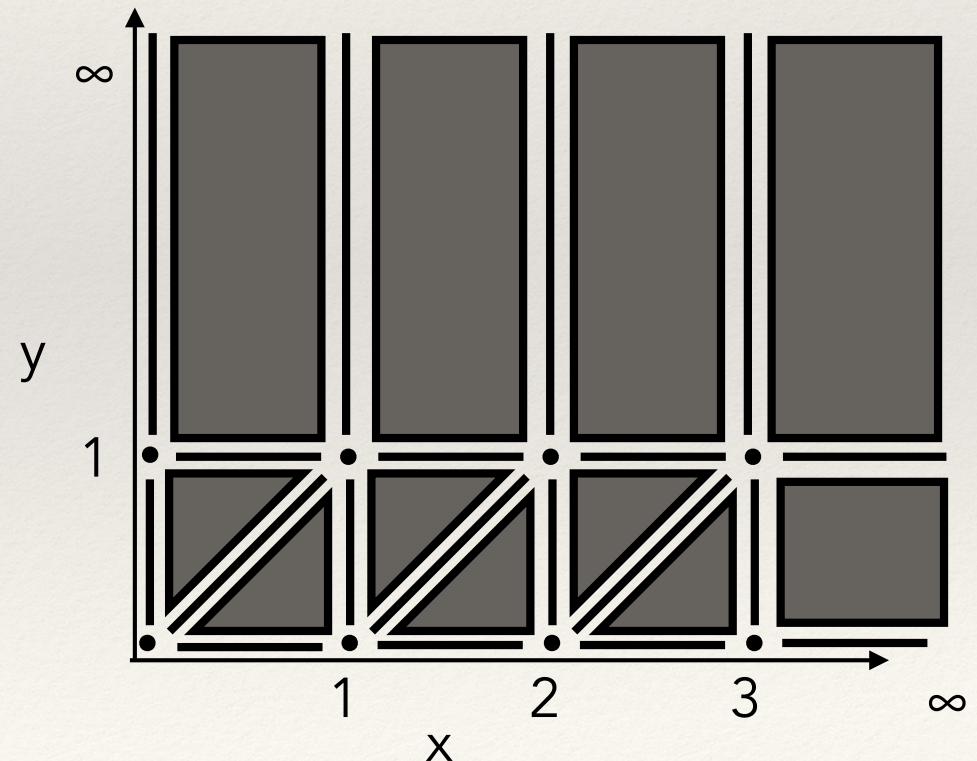
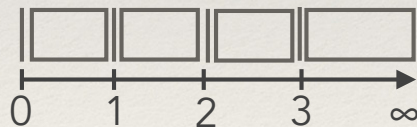


- ❖ The number of clock regions can be reduced if careful to avoid creating ambiguity about reachable regions
- ❖ Tools like UPPAAL don't divide up space into tiny clock regions, but instead minimize number of regions by smartly partitioning space

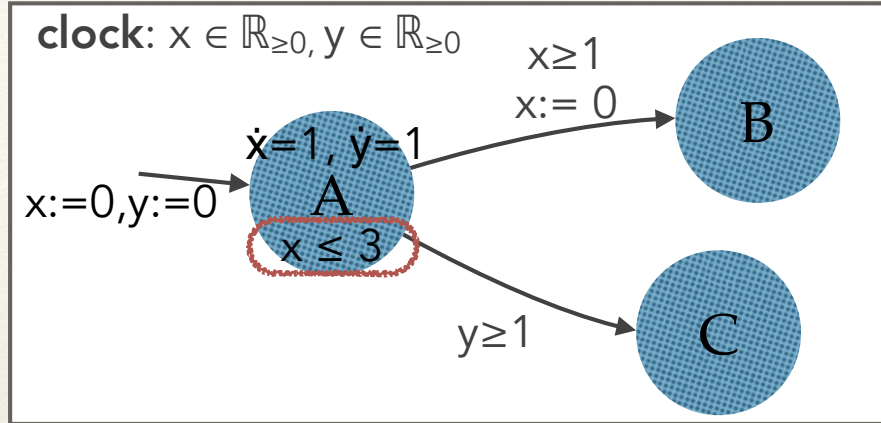
4 regions of interest for clock y because 1 is the largest value compared to it



8 regions of interest for clock x because 3 is the largest value compared to it



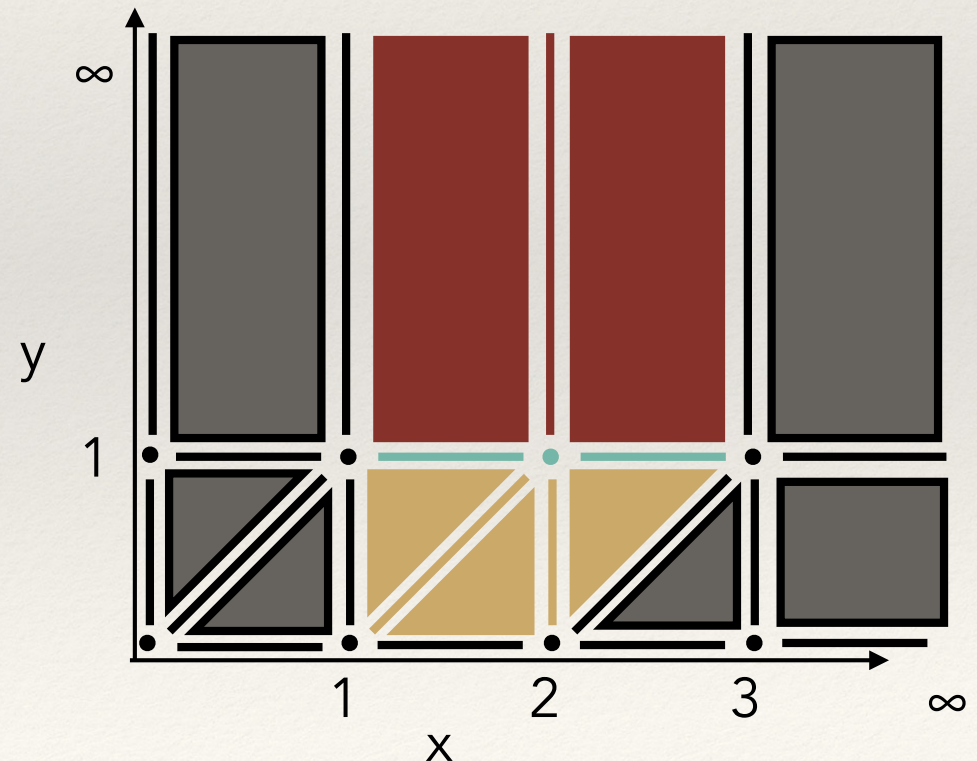
Reducing Number of Regions



- ❖ The number of clock regions can be reduced if careful to avoid creating ambiguity about reachable regions
- ❖ Tools like UPPAAL don't divide up space into tiny clock regions, but instead minimize number of regions by smartly partitioning space

3 constraints define each of the regions

x	y	y-x
(1,3)	(0,1)	(-2,0)
(1,3)	1	-
(1,3)	(1,∞)	-





Search or jump to...



Pull requests

Issues

Codespaces

Marketplace

Explore



UPPAALModelChecker / UDBM Public

Watch 6

Fork 7

Star 6

<> Code

Issues 3

Pull requests

Actions

Wiki

Security

Insights

Home

Kenneth Yrke Jørgensen edited this page on Jun 14, 2021 · 6 revisions

UPPAAL DBM Library

DBM: "difference bound matrices"

DBMs [dill89, rokicki93, lpw:fct95, bengtsson02] are efficient data structures to represent clock constraints in timed automata [ad90]. They are used in UPPAAL [lpy97, by04, bdl04] as the core data structure to represent time. The library features all the common operations such as up (delay, or future), down (past), general updates, different extrapolation functions, etc.. on DBMs and federations. The library also supports subtractions. The API is in C and C++. The C++ part uses active clocks and hides (to some extent) memory management.

- Reliable

The DBM library has an extensive test suite with an extra alternative implementation of the algorithms. This implementation has also been tested on countless case studies.

- Performant

The DBM library is the fruit of many years of development of UPPAAL, bringing to the mainstream efficient algorithms to manipulate DBMs. The API is available in C and C++.

Pages 5

Find a page...

Home

UPPAAL DBM Library

Download

References

Changelog

Help

Python Binding

Ruby Binding

2/26/26

lecture 8

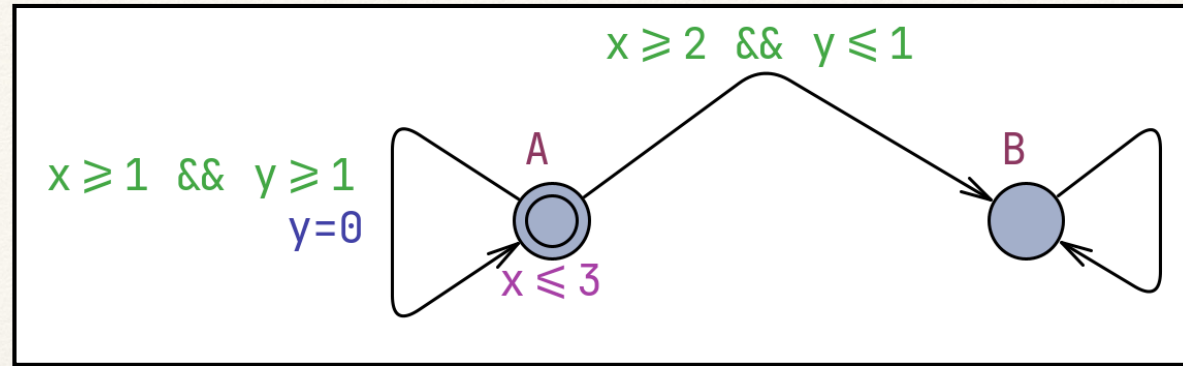
discuss:

- Lab 1 due 5pm on Weds 3/4
- Exam on 3/12. Email me by tonight 2/26 if you have a conflict (e.g. three exams on same day)
- PS2 and Lab 2 coming soon

Introduction to UPPAAL: Tool used for Timed Automata verification in Lab 2 and HW 2

Which conditions are reachable?

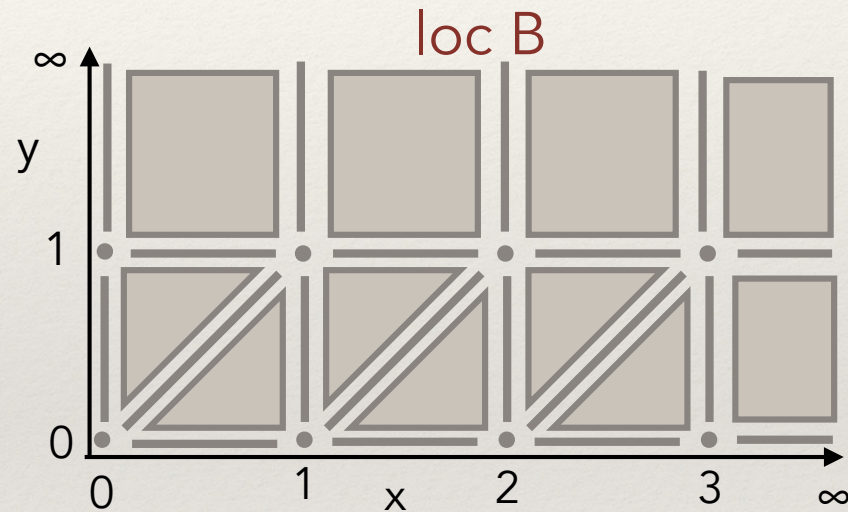
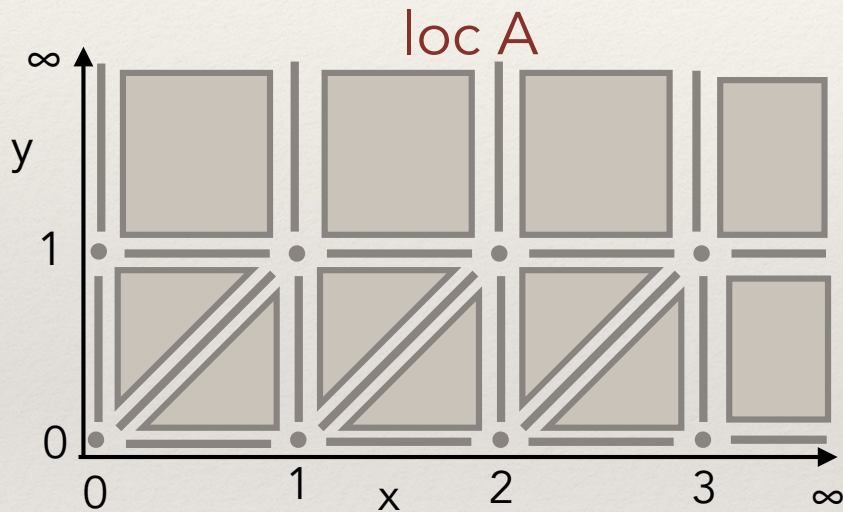
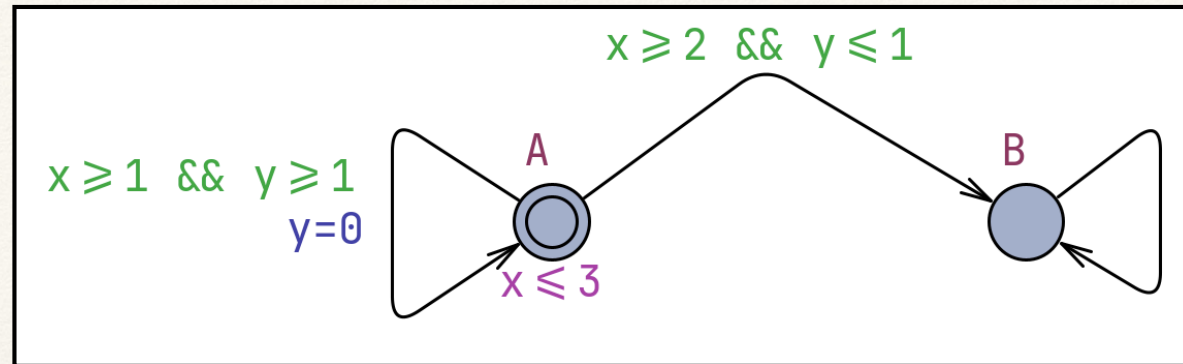
1. loc. B and $x > 2$
2. loc. B and $x == 1$ and $y == 1$
3. loc. A and $x == 1$ and $y == 1$
4. $x > 3$
5. $x == 1$ and $0 < y < 1$
6. $x == 1$ and $0 \leq y < 1$



Solve by hand using regions, then with UPPAAL

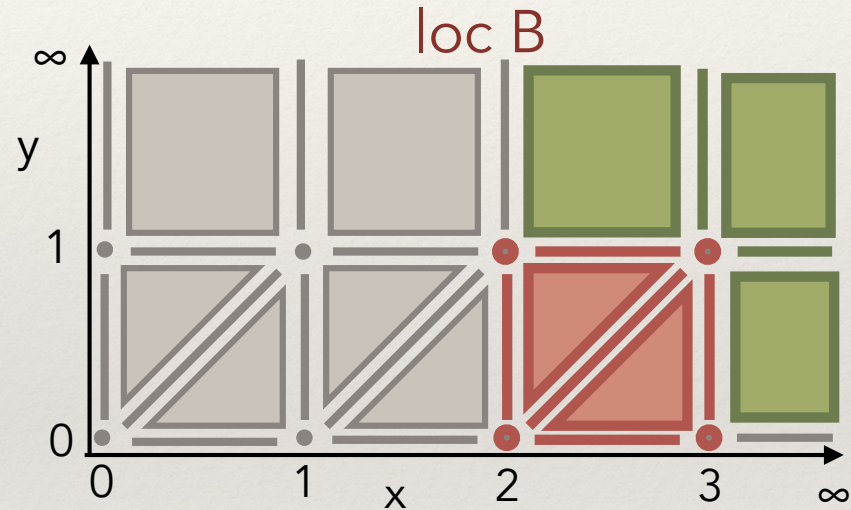
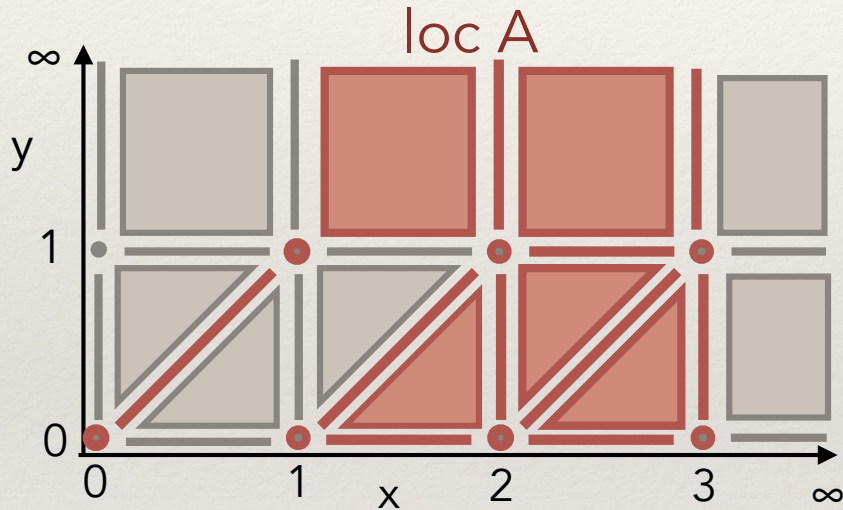
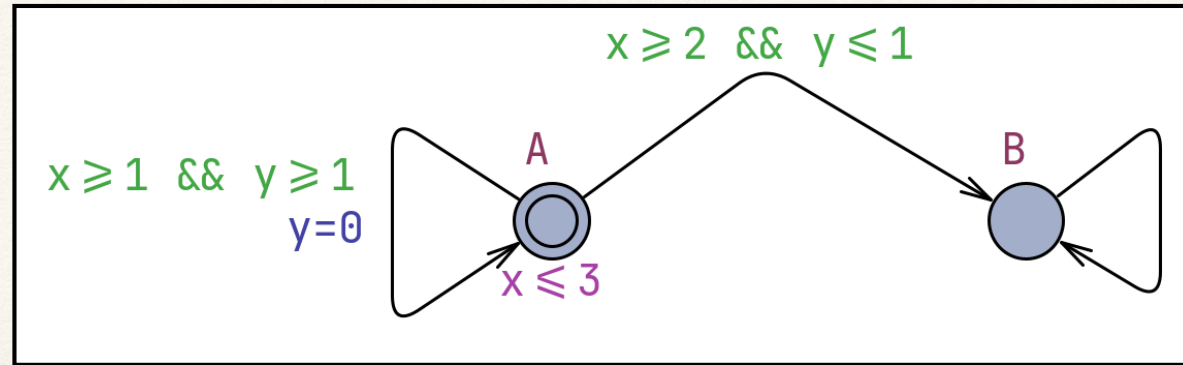
(Example on board to fill in)

1. loc. B and $x > 2$
2. loc. B and $x == 1$ and $y == 1$
3. loc. A and $x == 1$ and $y == 1$
4. $x > 3$
5. $x == 1$ and $0 < y < 1$
6. $x == 1$ and $0 \leq y < 1$



(Example on board, filled in)

1. loc. B and $x > 2$
2. loc. B and $x == 1$ and $y == 1$
3. loc. A and $x == 1$ and $y == 1$
4. $x > 3$
5. $x == 1$ and $0 < y < 1$
6. $x == 1$ and $0 \leq y < 1$



- ❖ Checking properties of Timed Automata models
- ❖ Using a popular TA model checker called UPPAAL (version 5.0). UPPAAL License key posted on Canvas
- ❖ **Homework 2** introduces UPPAAL using a well-defined problem
- ❖ **Lab 2** is a design problem
 - ❖ Part A: Vikings crossing bridge (as tutorial)
 - ❖ Part B: Traffic light controller
 - ❖ No coding. Create models using UPPAAL's GUI

- ❖ Next few slides introduce UPPAAL, but not meant to be a tutorial
 - ❖ You will need to go beyond what is shown in these slides
 - ❖ Good online resources for UPPAAL
 - ❖ UPPAAL download includes many examples
 - ❖ Expansive modeling language. Have fun and explore.
- ❖ You will use (at least) three components of UPPAAL:
 1. **Model editor:**
to create the TA models
 2. **Property Verifier:**
to check whether a target state of model is reachable
 3. **Simulator:**
to display execution that reaches target state from initial state

Step 1: create model

example.xml

Editor Symbolic Simulator Concrete Simulator Verifier

Project
Declarations
sys
Declarations
System declarations

Name: sys Parameters:

Need to modify these items and draw model for each process in your system

$x \geq 1 \ \&\& \ y \geq 1$
 $y=0$

$x \leq 3$

$x \geq 2 \ \&\& \ y \leq 1$

A B

This file is on Canvas: **example.xml**

UPPAAL 5.0.0

Step 2: check reachability

example.xml

Editor | Symbolic Simulator | Concrete Simulator | Verifier

Overview

E<> (sys.B and sys.x>2)

Check

Insert above

Insert below

Remove

Comments

Clear results

Query

E<> (sys.B and sys.x>2)

Can the TA model reach a state in which the process named sys is in loc. "B" and clock "x" has a value exceeding 2?

Comment

If "options->diagnostic-trace->shortest" is set and property is satisfiable, then a simulation trace loads after checking, to show how the query condition is reached by the model

Status

Property is satisfied.

Step 3: view trace

example.xml

Editor Symbolic Simulator Concrete Simulator Verifier

Enabled Transitions

sys →
sys →

Reset Next

Simulation Trace

(A)
sys →
(A)
sys →
(B)

Prev Next Replay
Open Save Random

Slow Fast

Constraints
sys.x ∈ [2,3]
sys.y ∈ [0,1]

These are the constraints that UPPAAL infers for the transition shown at right

sys

$x \geq 1 \ \&\& \ y \geq 1$
 $y = 0$
 $x \leq 3$
 $x \geq 2 \ \&\& \ y \leq 1$

sys

Condition reached by jumping from A back to A and then to B

Step 3: view trace

example.xml

Editor Symbolic Simulator Concrete Simulator Verifier

Enabled Transitions

sys →
sys →

Reset Next

Simulation Trace

(A)
sys →
(A)
sys →
(B)

Prev Next Replay
Open Save Random

Slow Fast

Constraints
sys.x ∈ [2,3]
sys.y ∈ [0,1]

These are the constraints that UPPAAL infers for the transition shown at right

sys

$x \geq 1 \ \&\& \ y \geq 1$
 $y = 0$
 $x \leq 3$
 $x \geq 2 \ \&\& \ y \leq 1$

sys

Condition reached by jumping from A back to A and then to B

- Can also use random/concrete simulation instead of exhaustively checking reachability.
- This is like verilog testbench simulation in lab 1

Step 3: view trace (1st jump)

example.xml

Editor Symbolic Simulator Concrete Simulator Verifier

Enabled Transitions

sys →

Reset Next

Simulation Trace

(A)
sys →
(A)
sys →
(B)

Prev Next Replay
Open Save Random

Slow Fast

Constraints

sys.x ∈ [1,3]
sys.y ∈ [1,3]
sys.x = sys.y

sys

$x \geq 1 \ \&\& \ y \geq 1$
 $y = 0$
 $x \leq 3$

A B

sys

A A B

Step 3: view trace (1st jump)

example.xml

Editor Symbolic Simulator Concrete Simulator Verifier

Enabled Transitions

sys →

Reset Next

Simulation Trace

(A)
sys →
(A)
sys →
(B)

Prev Next Replay
Open Save Random

Slow Fast

Constraints

sys.x ∈ [1,3]
sys.y ∈ [1,3]
sys.x = sys.y

sys

$x \geq 1 \ \&\& \ y \geq 1$
 $y = 0$
 $x \leq 3$

$x \geq 2 \ \&\& \ y \leq 1$

A B

The graph shows the state space of the system. The x and y axes range from 0 to infinity, with grid lines at 1, 2, and 3. A solid gray line represents the constraint $x = y$. A dashed line represents the constraint $x = y + 1$. A red dot at (1, 1) is labeled A, and a blue dot at (3, 3) is labeled B. A red arrow points from A to B, and a blue arrow points from B to A. A purple arrow points from A to the right, and a green arrow points from B to the left.

Step 3: view trace (2nd jump)

example.xml

Editor Symbolic Simulator Concrete Simulator Verifier

Enabled Transitions

sys →
sys →

Reset Next

Simulation Trace

(A)
sys →
(A)
sys →
(B)

Prev Next Replay
Open Save Random

Slow Fast

Constraints

sys.x ∈ [2,3]
sys.y ∈ [0,1]

sys

$x \geq 1 \ \&\& \ y \geq 1$
 $y = 0$
 $x \leq 3$

$x \geq 2 \ \&\& \ y \leq 1$

sys

A
A
B

Step 3: view trace (2nd jump)

example.xml

Editor Symbolic Simulator Concrete Simulator Verifier

Enabled Transitions

sys →
sys →

Reset Next

Simulation Trace

(A)
sys →
(A)
sys →
(B)

Prev Next Replay
Open Save Random

Slow Fast

Constraints

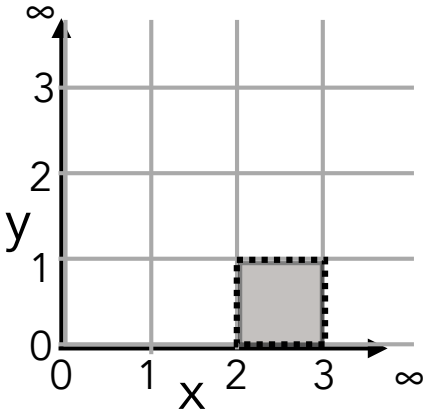
sys.x ∈ [2,3]
sys.y ∈ [0,1]

sys

$x \geq 1 \ \&\& \ y \geq 1$
 $y = 0$
 $x \leq 3$

A B

$x \geq 2 \ \&\& \ y \leq 1$



3/3/26

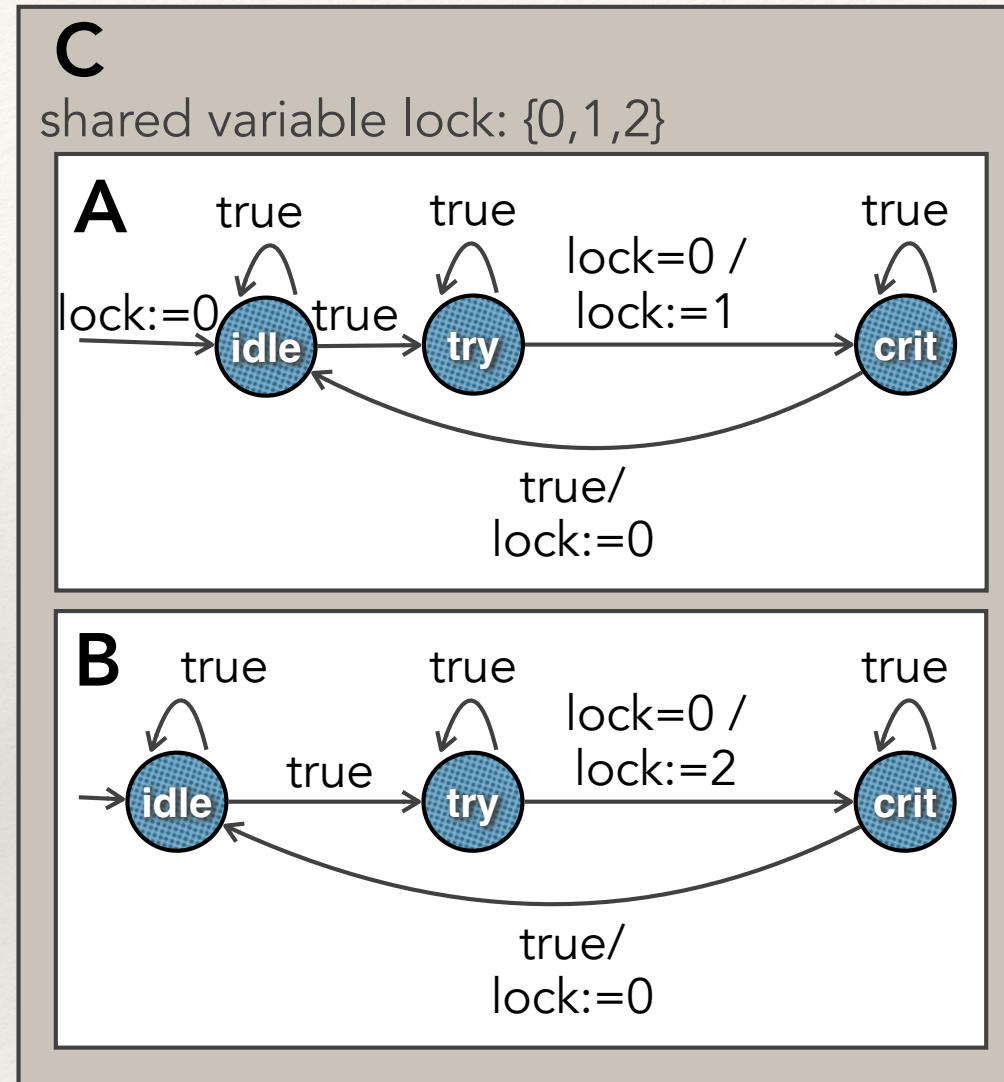
lecture 9

discuss:

- Lab 1 due 5pm on Weds 3/4
- Exam 1 on 3/12.
- Exam 1 coverage posted on Canvas
- PS2 and Lab 2 posted. Due on 3/10 and 3/24

Mutual Exclusion (will revisit in module 4)

- ❖ Discrete composition using shared variables
- ❖ Processes A and B both need exclusive access to a shared critical resource
 - ❖ In synchronous composition, model does not ensure mutual exclusion (both A and B can step into crit at same time).
 - ❖ In asynchronous with interleaving, it ensures mutual exclusion
 - ❖ But real systems can't check and set the value of lock instantaneously.
- ❖ How does this change if time involved?

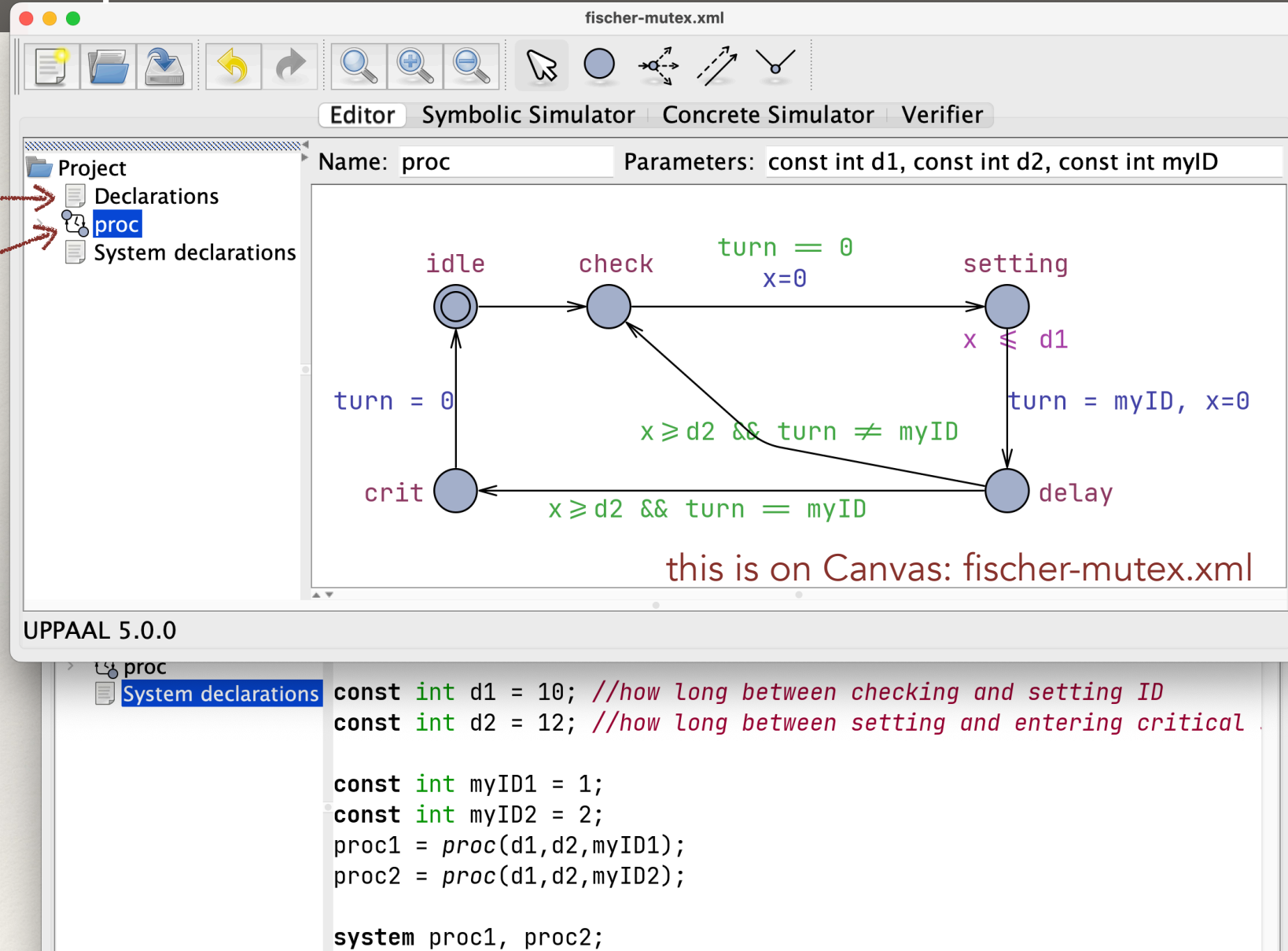


(Note: we have not covered composition before reaching this slide, so it is expected that some terms here will be unknown)

UPPAAL example: Fischer's time-based mutex

turn is a shared
global variable
declared here

x is a **clock**
belonging to each
proc. declared here



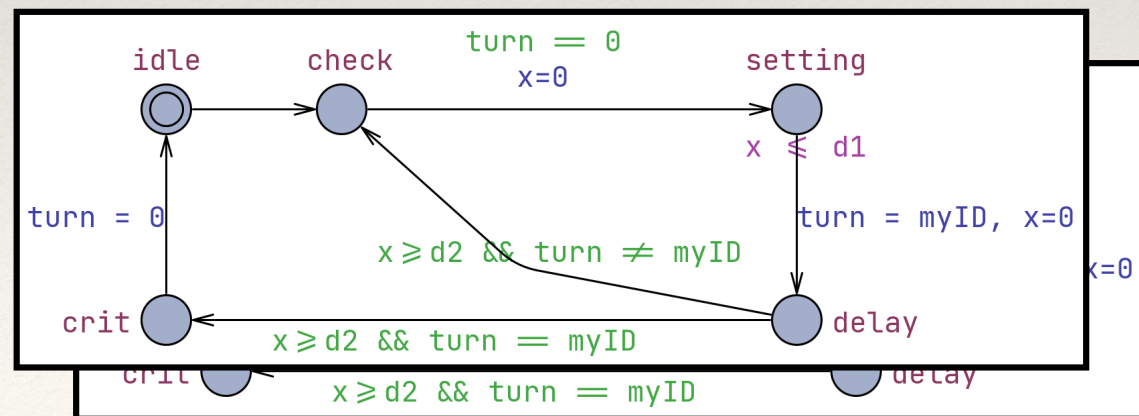
- ❖ Can proc1 and proc 2 be in the critical location at the same time?
- ❖ Under what values of d1 and d2 can this occur? Can check with UPPAAL

Explanation of Fischer's mutex

- ❖ Checking and setting shared variable **turn** is not instantaneous.
 - ❖ **Up to d1** time can elapse between a process checking and setting turn
 - ❖ Conflicts possible from multiple processes checking, and then setting, turn
- ❖ Fischer's mutex protocol tries to disallow two processes in critical location:
 - ❖ Each process must wait **at least d2** time after setting **turn** to their own ID
 - ❖ Each process rechecks **turn** before entering critical location,
to see whether another process has changed it
 - ❖ If **turn** has not been changed, proceed to critical location
 - ❖ If **turn** has been changed, abort attempt to enter critical location

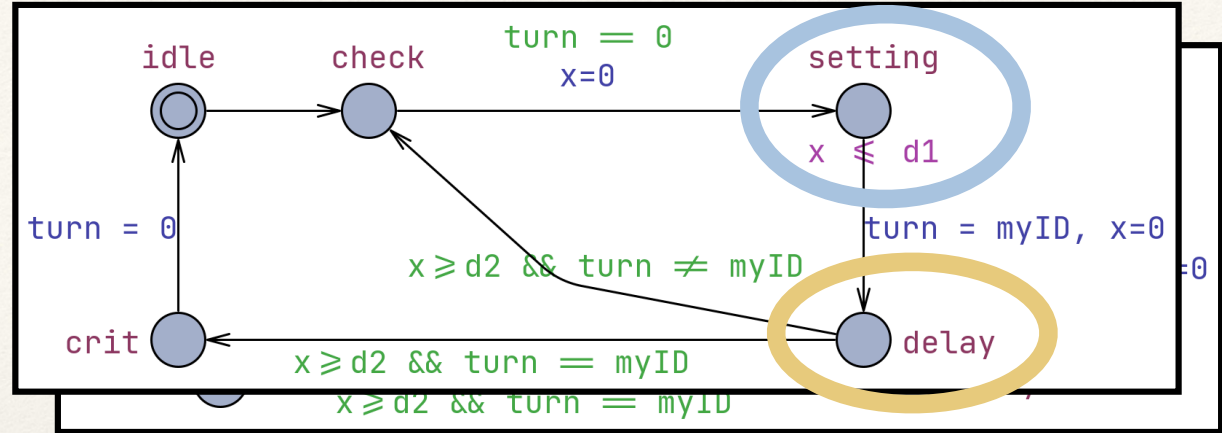
Questions:

1. What is state space?
2. What values of **d1** and **d2** make this protocol safe?
3. What reachability query to check?



On Board: Fischer's Mutex Example

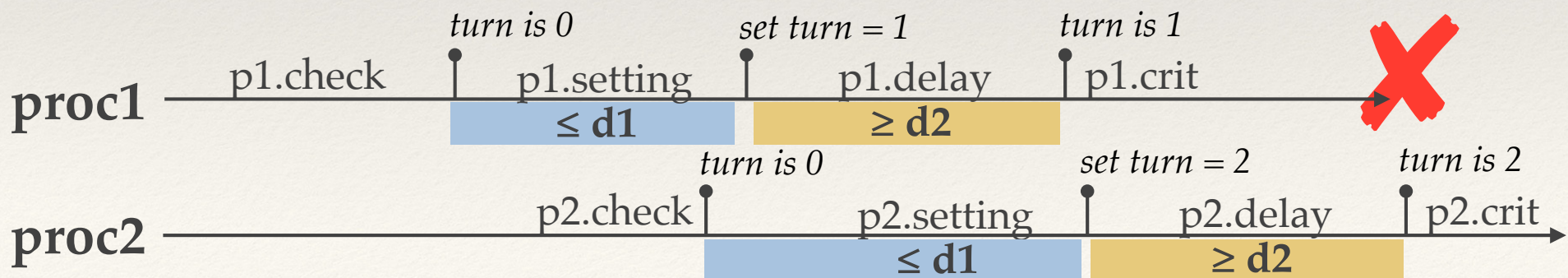
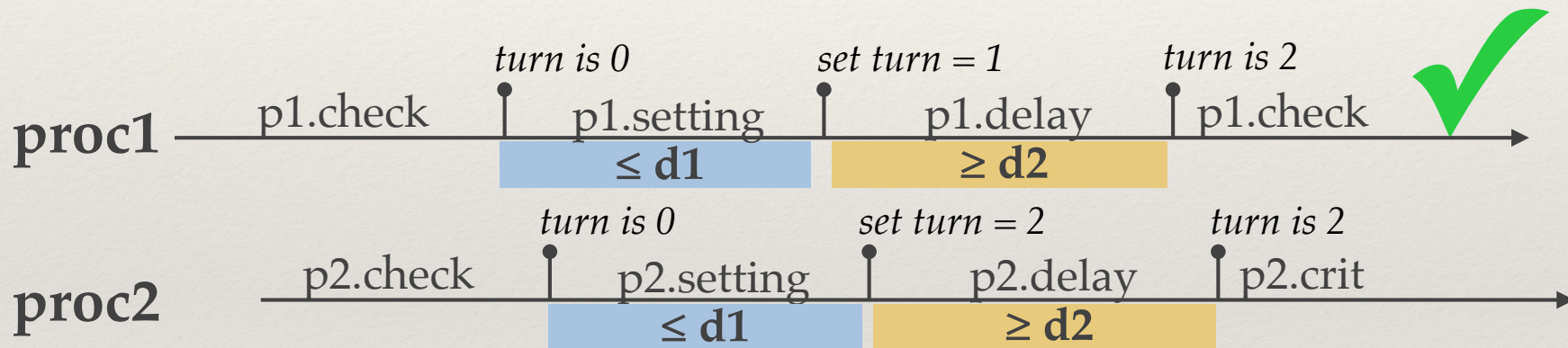
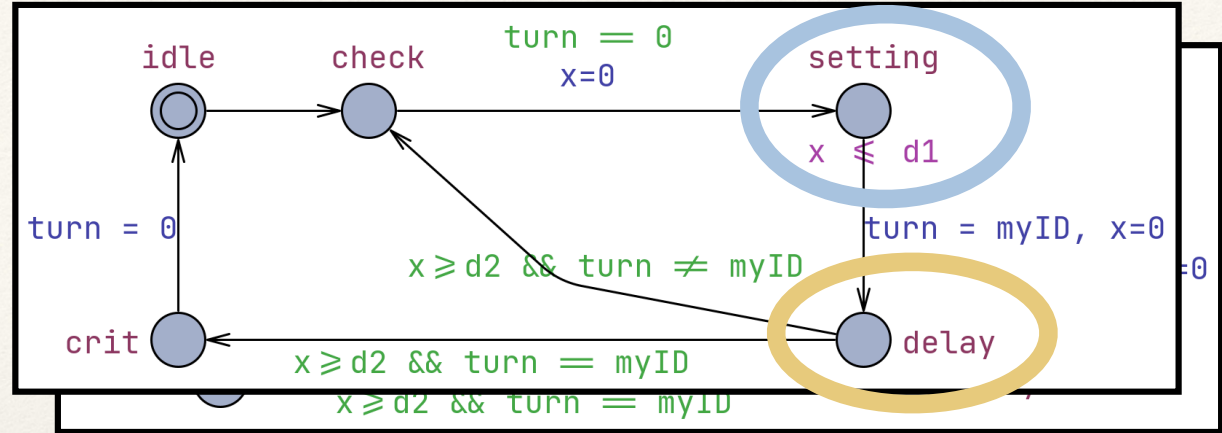
What values of **d1** and **d2** make this protocol safe?



On Board: Fischer's Mutex Example

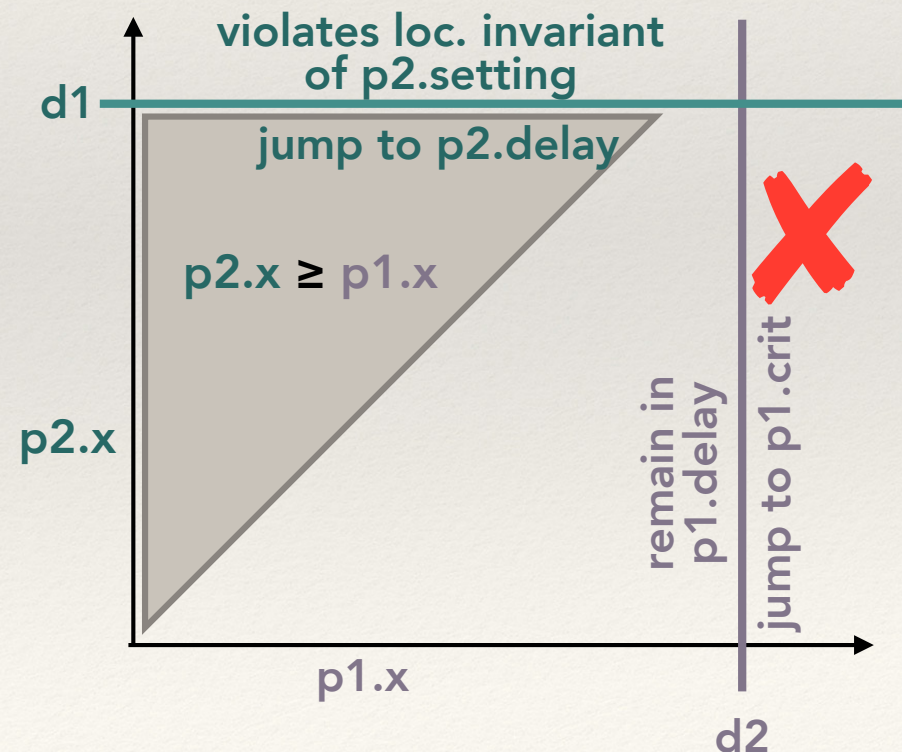
What values of **d1** and **d2**
make this protocol safe?

Answer: **d2 > d1**



On Board: Fischer's Mutex Analysis

- ❖ Plot shows the reachable clock regions for the combined location **(p1.delay, p2.setting)** from the model in previous slide
- ❖ The value of $p2.x$ can never be less than $p1.x$ in this location because **p2.setting** was reached before **p1.delay** (and each of those jumps reset the clock variable of their process)
- ❖ Therefore, if $d2 > d1$, the next move has to be the jump to **p2.delay** to avoid violating the location invariant on **p2.setting**



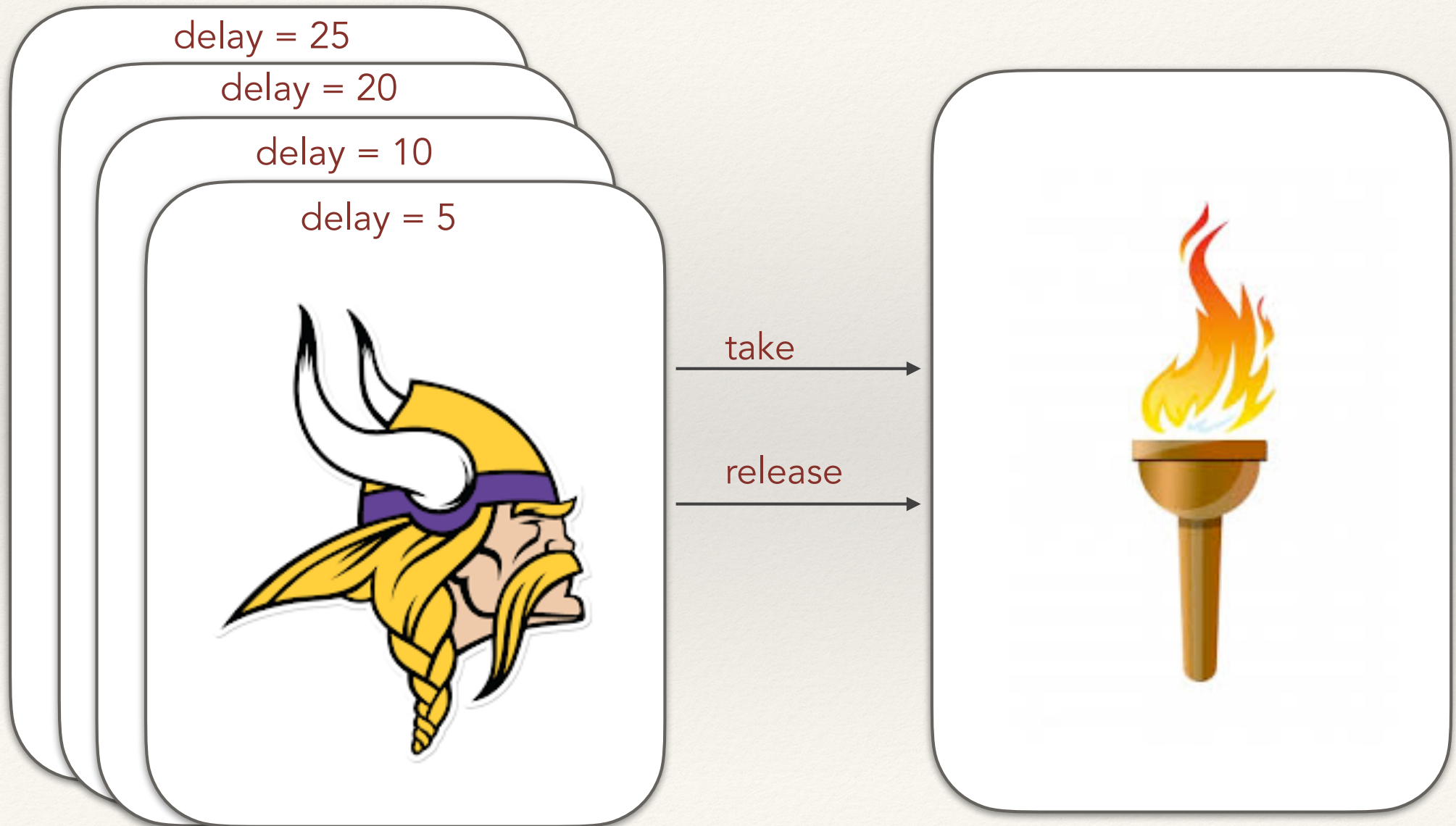
Lab 2, Part A: Vikings

Four Vikings need to cross a damaged bridge in the middle of the night. The bridge can only carry two Vikings at a time and to find their way over the bridge (in either direction) the Vikings need to bring a torch. The Vikings have different max speeds, and need 5, 10, 20 and 25 minutes respectively to cross the bridge in each direction.

- ❖ Parameterized model of viking, instantiated 4x
- ❖ Torch model, instantiated 1x
- ❖ Interaction between viking and torch using **take** and **release** torch events over communication channels
- ❖ Note: the **take!** event in viking model and **take?** event in torch model are synchronized to happen together

Lab 2: Complete System

- ❖ Model is a composition of vikings and torch
- ❖ Interaction through take and release signals

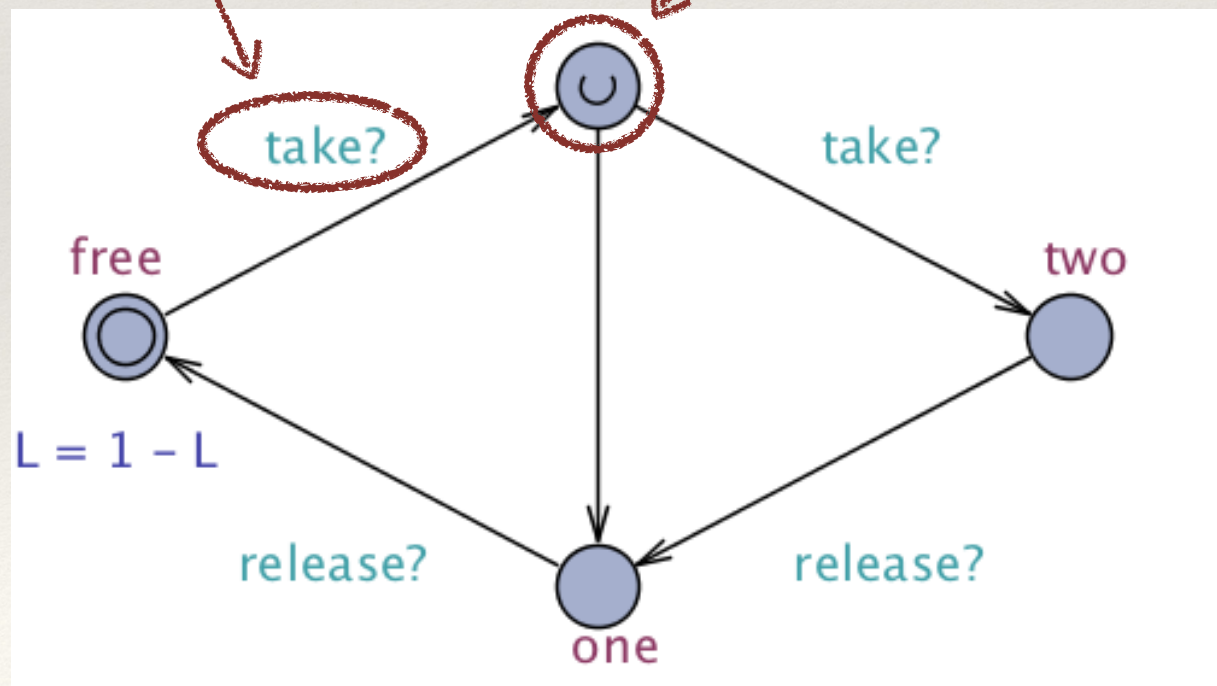


Lab 2: Model of Torch

- ❖ **Take** and **release** are synchronized to vikings
- ❖ **L** is a variable
 - ❖ $L == 0$ means torch on near side of bridge
 - ❖ $L == 1$ means torch is across bridge

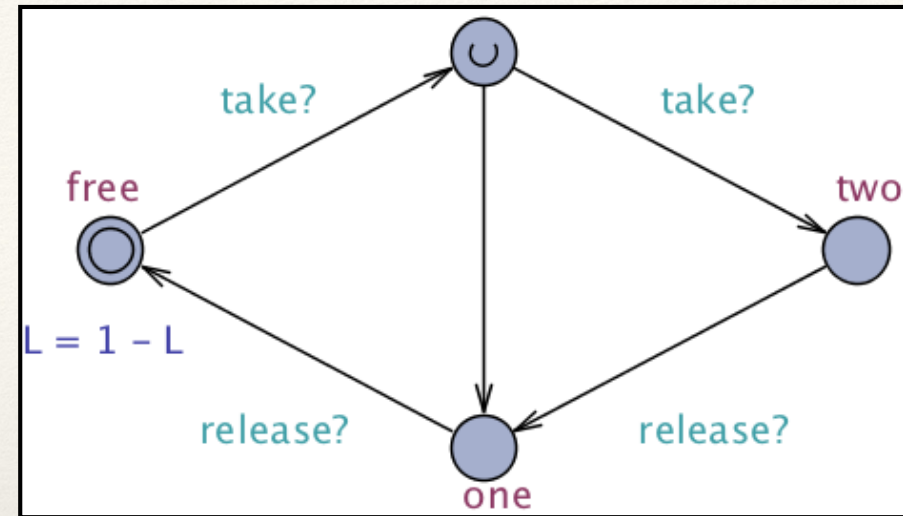
sync to viking
(**take?** of torch must happen
together with **take!** of viking)

u symbol means "urgent loc."
in which time cannot elapse
(must leave state immediately)

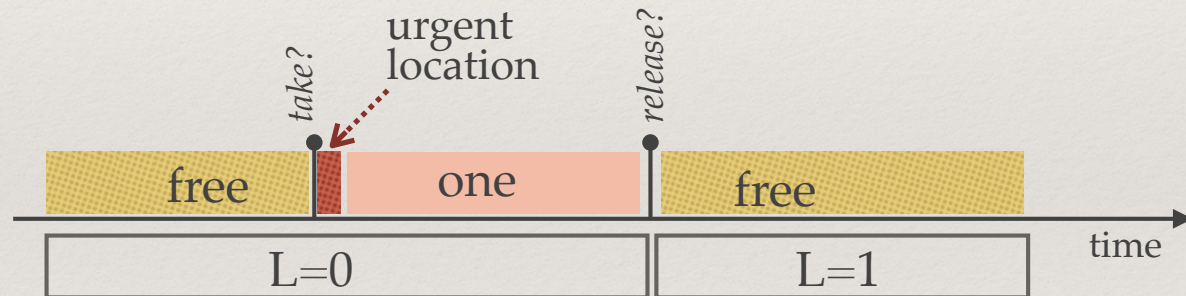


(Example on board: just torch)

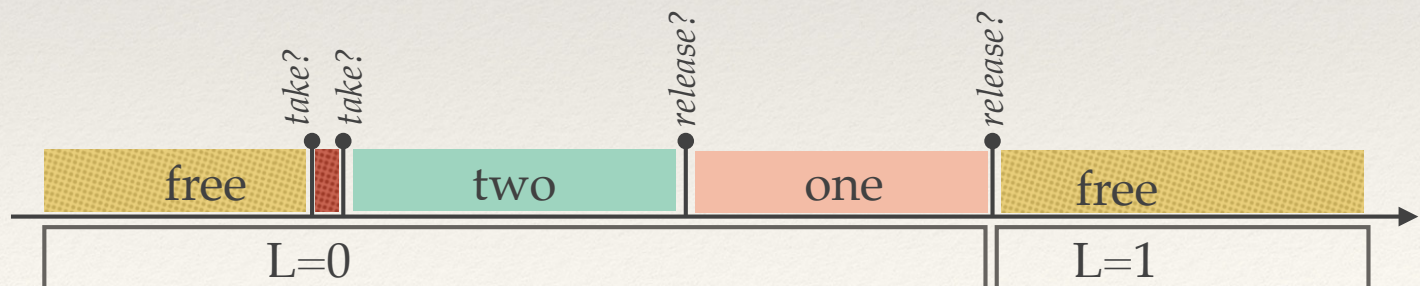
- ❖ Possible executions of torch model when torch crosses bridge once and becomes free on the far (safe) side of bridge.
- ❖ If two “takes” occur, there cannot be any time between them, due to urgent location.
- ❖ Note that torch imposes no restriction on timing between take and release.
Timing comes from the viking models.



Torch carried across
by one viking:

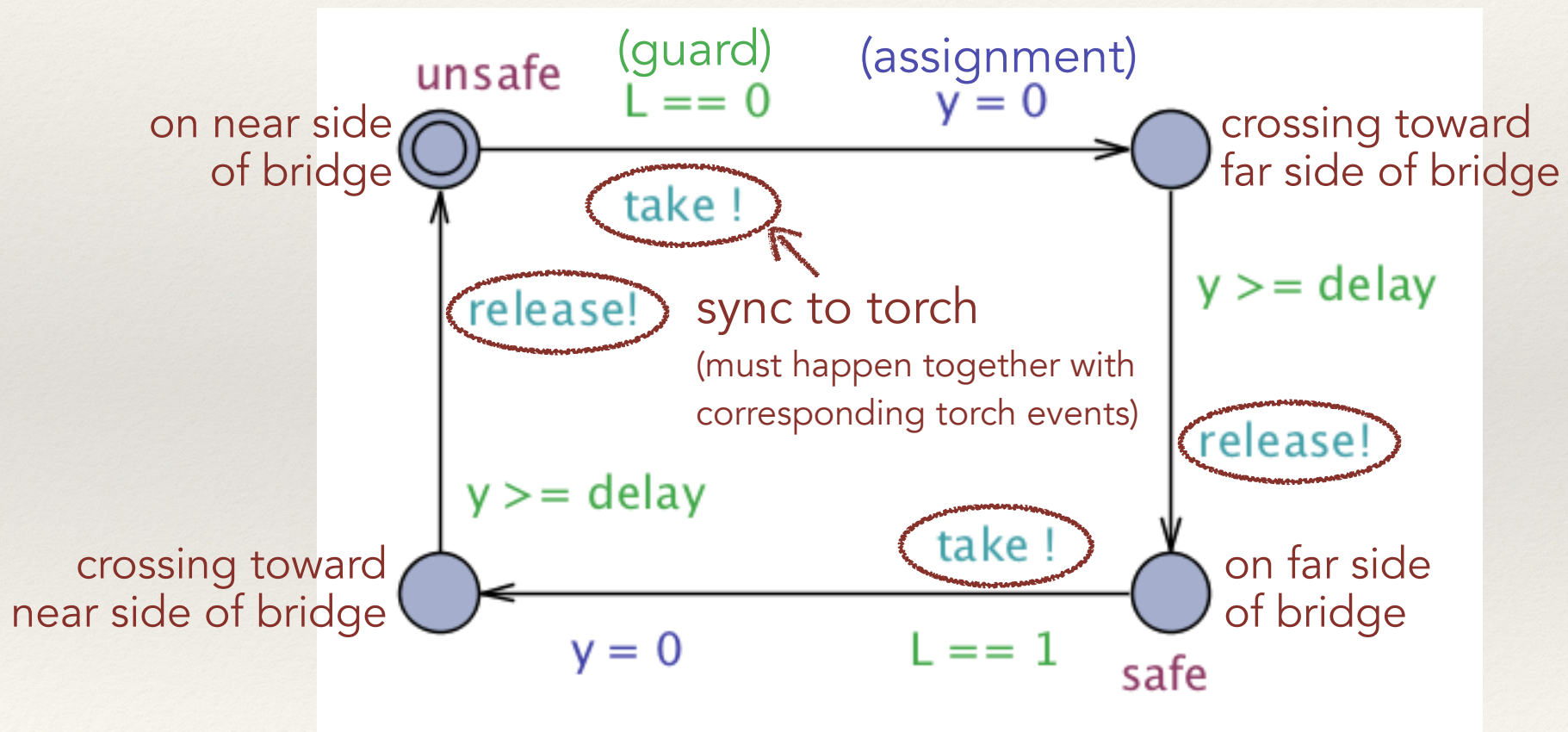


Torch carried across
by two vikings:

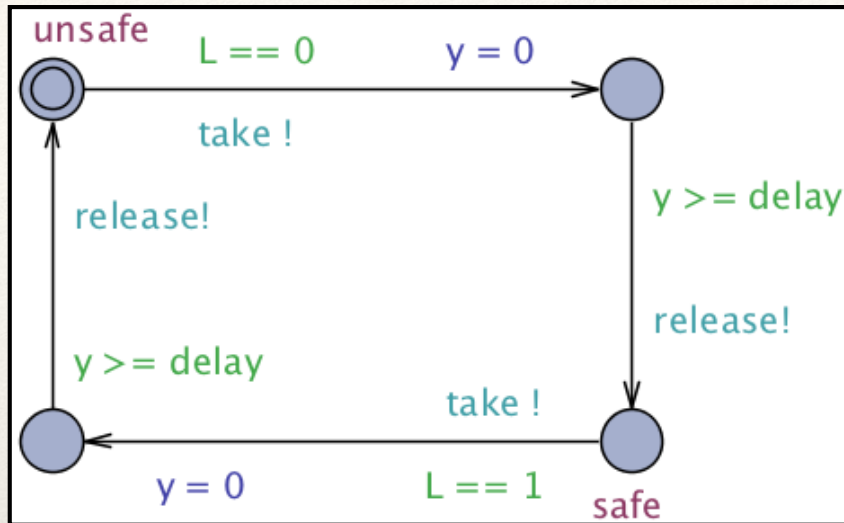


Lab 2: Model of a Viking

- ❖ **y** is local clock variable (unique to this viking)
- ❖ **delay** is parameter of viking model, indicates time to cross bridge

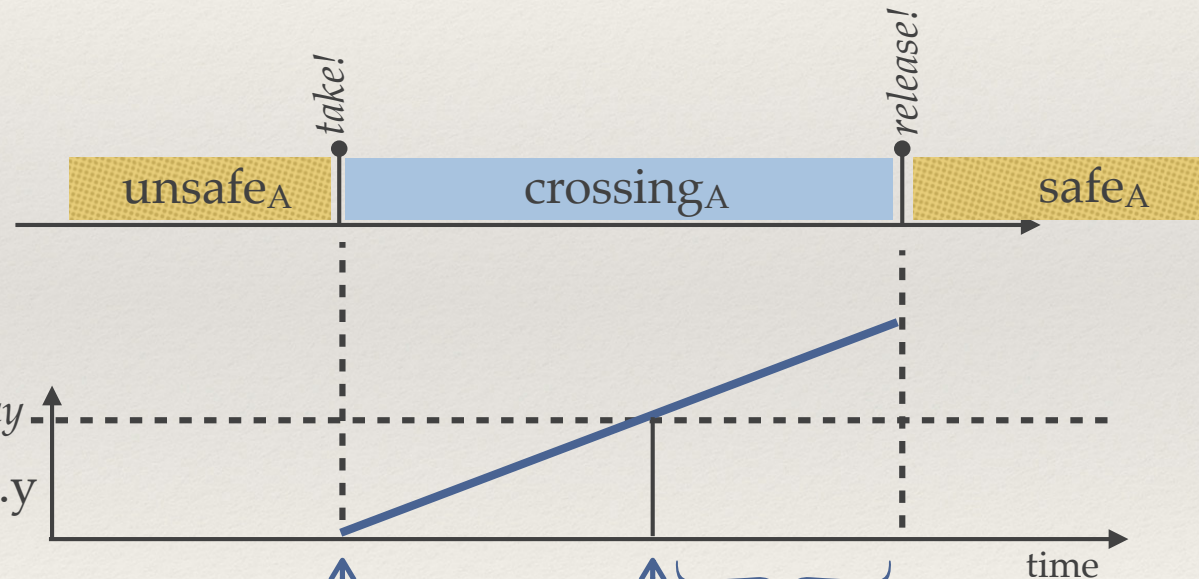


(Example on board: just viking)



- ❖ Possible execution of viking "A" crossing bridge alone, neglecting torch
- ❖ The guard condition on clock y enforces minimum crossing time
- ❖ In other words, enforces a minimum delay between **take** and **release** events

location of viking A:

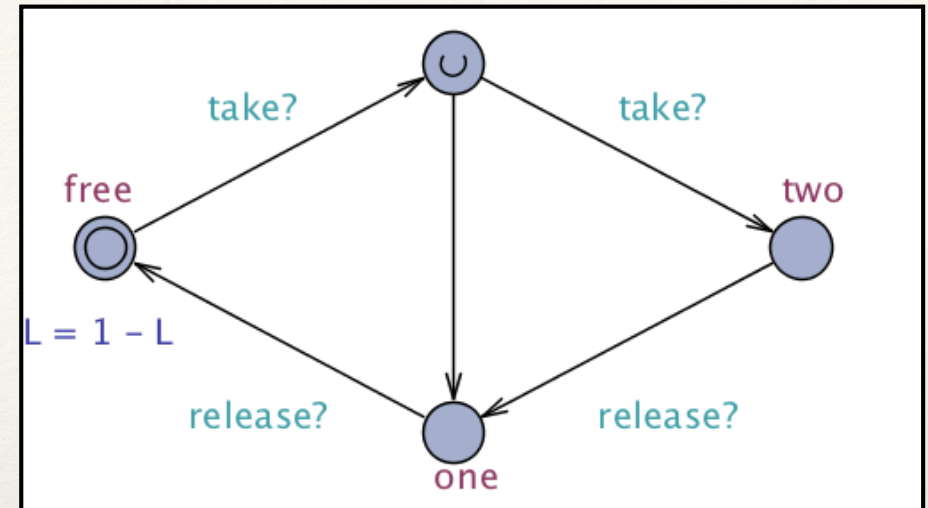
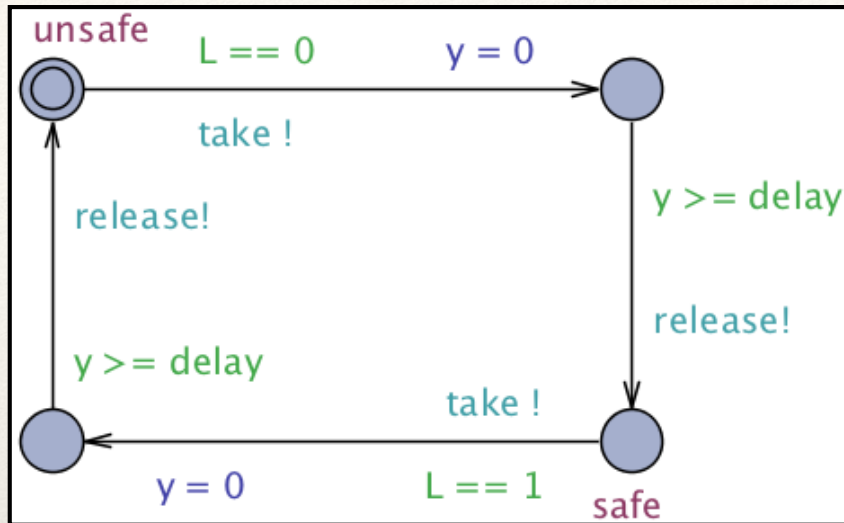


clock y of viking A:

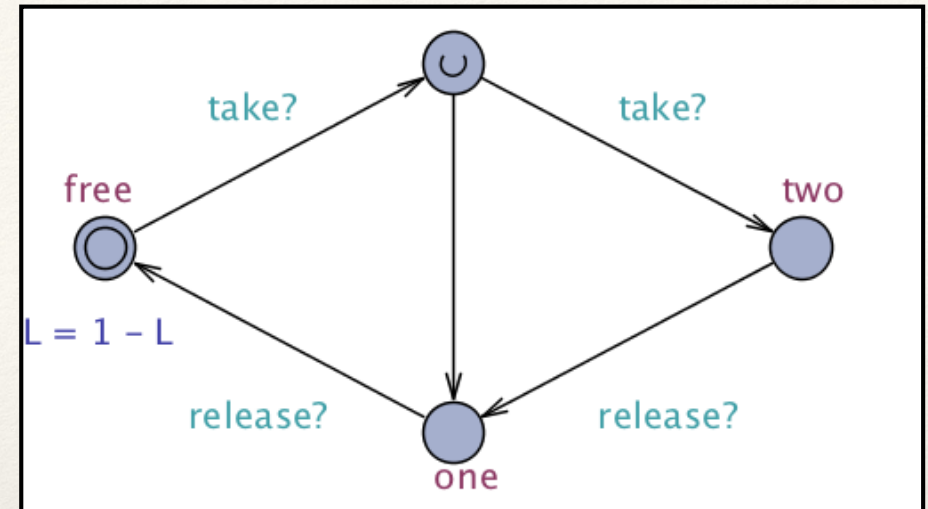
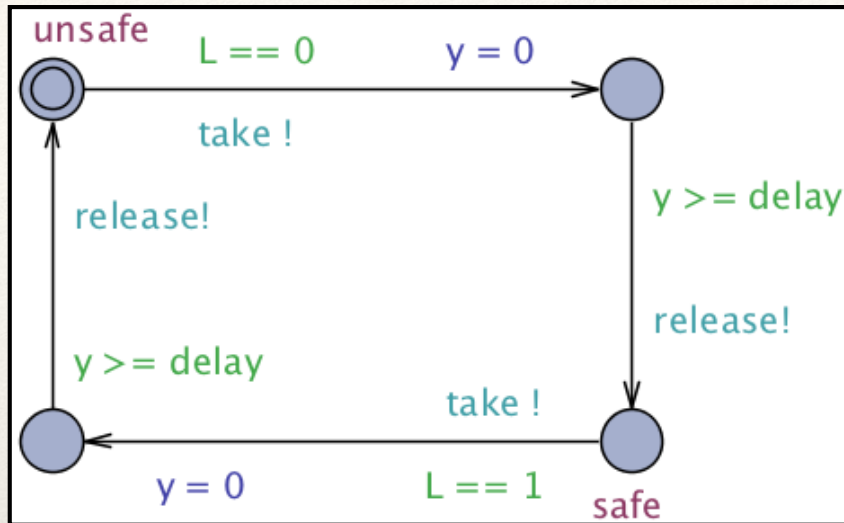
clock y gets set to 0 when entering crossing location

clock y exceeds delay param, model can change location now

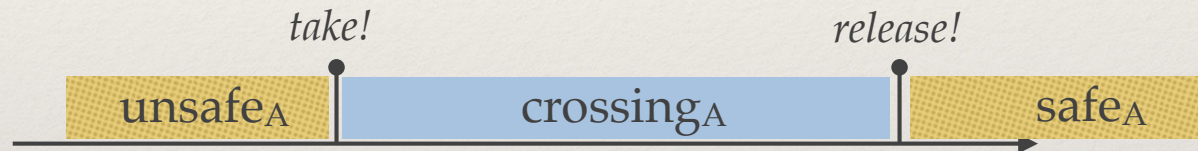
(Example on board: vikings cross with torch)



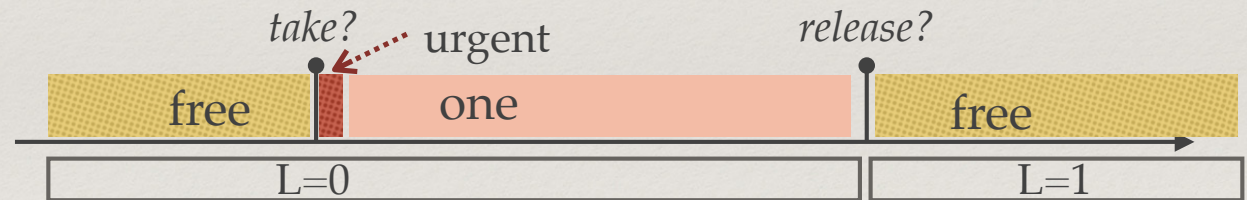
(Example on board: 1 viking crosses with torch)



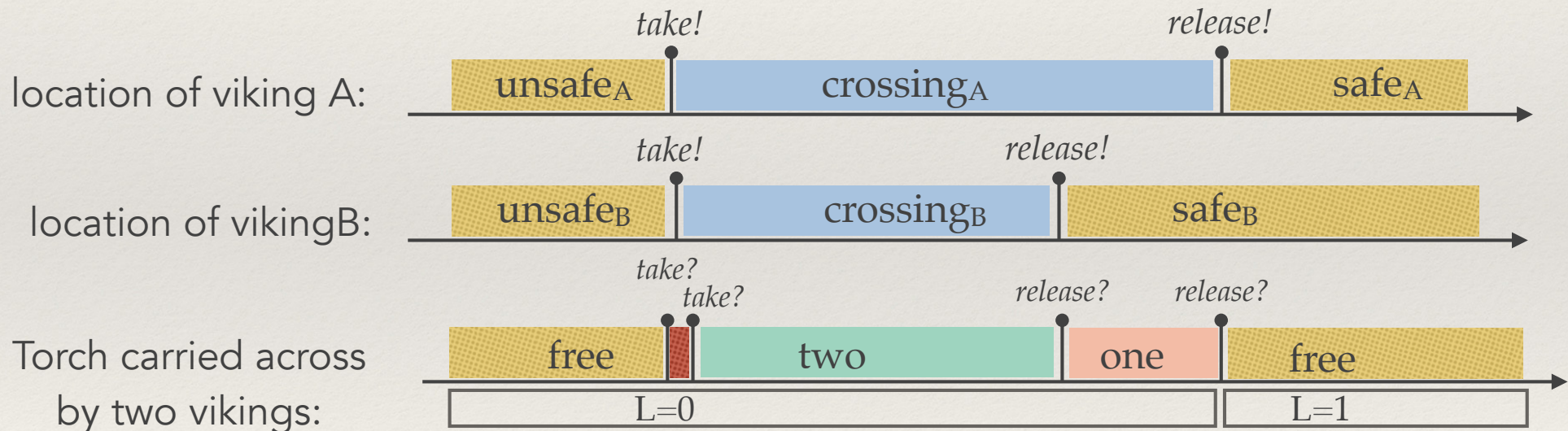
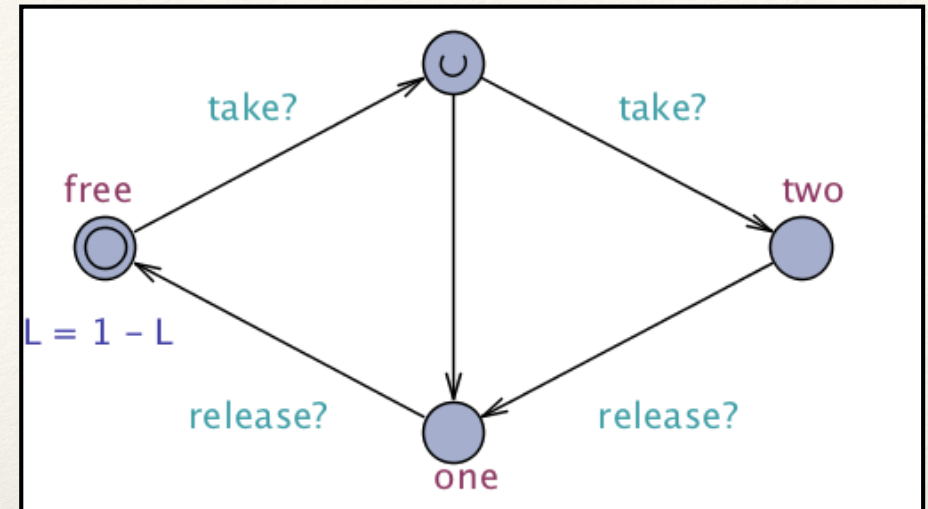
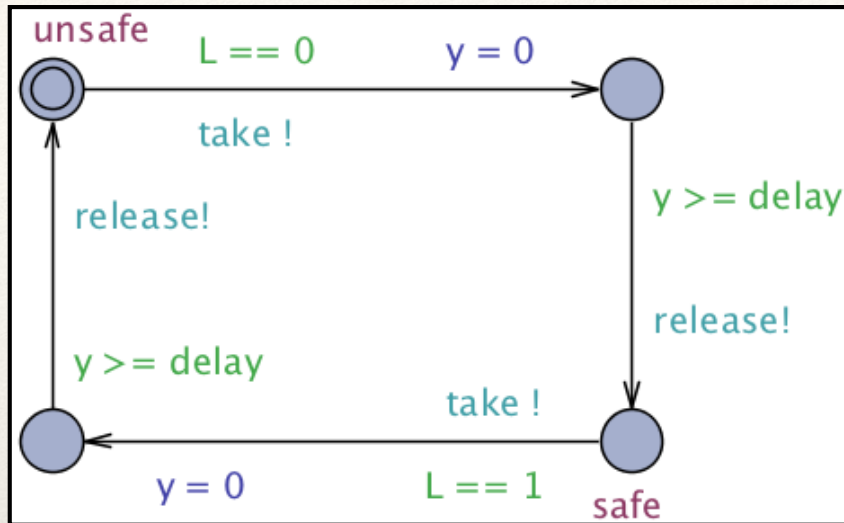
location of viking A:



Torch carried across
by one viking:



(Example on board: 2 vikings cross with torch)



3/5/26

lecture 10

discuss:

- Lab 1
- Exam 1 on 3/12. Coverage is on Canvas
- Exam 1 office hours
- Deadlock question in UPPAAL
- PS2 and Lab 2. Due on 3/10 and 3/24

example.xml



Editor Symbolic Simulator Concrete Simulator Verifier

Enabled Transitions

sys →

Reset Next

Simulation Trace

(A)

sys →

(A)

sys →

(B)

Prev Next Replay

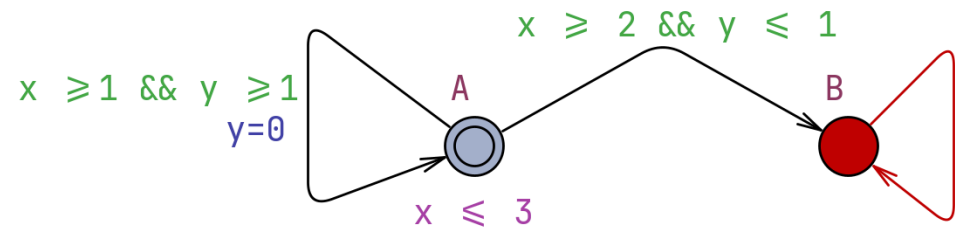
Open Save Random

Slow Fast

Constraints

sys.x ≥ 2
sys.y ≥ 0

sys



sys



example.xml



Editor Symbolic Simulator Concrete Simulator Verifier

Enabled Transitions

sys →

Reset Next

Simulation Trace

(A)

sys →

(A)

sys →

(B)

Prev Next Replay

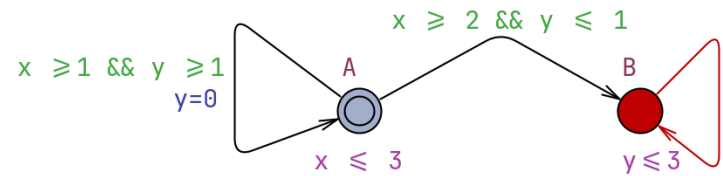
Open Save Random

Slow Fast

Constraints

sys.x ∈ [2,6]
sys.y ∈ [0,3]

sys



sys



example.xml



Editor Symbolic Simulator Concrete Simulator Verifier

Enabled Transitions

deadlock

Reset Next

Simulation Trace

(A)
sys →
(A)
sys →
(B)

Prev Next Replay

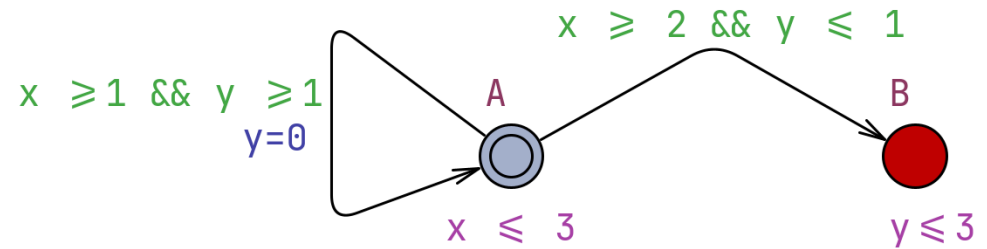
Open Save Random

Slow Fast

Constraints

$\text{sys.x} \in [2,6]$
 $\text{sys.y} \in [0,3]$

sys



sys



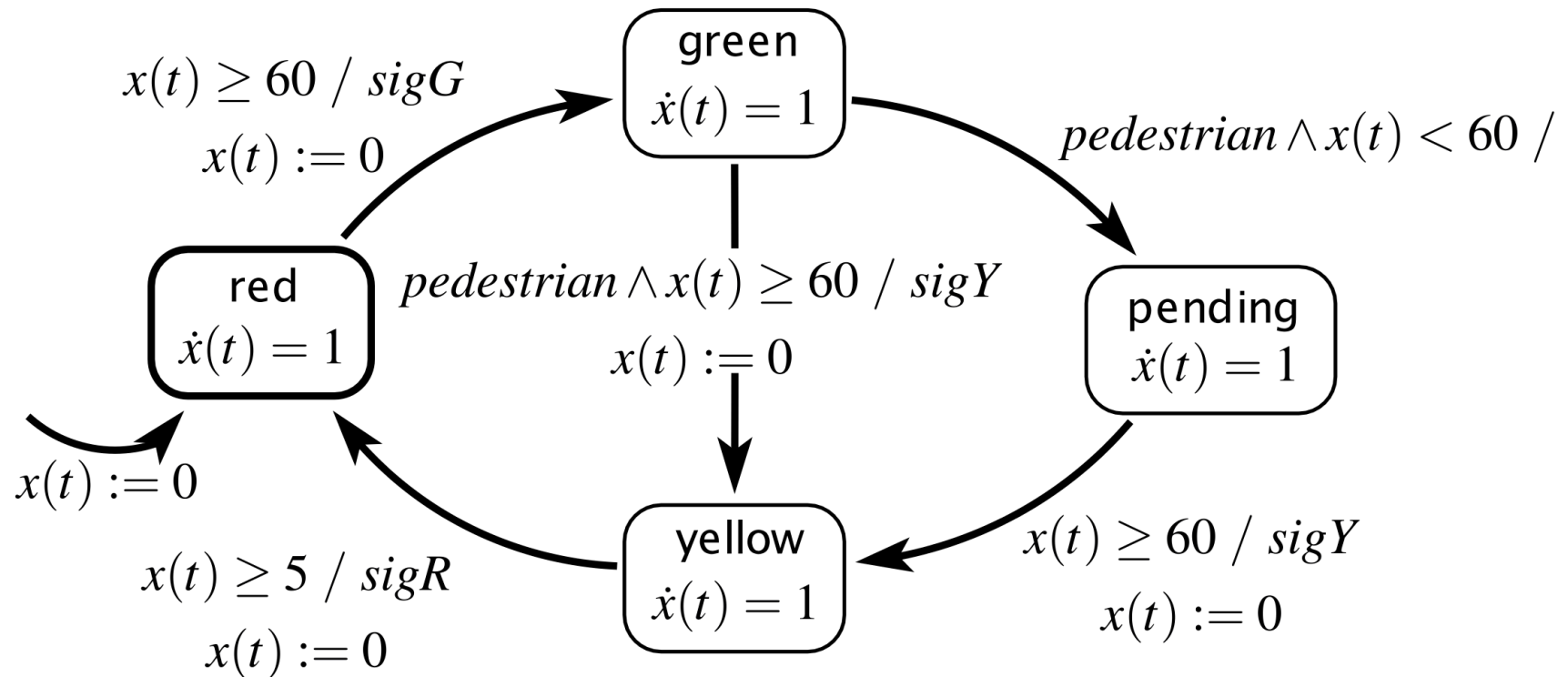
Lab 2: Traffic Light Controller

- ❖ Model of traffic light controller
- ❖ Contrast this model against extended FSM

continuous variable: $x(t): \mathbb{R}$

inputs: *pedestrian*: pure

outputs: *sigR*, *sigG*, *sigY*: pure



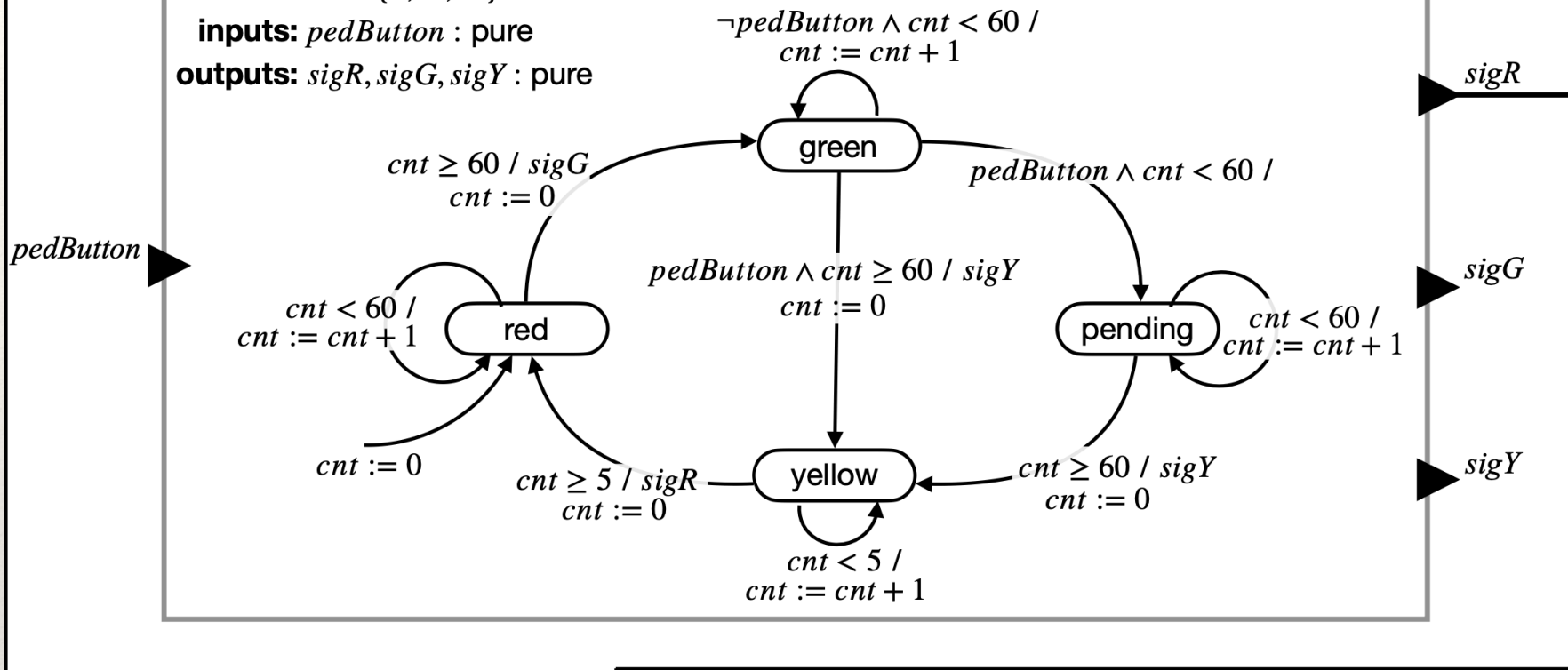
Lab 2: Traffic Light Controller

traffic light

variable: $cnt : \{0, \dots, 60\}$

inputs: $pedButton : \text{pure}$

outputs: $sigR, sigG, sigY : \text{pure}$

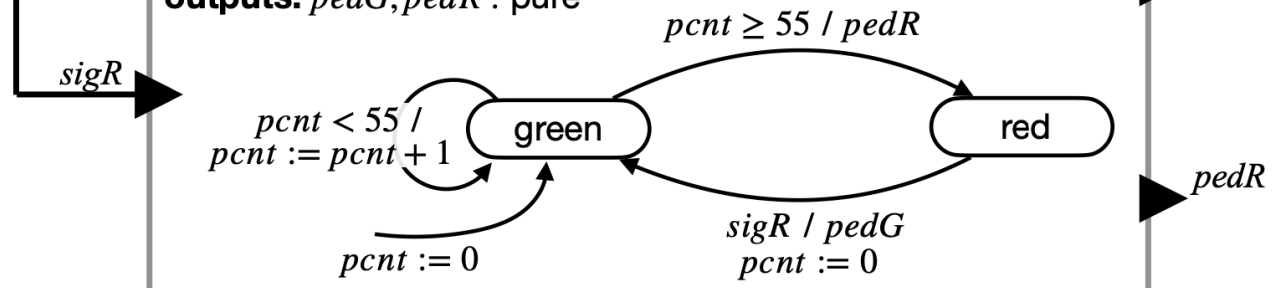


crosswalk light

variable: $pcnt : \{0, \dots, 55\}$

inputs: $sigR : \text{pure}$

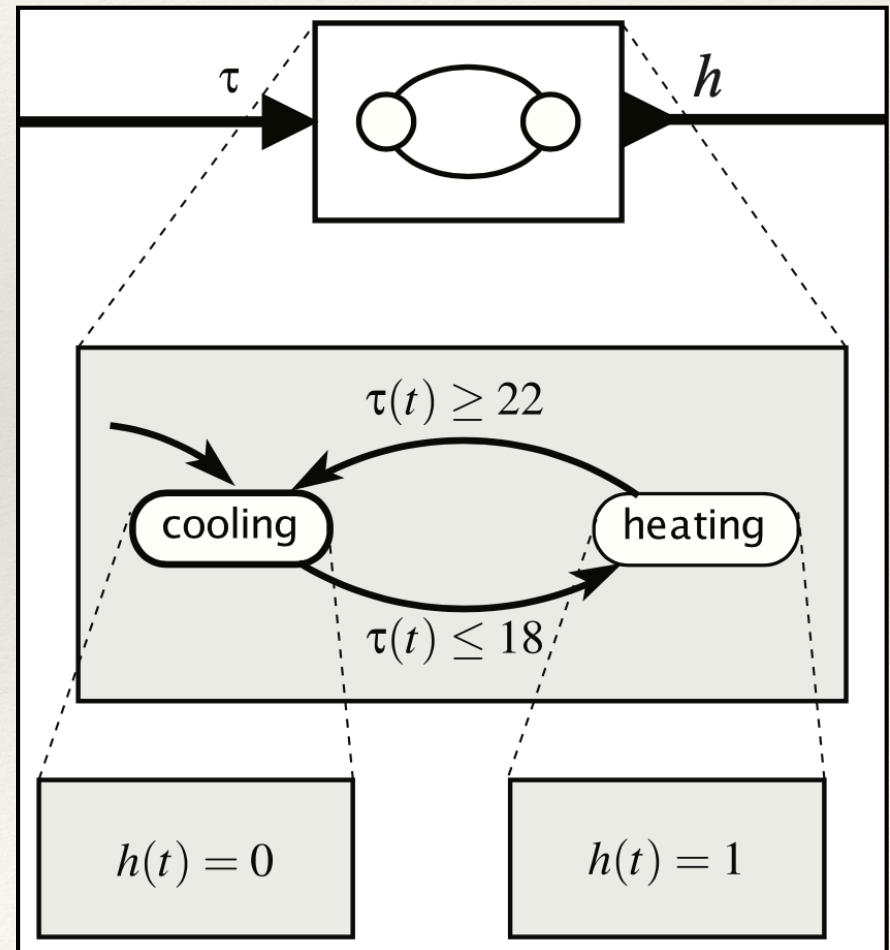
outputs: $pedG, pedR : \text{pure}$



Changing from Timed
automata to hybrid
automata...

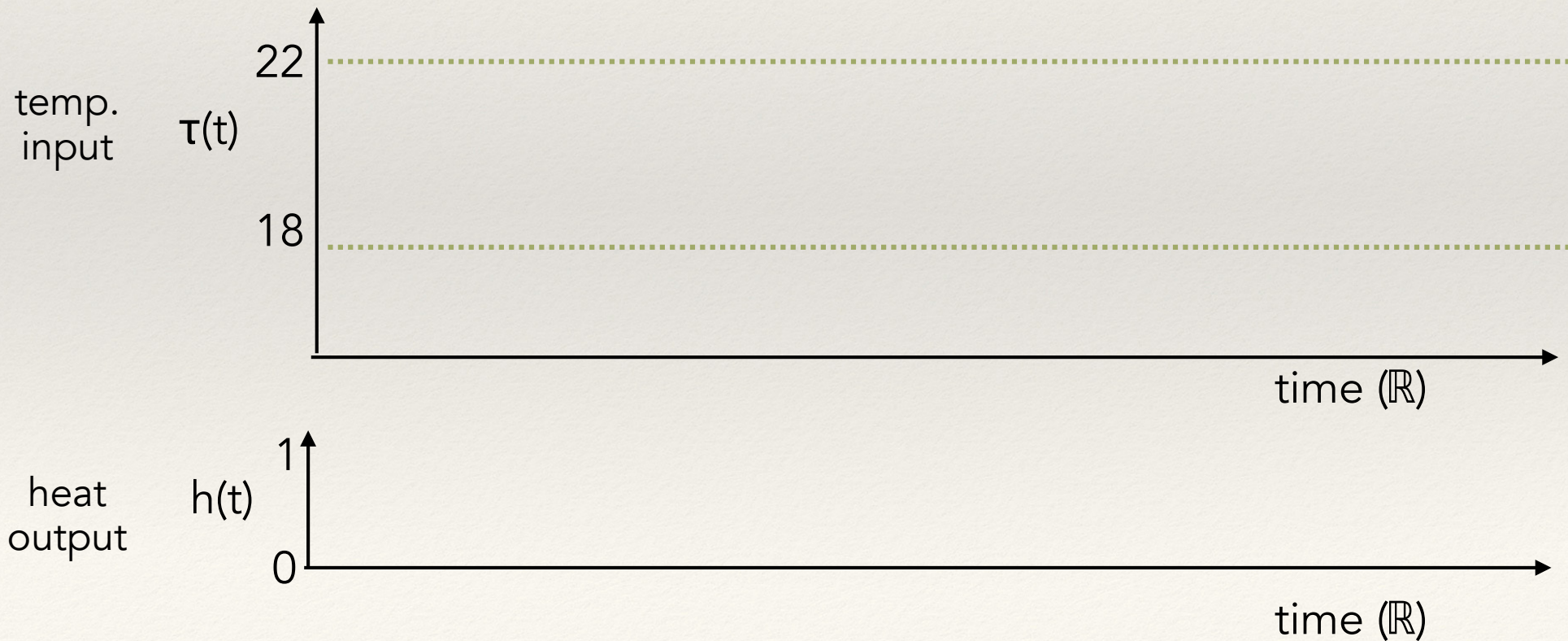
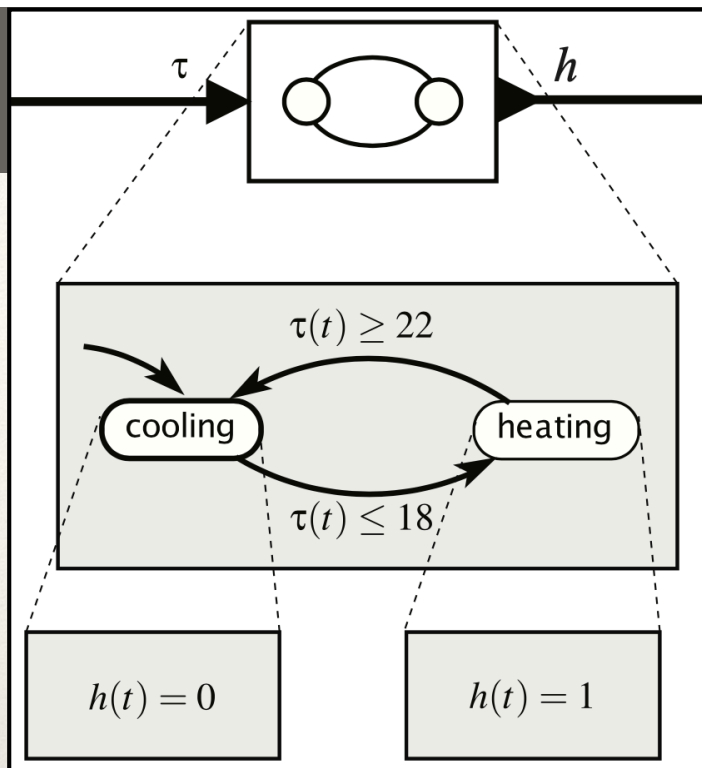
Thermostat as Simple 2-state Model

- ❖ Thermostat trying to keep temp. at around 20 degrees
- ❖ τ is a continuous time input representing temperature
- ❖ h is a cont. time output. Values 0/1 represent heat off/on
- ❖ Switching rules:
 - ❖ Heat can turn on when temperature is below 18
 - ❖ Heat can turn off when temperature is above 22



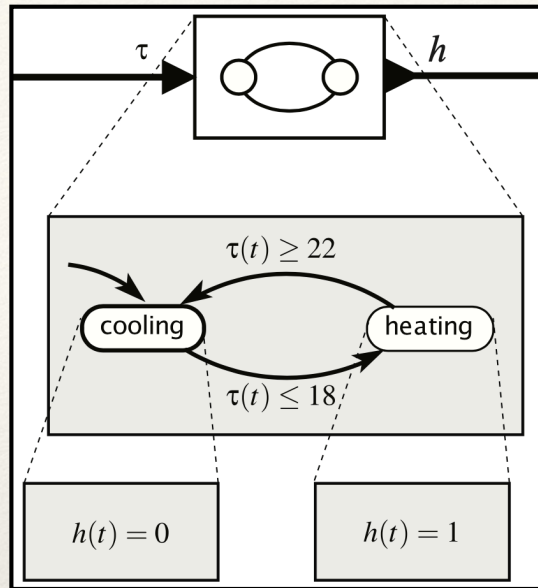
(Example for Board)

plot behavior of
this thermostat
model

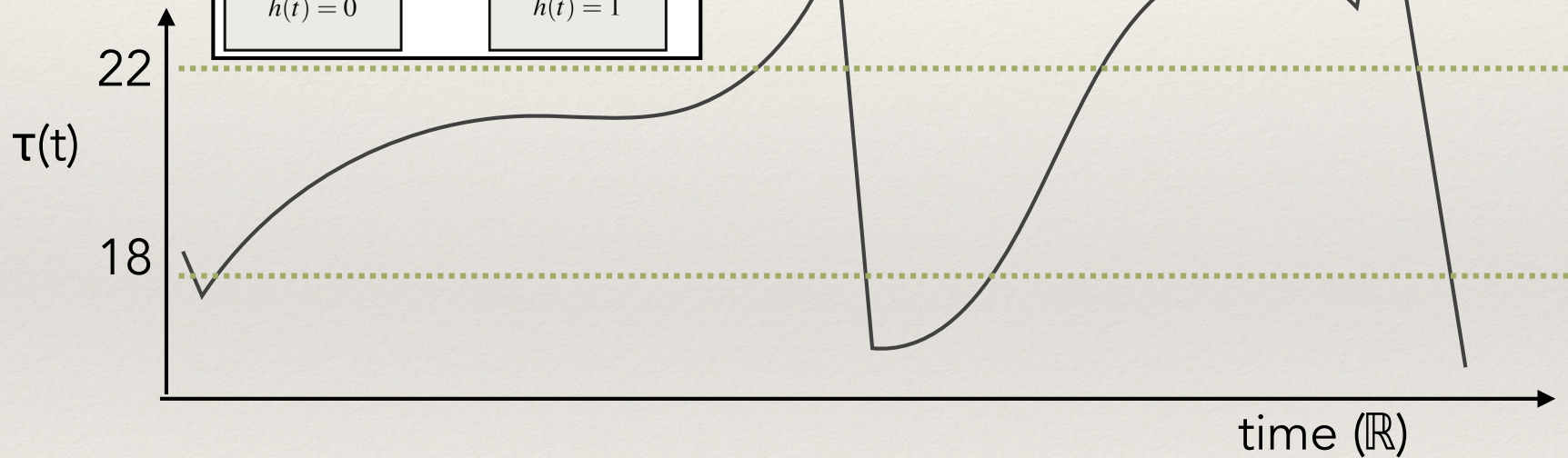


(Example on Board)

plot behavior of
this thermostat
model



temp.
input

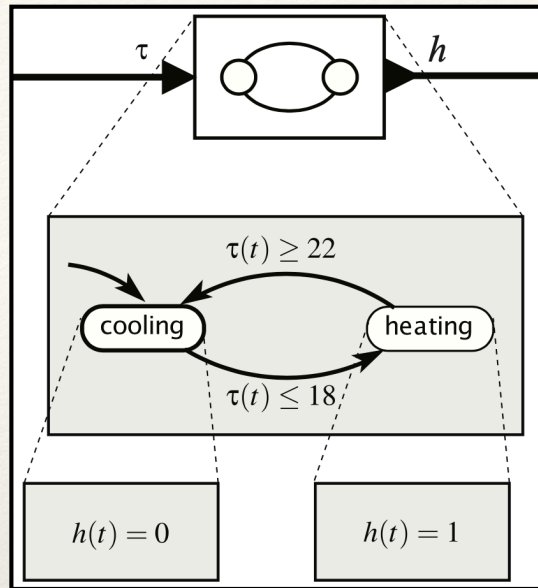


heat
output

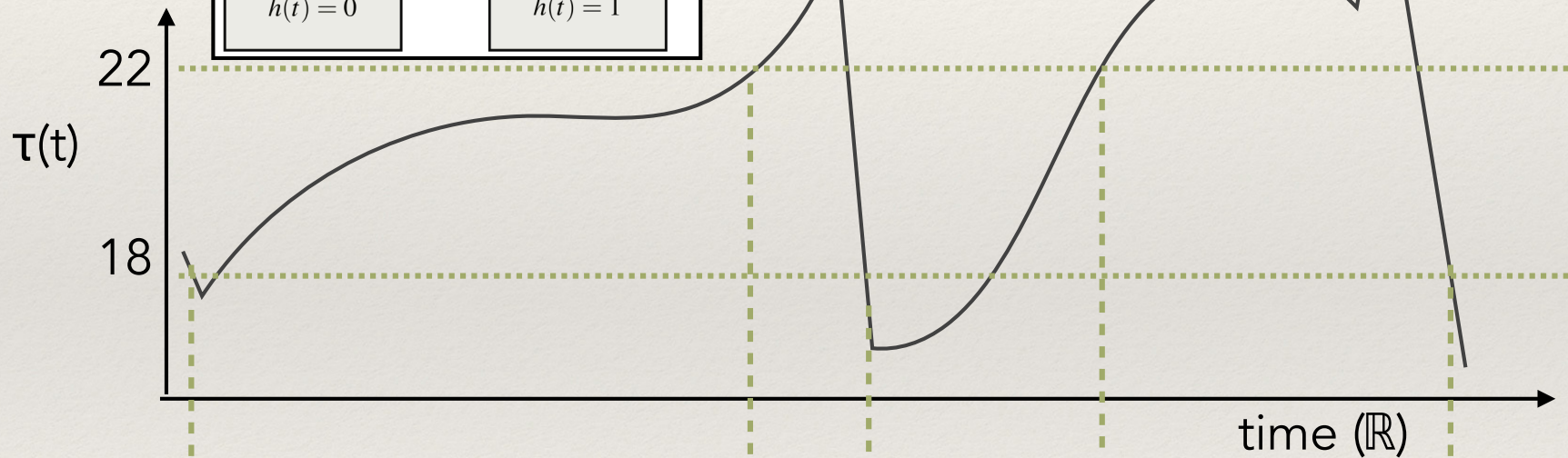


(Example on Board)

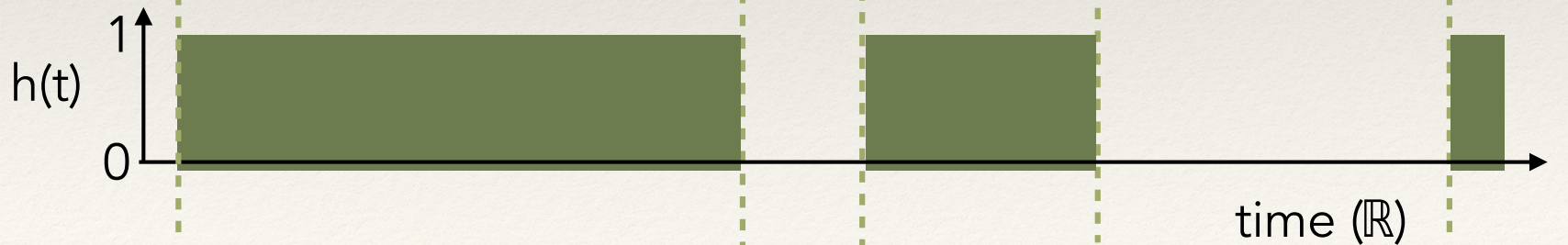
plot behavior of
this thermostat
model



temp.
input

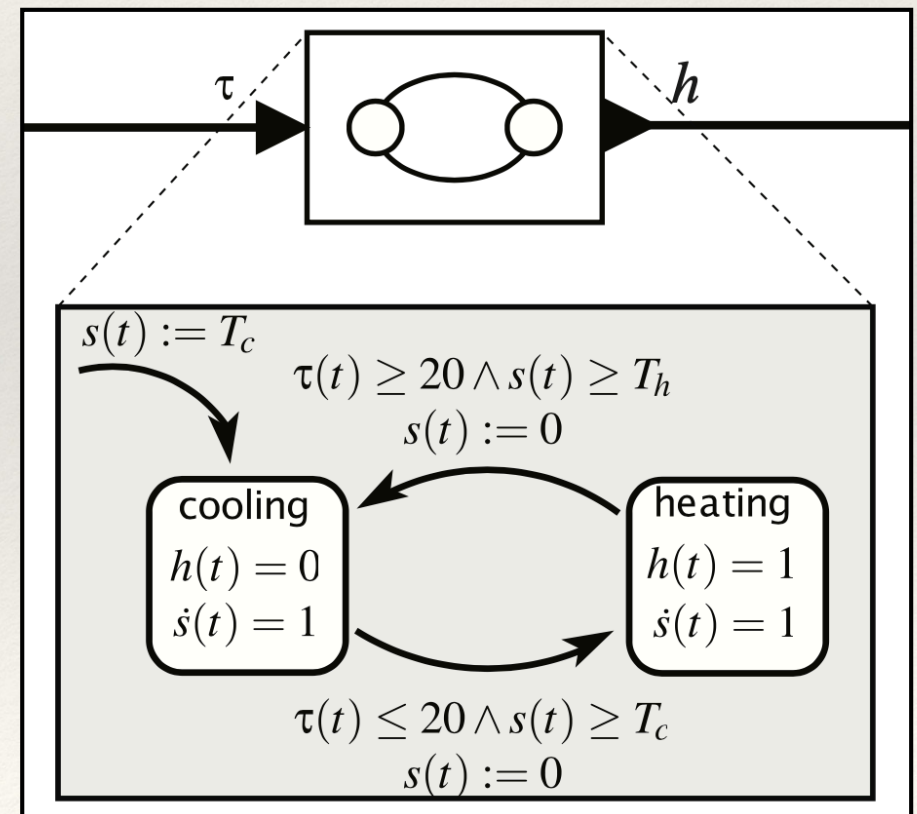


heat
output



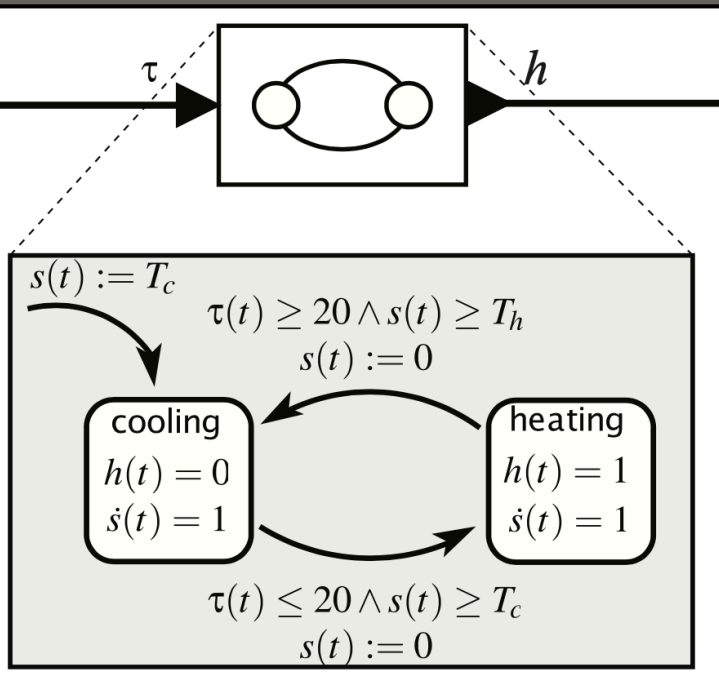
Thermostat as Timed Automaton

- ❖ Thermostat trying to keep temperature at around 20 degrees
 - ❖ τ is a continuous time input representing temperature
 - ❖ s is a clock variable
 - ❖ h is a cont. time output. Values 0/1 represent heat off/on
- ❖ Switching rules:
 - ❖ Heat can turn on when temperature is below 20 **and** system has been in cooling location for at least duration T_c
 - ❖ Heat can turn off when temperature is above 20 **and** system has been in heating loc. for at least duration T_h



(Example for Board)

behavior of
Timed
Automaton
thermostat
model



temp.
input

$\tau(t)$

20

time ($\mathbb{R}$)

clock

$s(t)$

0

heat
output

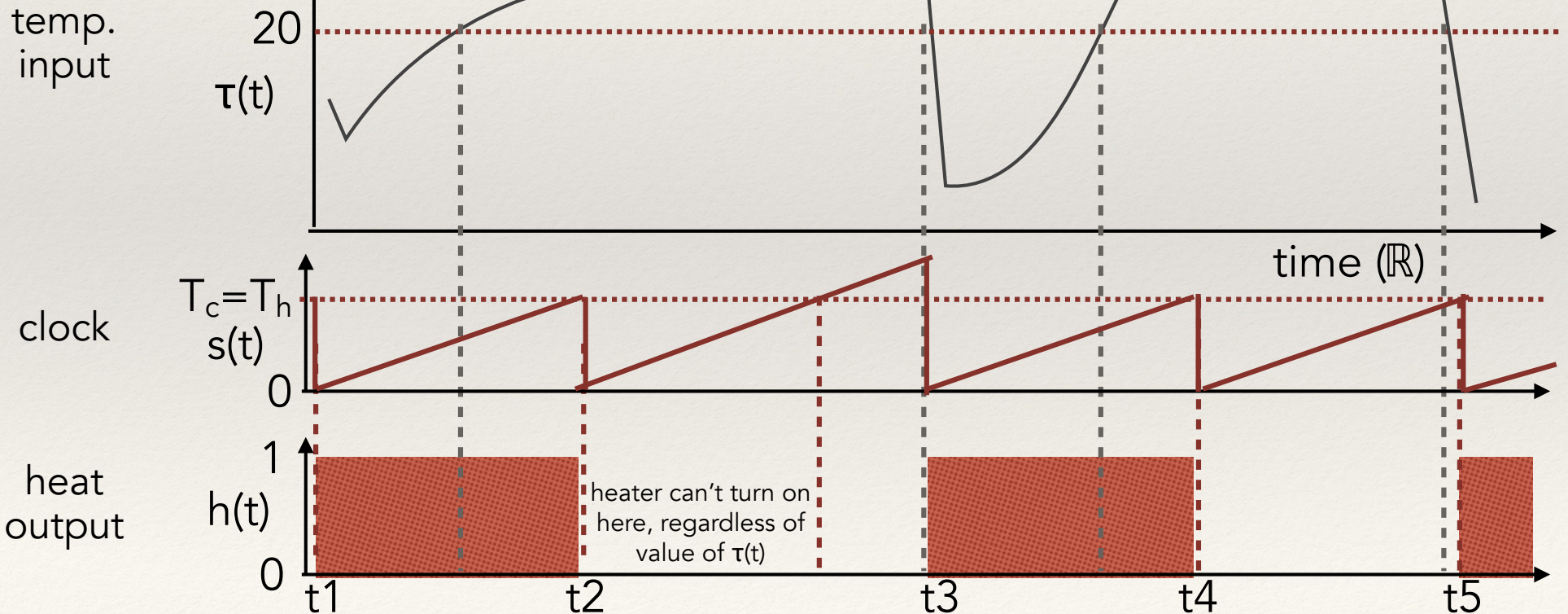
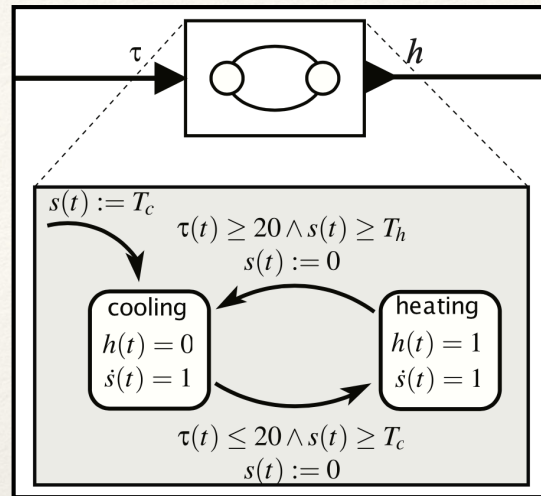
$h(t)$

0

1

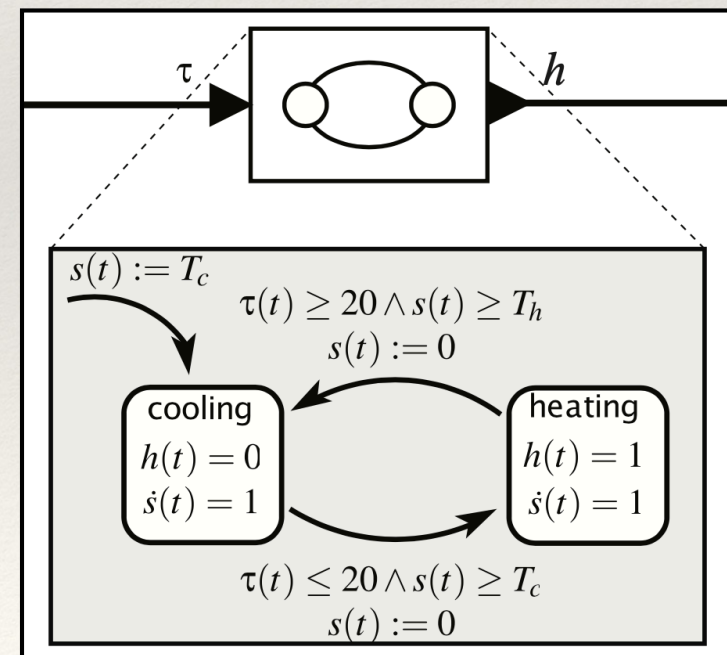
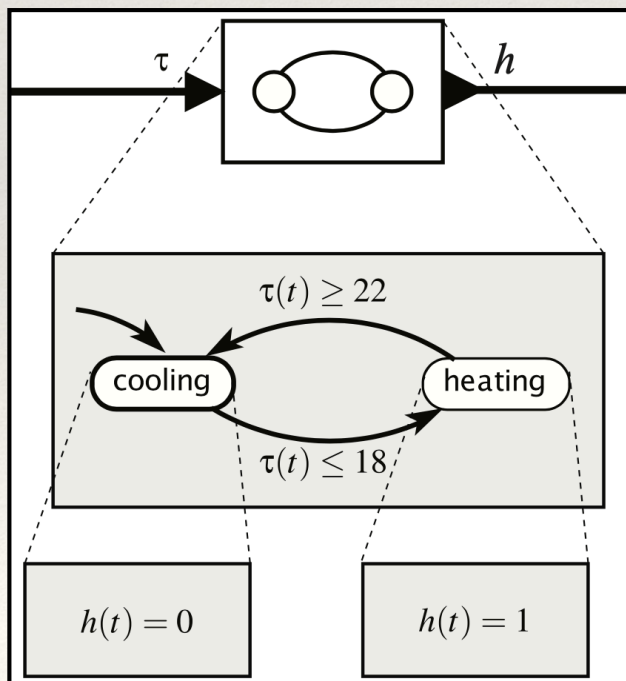
(Example on Board)

behavior of
Timed
Automaton
thermostat
model



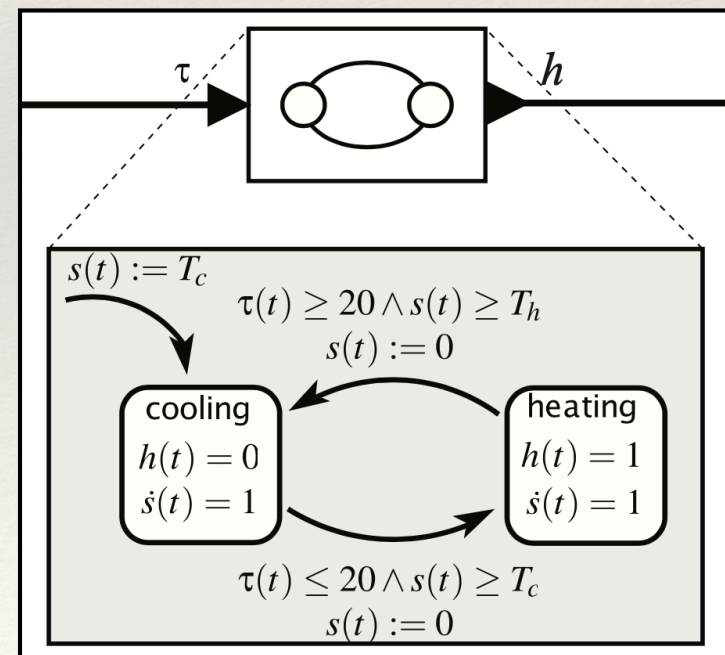
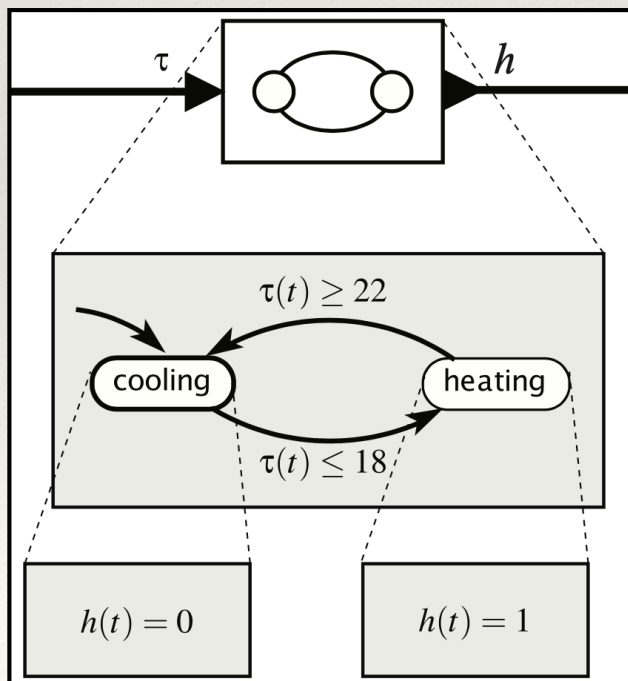
Compare the Two Models

- ❖ Which model guarantees heat can turn off when temp. is above 23?
- ❖ Which model has a finite bound on number of on/off cycles per hour

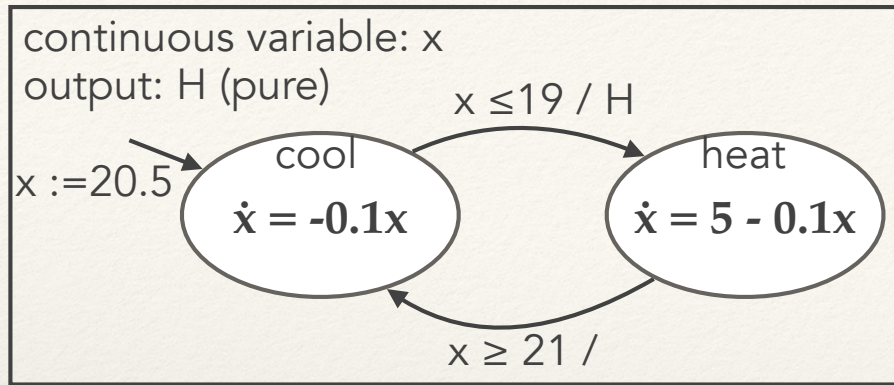


Compare the Two Models

- ❖ Which model guarantees heat can turn off when temp. is above 23?
- ❖ Which model has a finite bound on number of on/off cycles per hour
- ❖ More interesting properties can only be checked once we close loop by adding a model of how room temperature (τ) responds to heat (h) (and location invariants that force enabled transitions to occur)
- ❖ Closing the loop from (h) to (τ) will lead to a **hybrid system** model



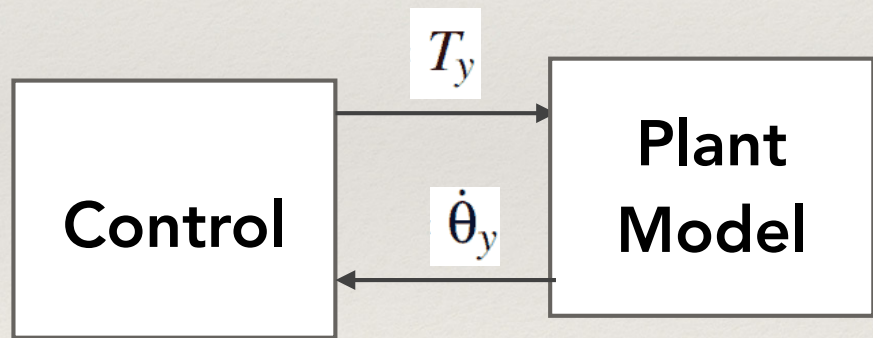
(General) Hybrid System



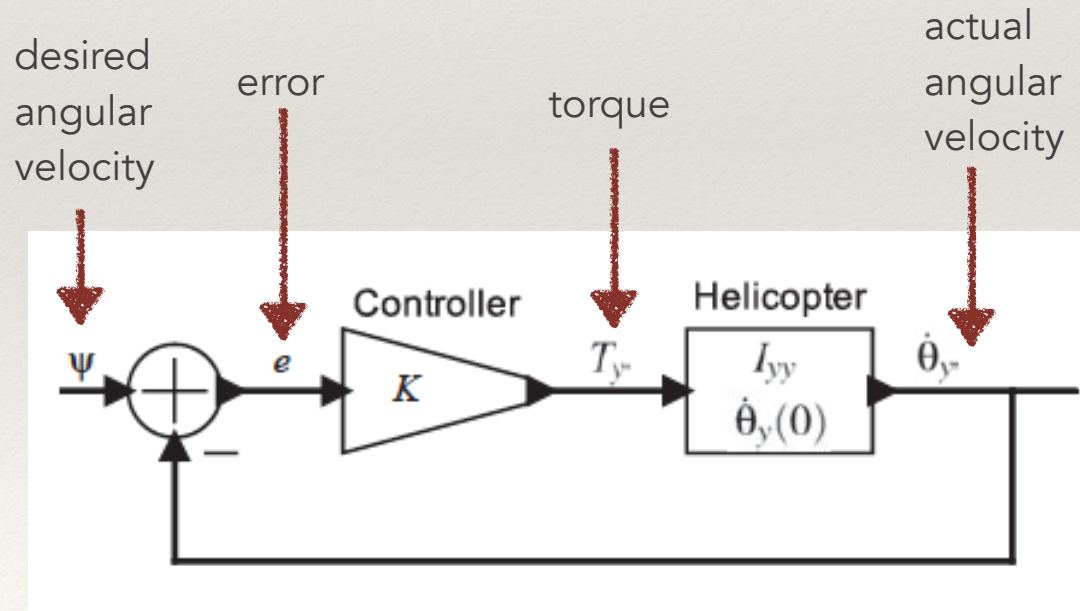
- ❖ Timed Automata often model protocols or controllers
- ❖ How does the system respond to the stimulus from the controller?
 - ❖ Reasoning about complete systems requires a plant model
 - ❖ Not just simple clocks to describe system dynamics
 - ❖ Thermal model of room, motion of rigid airplane, physiological model of medical patient, etc
- ❖ Exact reachability becomes problematic/impossible

motivation: Systems with Control

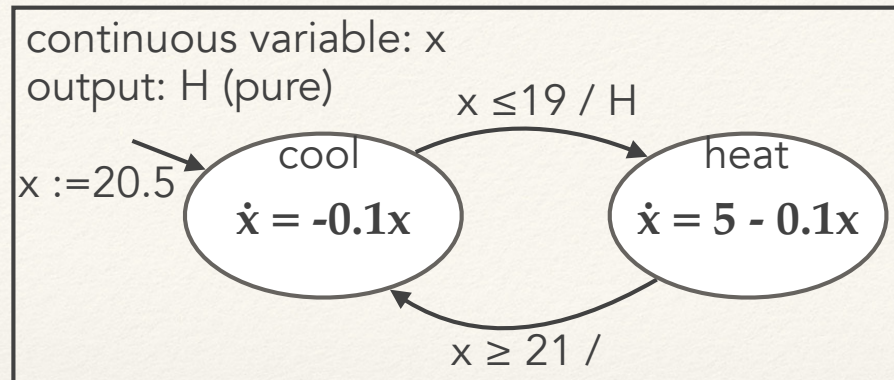
- ❖ Like cascade compositions we'll see in module 4, but with cycles in model
 - ❖ Output of control is input of plant
 - ❖ Input of plant is output of control
 - ❖ The two subsystems must agree on the value of these signals
- ❖ Sense and actuate



$$\dot{\theta}_y(t) = \dot{\theta}_y(0) + \frac{1}{I_{yy}} \int_0^t T_y(\tau) d\tau$$



(General) Hybrid System



Sketch of general approach for reasoning about hybrid systems:

- ❖ Break the infinite state space into regions like in Timed Automata
- ❖ Conservatively **over-approximate** the system dynamics in each region by considering all possible rates of change of variables in region
 - ❖ Allow that all rates of change for region can occur everywhere in region
- ❖ Anything reachable in the original system model is reachable in abstracted/over-approximate model (i.e. abstracted model using regions is sound)
- ❖ Abstracted model may reach parts of state space that original model cannot
 - ❖ A consequence of conservative over-approximation

How to find a useful abstraction for proving property?

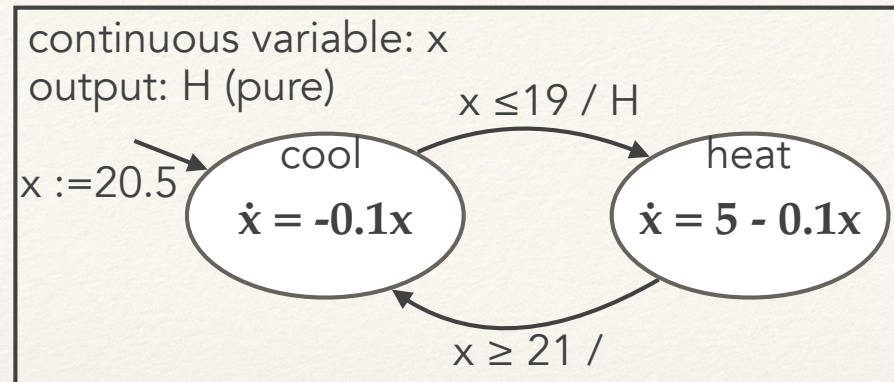
3/10/26

lecture 11

discuss:

- Exam 1 on 3/12. Coverage is on Canvas
- Exam 1 office hours:
 - Tues 3/10 @ 4-6
 - Weds 3/11 @ 2-4
- PS2 and Lab 2. Due on 3/10 and 3/24

Example on Board



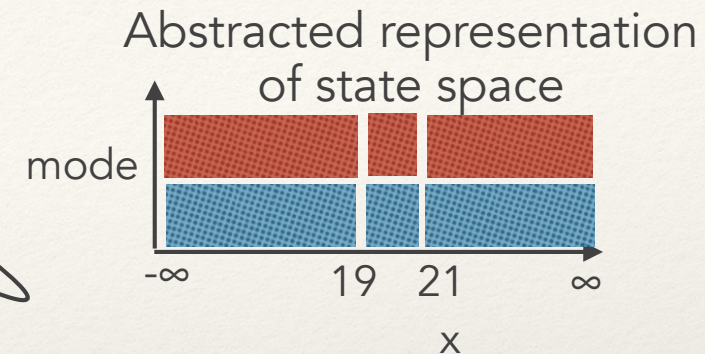
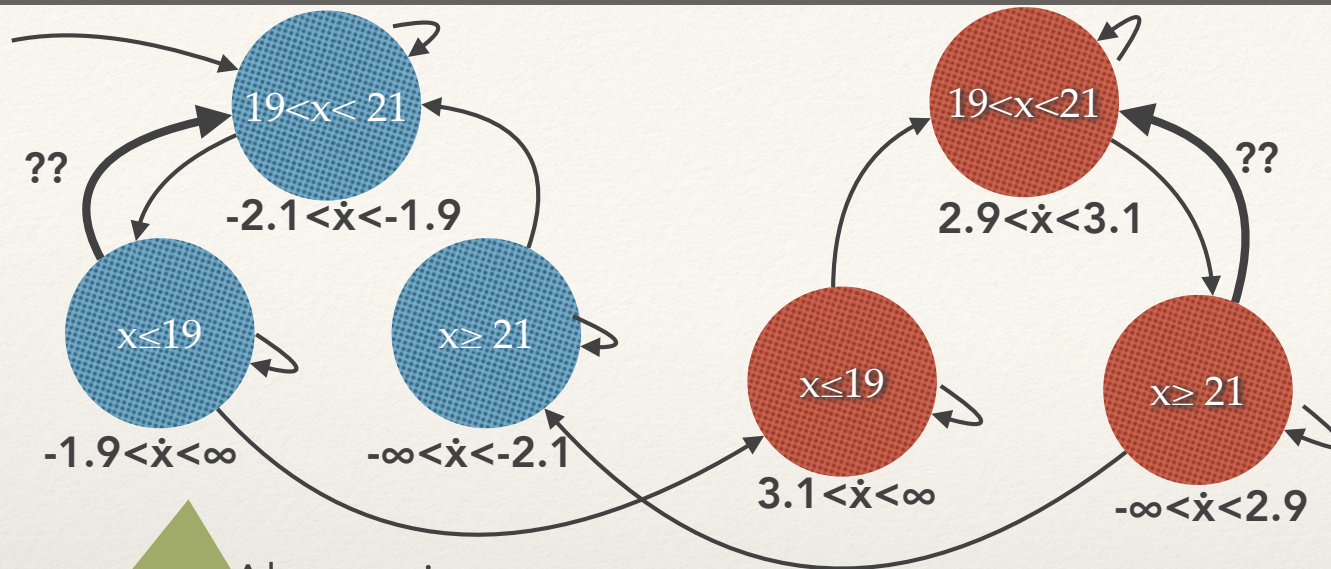
points

cooling	
x	$\dot{x}$
20.5	-2.05
20	-2.00
19	-1.90
18	-1.80
17	-1.70
16	-1.60
0	0
-1	0.10
-10	1.00

regions

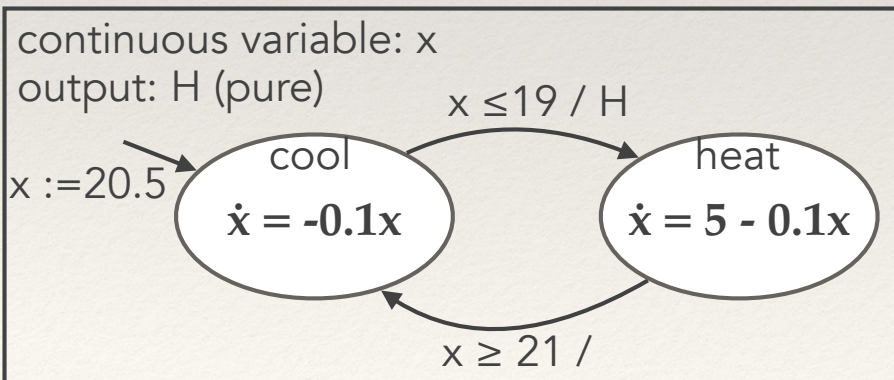
cooling	
x	$\dot{x}$
[21, 22]	[-2.2, -2.1]
[20, 21]	[-2.1, -2.0]
[19, 20]	[-2.0, -1.9]
[18, 19]	[-1.9, -1.8]
[20, 22]	[-2.2, -2.0]
[18, 20]	[-2.0, -1.8]
...	

Abstraction of Hybrid Systems



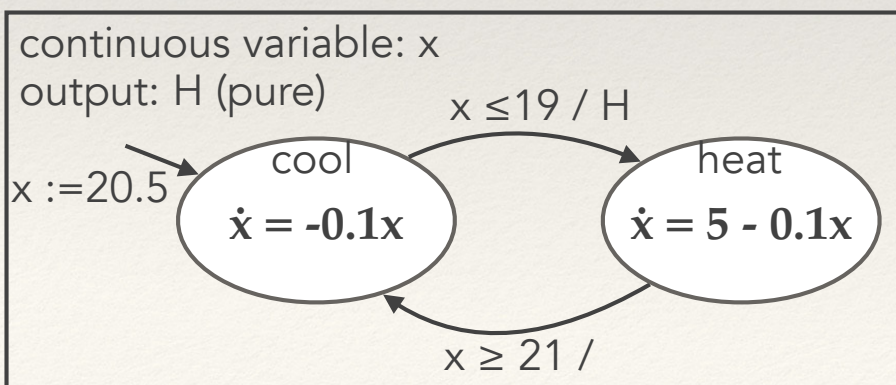
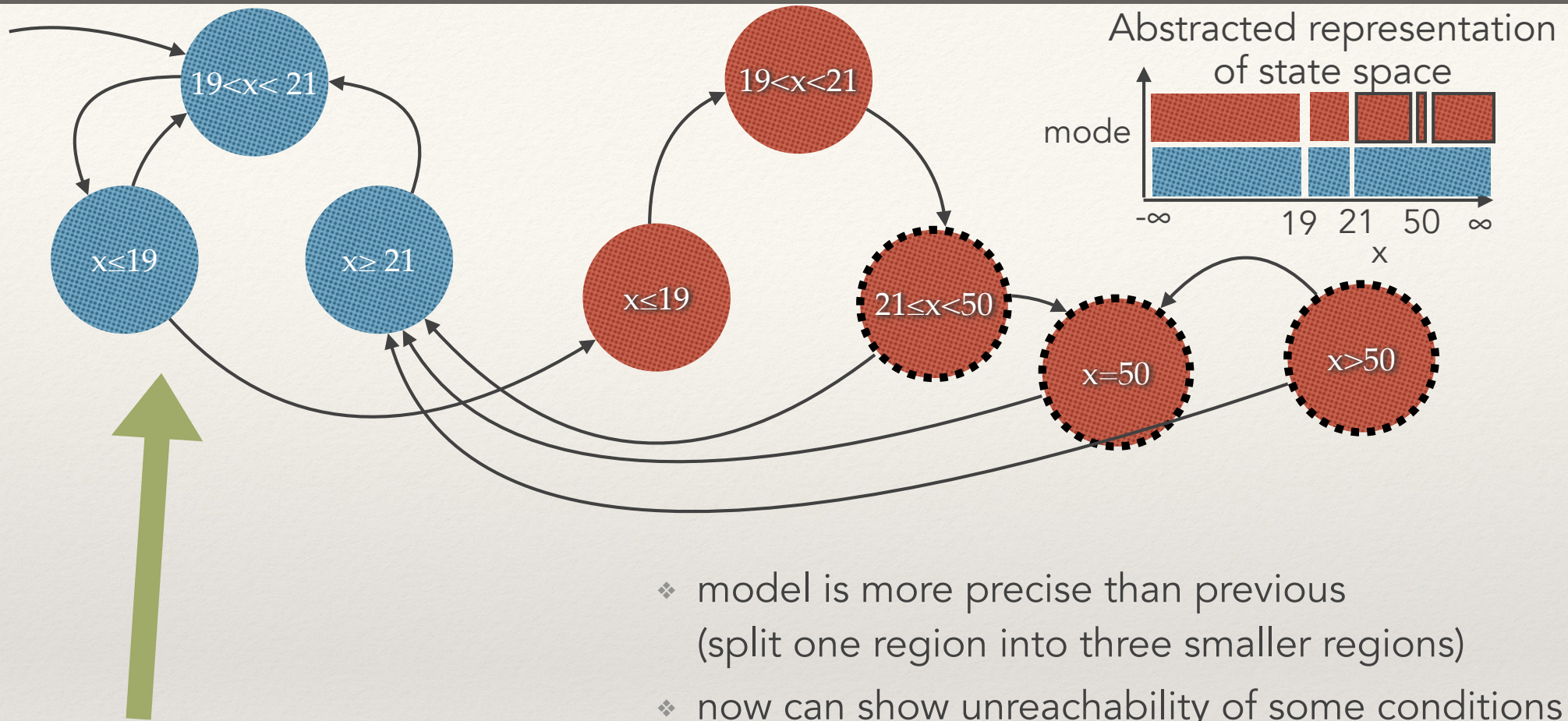
Abstraction

Key point: abstraction is **sound by construction** because it conservatively over-approximates system dynamics using bounds of $\dot{x}$. The abstract model can therefore reach every part of state space that refined model can reach, but due to over-approximation may also reach states that refined model can't

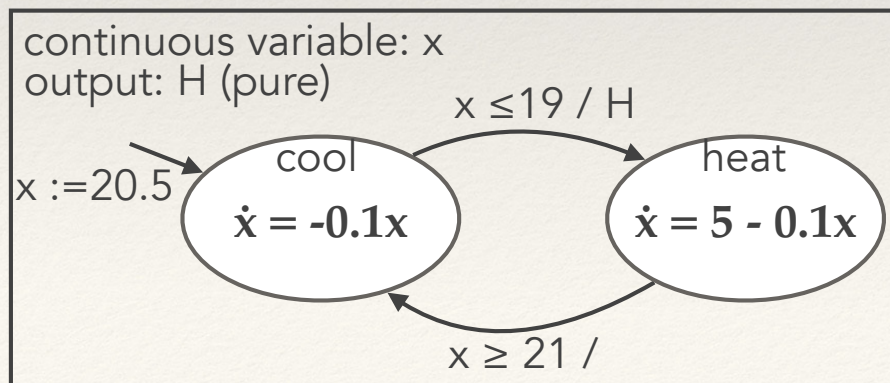
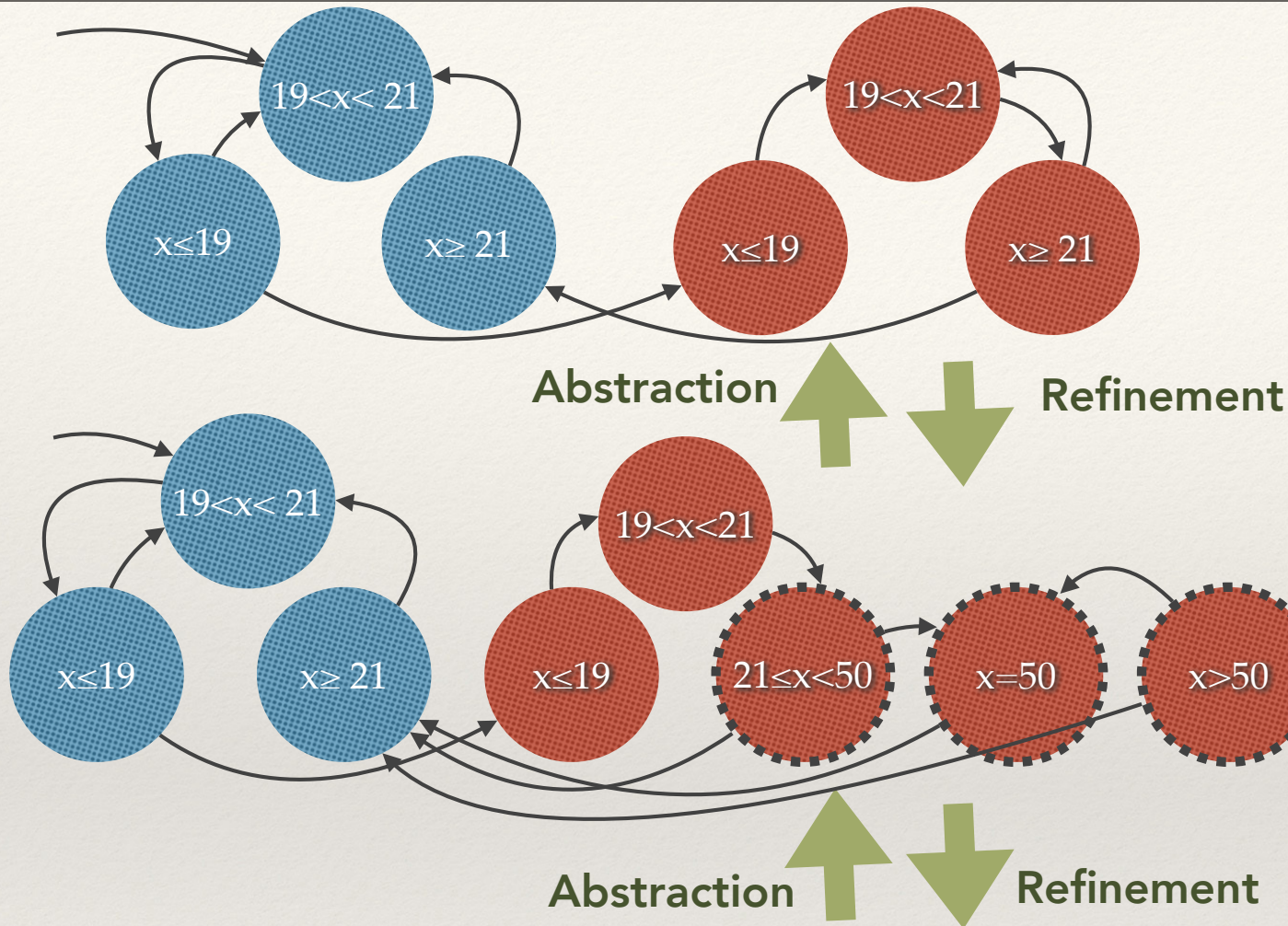


- ❖ What states are unreachable?
- ❖ What properties could be proved?
- ❖ E.g. is temp. always increasing in heating location?
- ❖ E.g. is temp. less than 50 in heating location?
- ❖ What to conclude from successful reachability?

Abstraction of Hybrid Systems

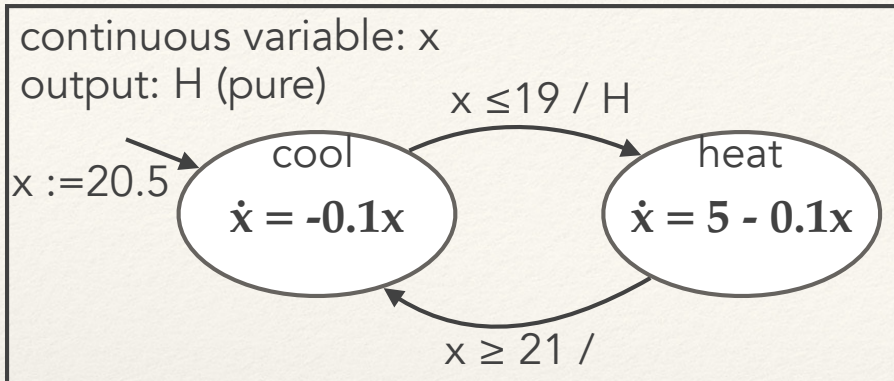


Hybrid Abstraction/Refinement



most precise/refined,
not at all conservative (exact),
number of regions is infinite

Hybrid Abstraction/Refinement Including Time



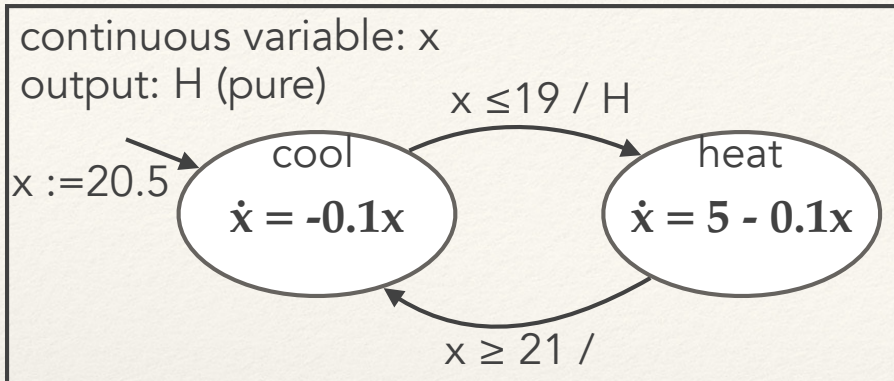
Property: When temperature is 22 in the cooling location, the temperature drops to 20 within 0.98 seconds

How to check prop. with reachability?



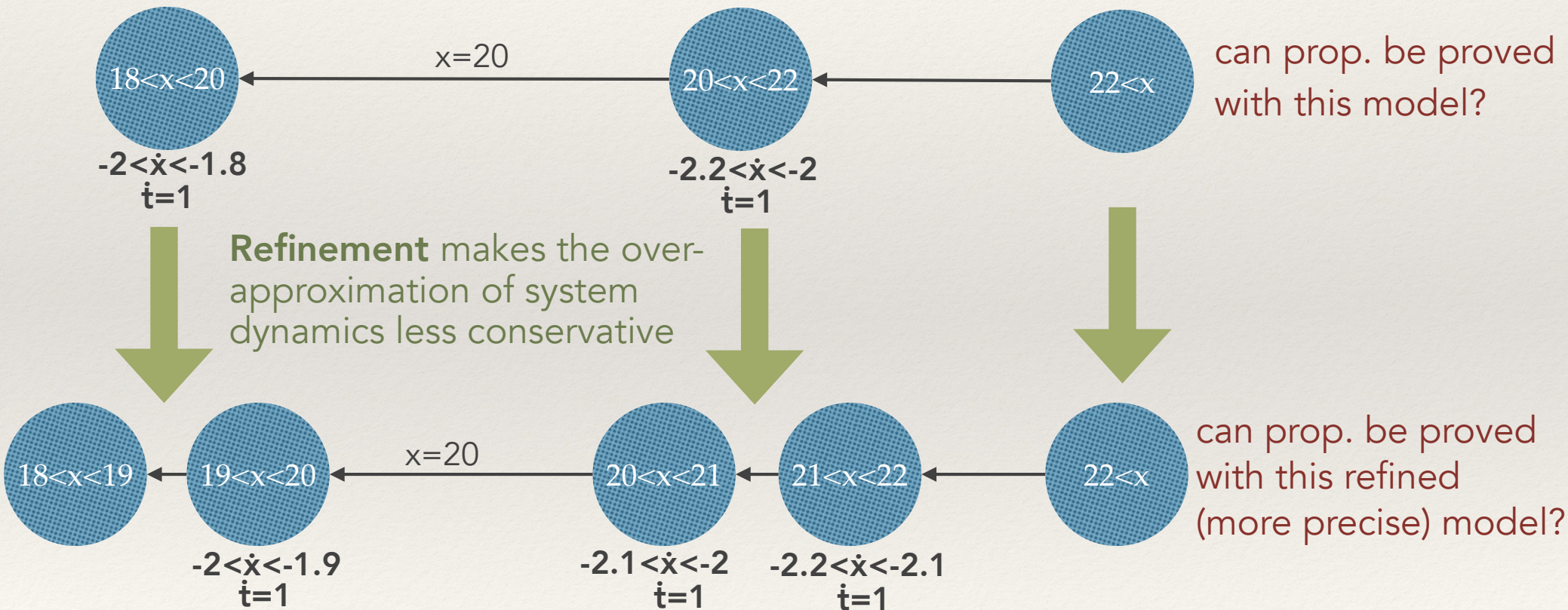
can prop. be proved with this model?

Hybrid Abstraction/Refinement Including Time

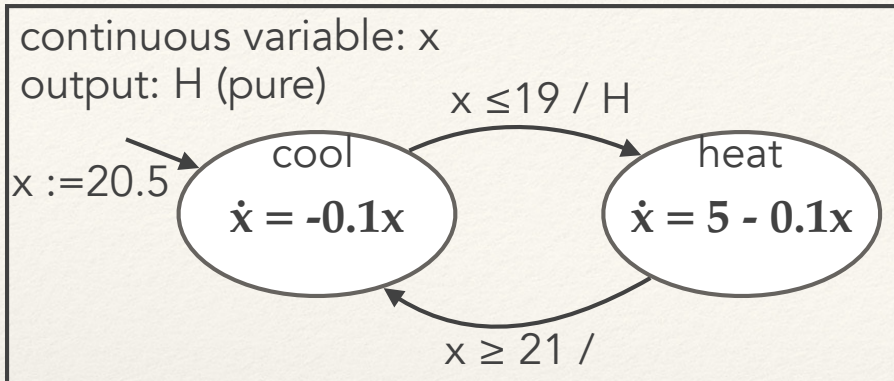


Property: When temperature is 22 in the cooling location, the temperature drops to 20 within 0.98 seconds

How to check prop. with reachability?

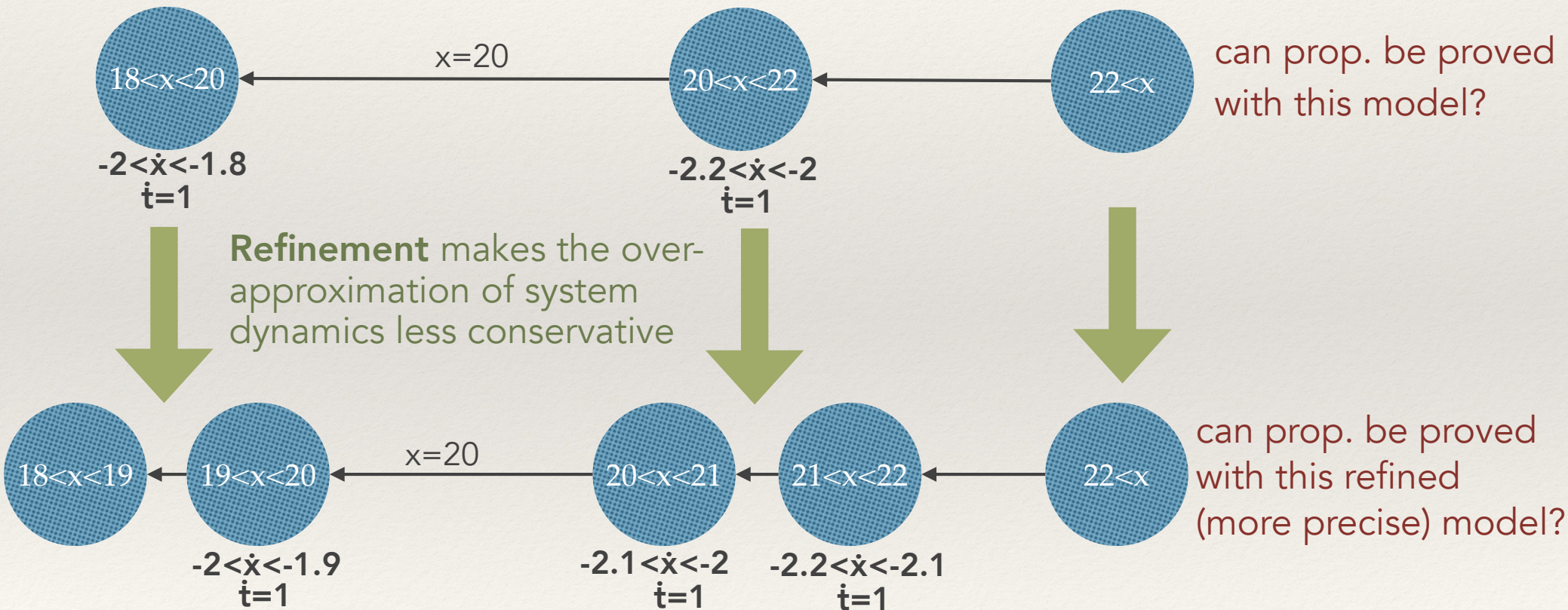


Hybrid Abstraction/Refinement Including Time



Property: When temperature is 22 in the cooling location, the temperature drops to 20 within 0.98 seconds

How to check prop. with reachability?



Is there a procedure for discovering appropriate level of abstraction?

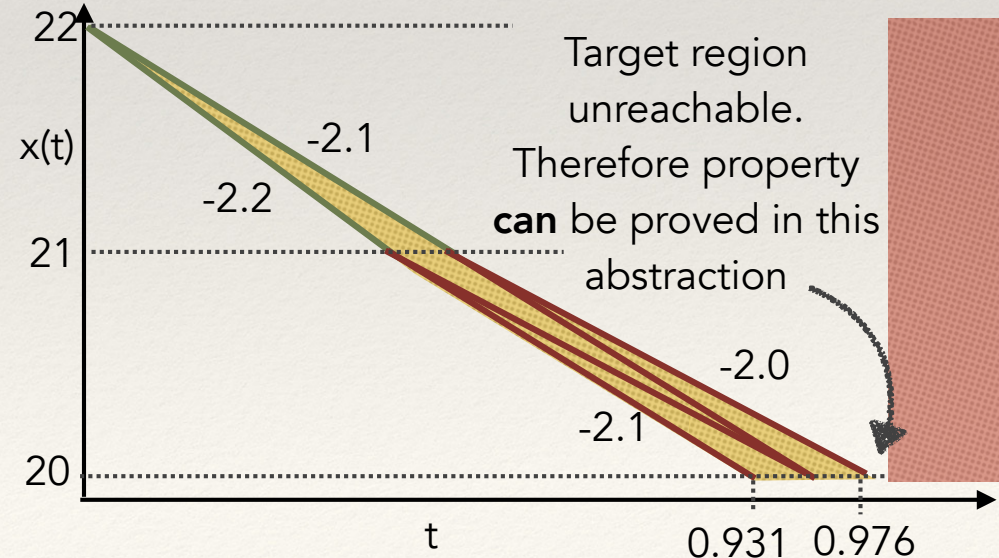
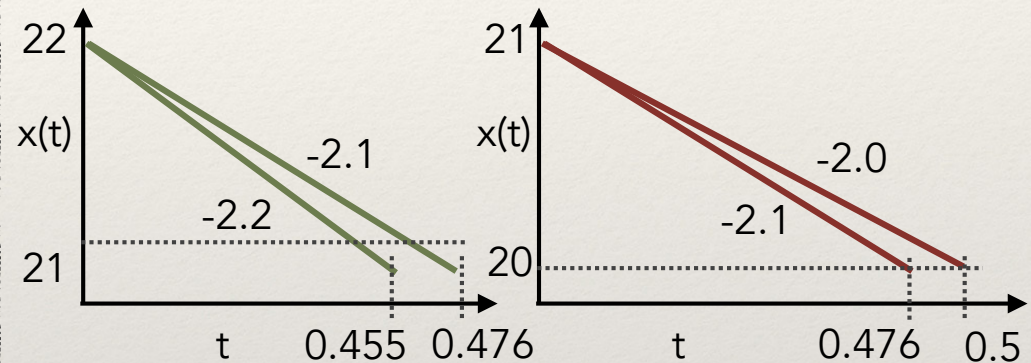
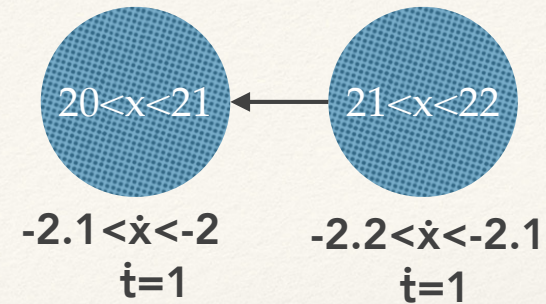
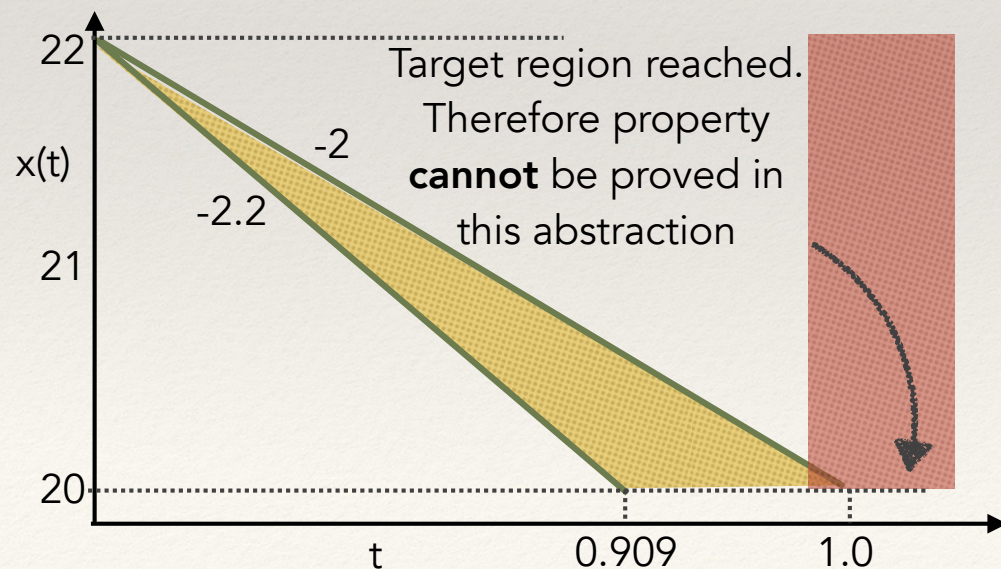
Example from Previous Slide

Tradeoff for a less abstract (more refined/precise) model is that it will have an increase in the runtime/complexity/num-of-regions

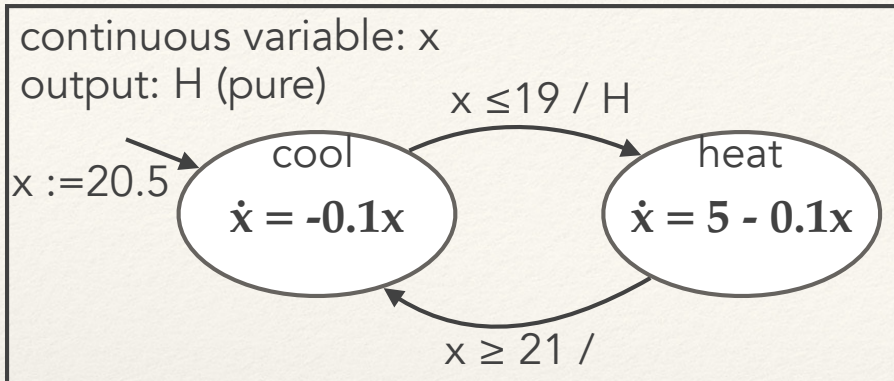
To prove that $x(t)$ is 20 within 0.98s, we try to show that the condition disproving the claim ($x(t) > 20$ and $t \geq 0.98$) cannot be reached in state space



$-2.2 < \dot{x} < -2$
 $t=1$

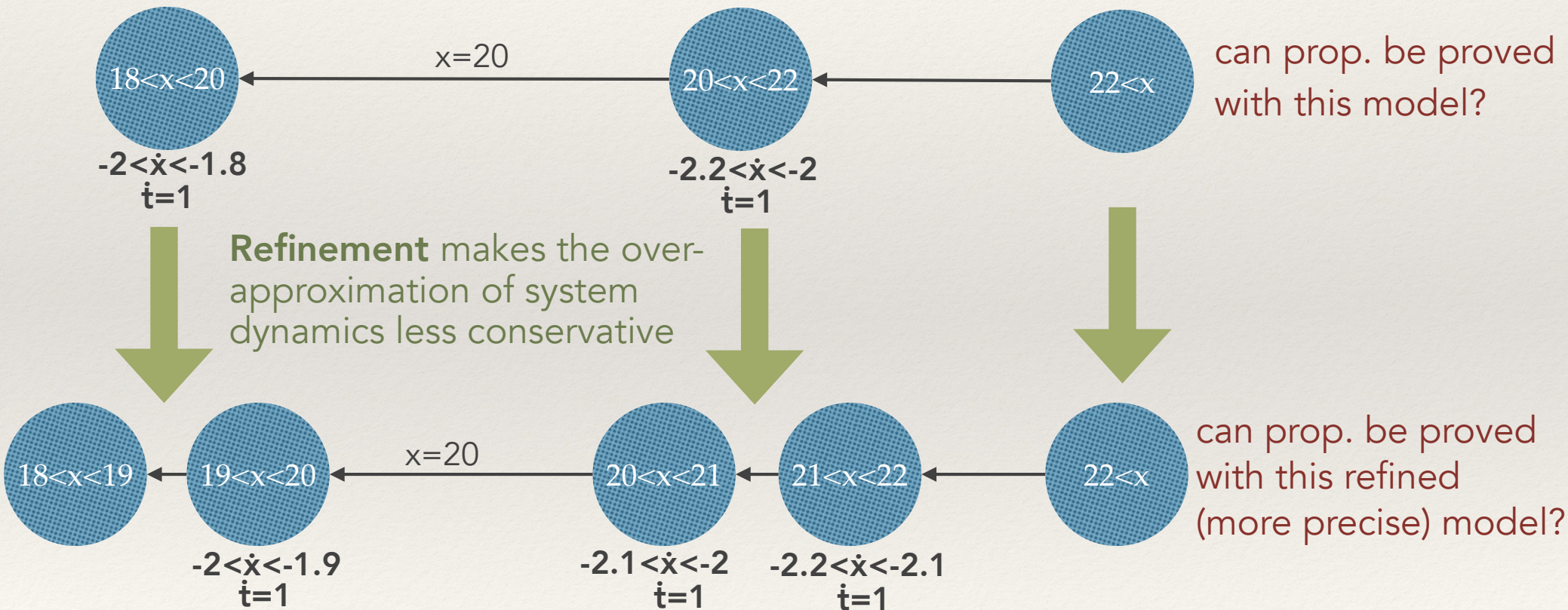


Hybrid Abstraction/Refinement Including Time



Property: When temperature is 21 in the cooling location, the temperature drops to 19 within 1.03 seconds

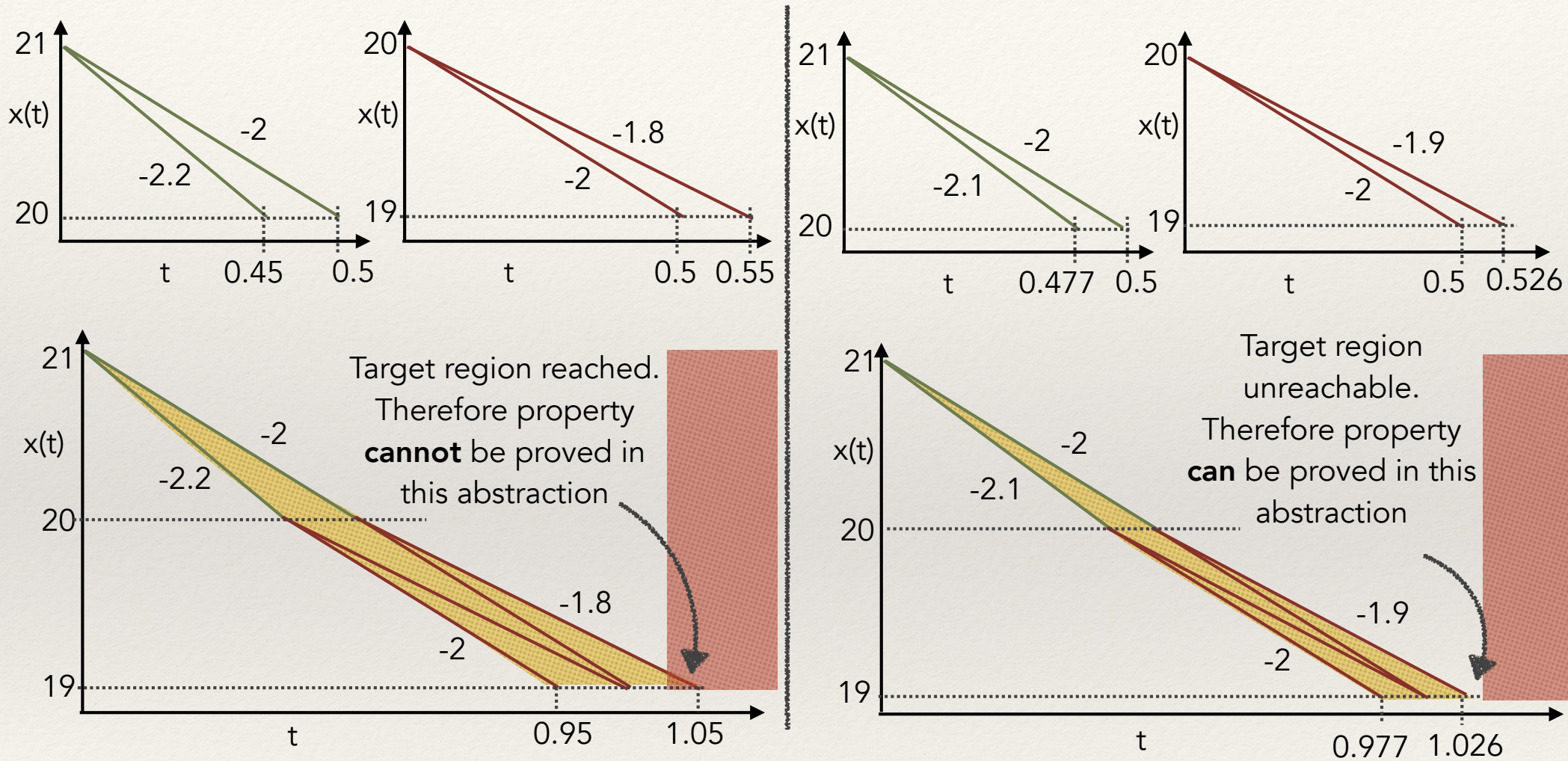
How to check prop. with reachability?



Is there a procedure for discovering appropriate level of abstraction?

Example from Previous Slide

To prove that $x(t)$ is 19 within 1.03s, we try to show that the condition which disproves it ($x(t) > 19$ and $t \geq 1.03$) cannot be reached in state space



Question: what is disadvantage of using a less abstract (more refined/precise) model?
Answer: Increase in runtime/complexity/num-of-regions

Hybrid Reachability using PHAVER

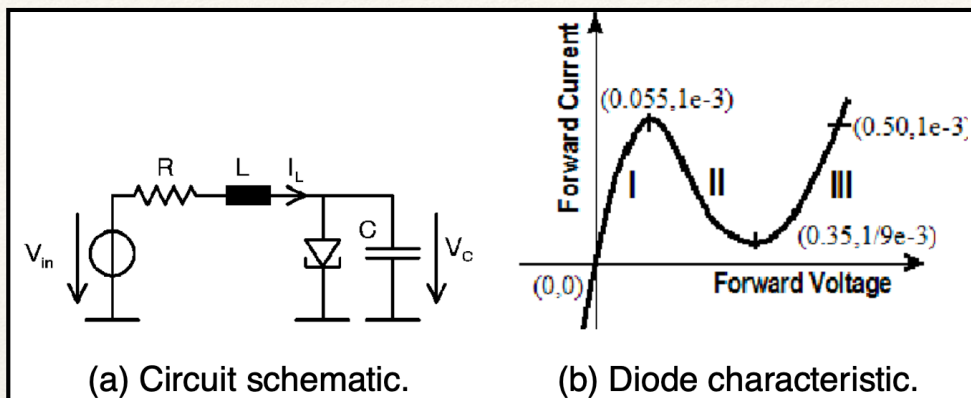
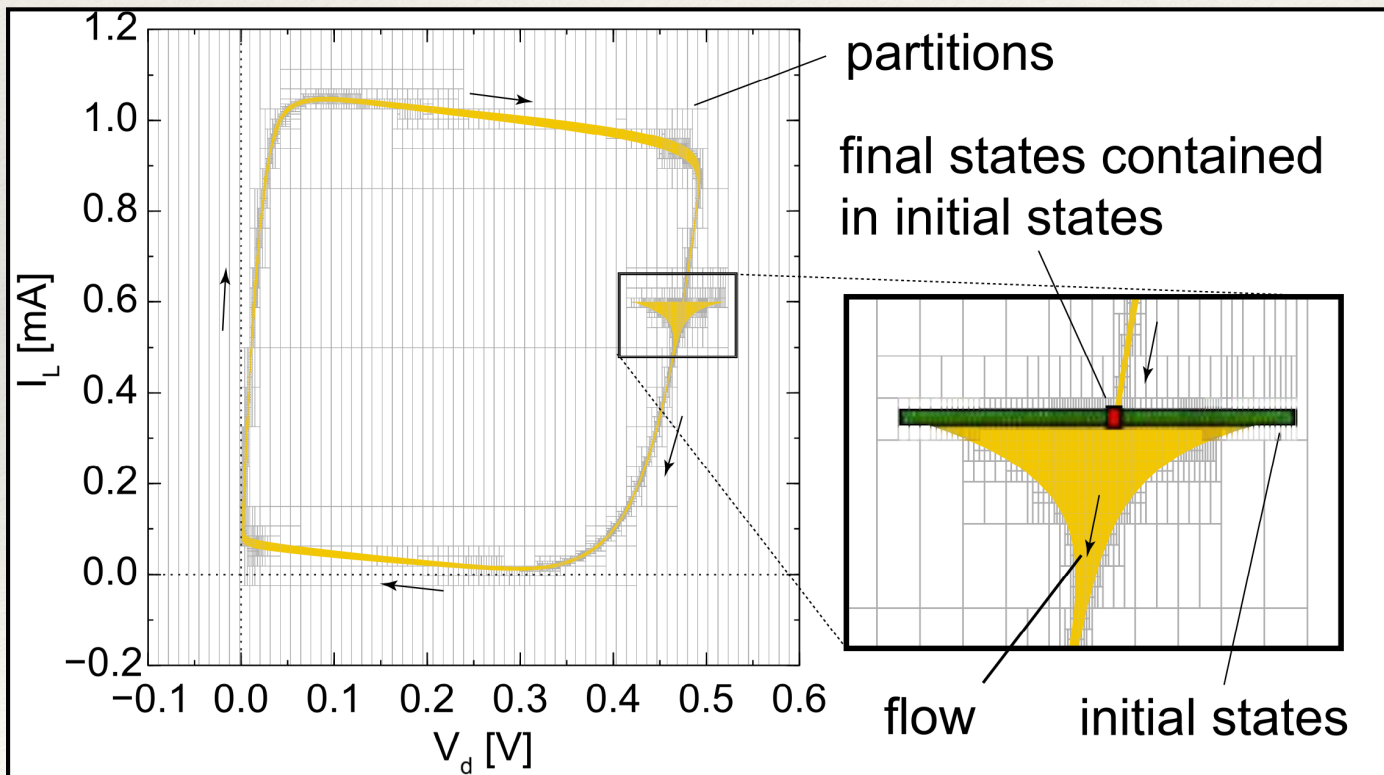


Figure 1. Tunnel diode oscillator circuit.

PHAVER (PHAVerLite) is the tool that you will use in PS3 and Lab3

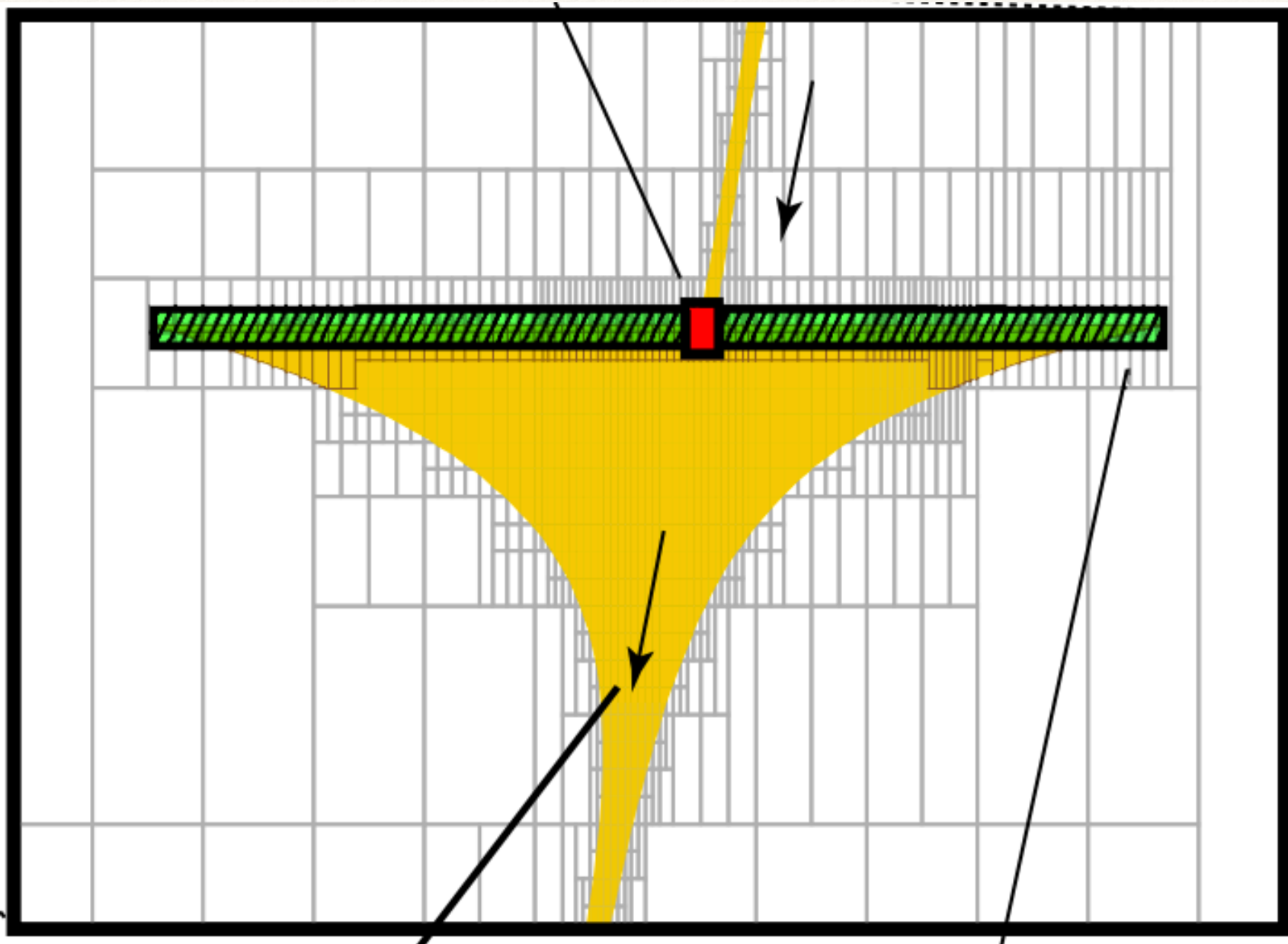
$$\begin{aligned}\dot{V}_d &= 1/C(-I_d(V_d) + I_L), \\ \dot{I}_L &= 1/L(-V_d - 1/G \cdot I_L + V_{in}),\end{aligned}$$



Verifying analog oscillator circuits using forward/backward abstraction refinement.

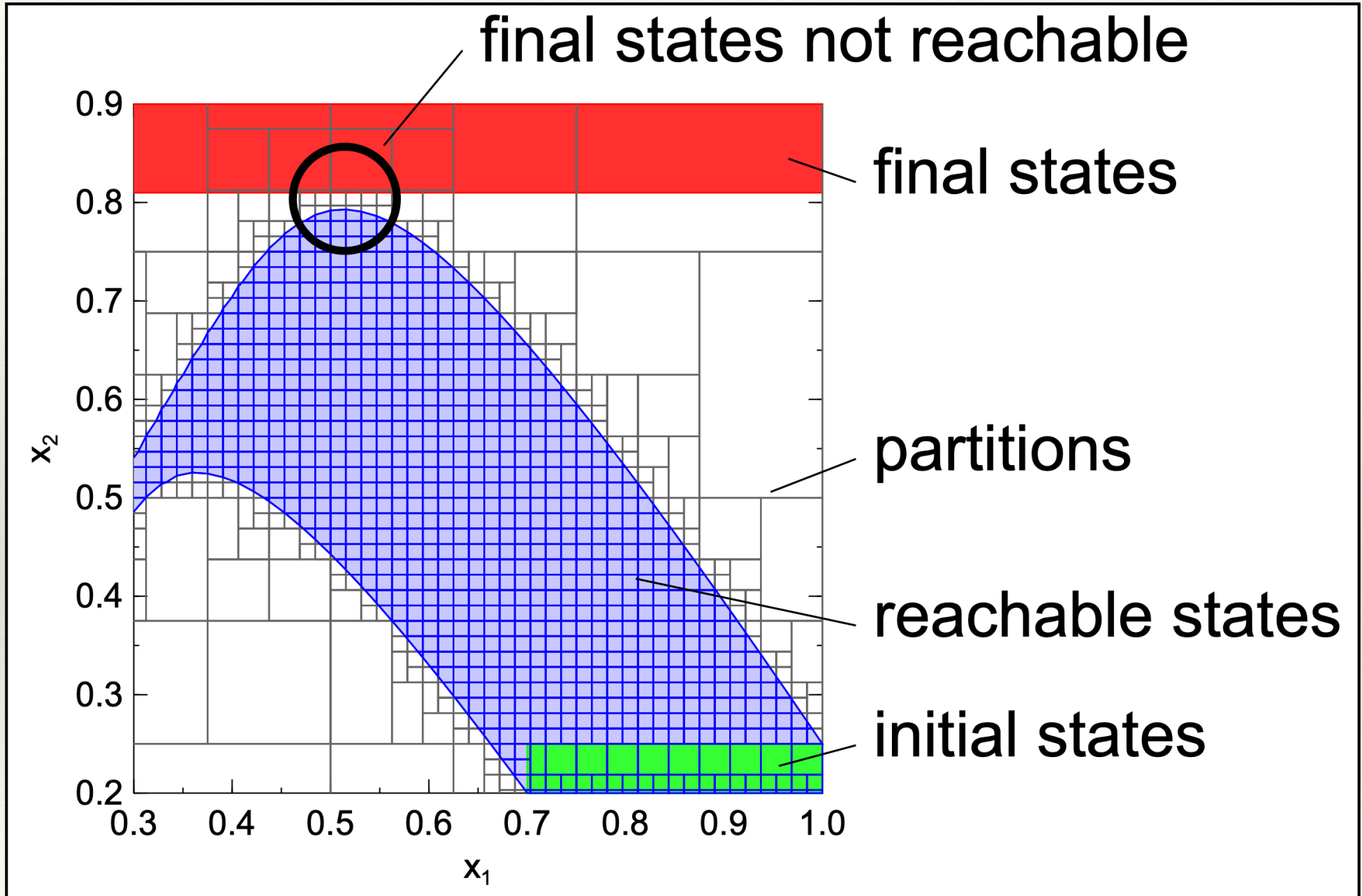
Goran Frehse, Bruce H. Krogh, and Rob A. Rutenbar. DATE '06 (Posted on Canvas)

Abstraction and Reachability



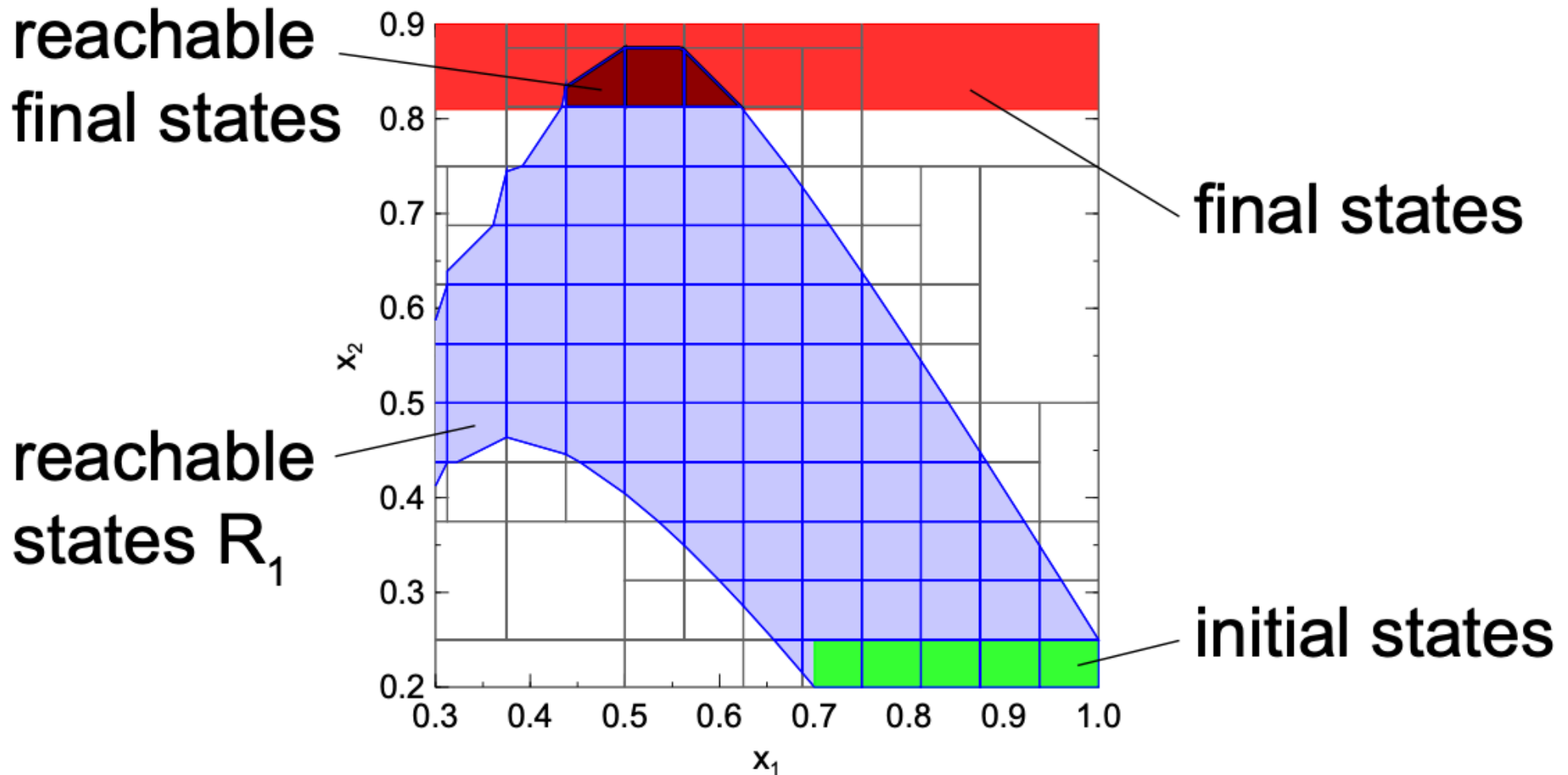
Verifying analog oscillator circuits using forward/backward abstraction refinement.
Goran Frehse, Bruce H. Krogh, and Rob A. Rutenbar. DATE '06

Forward Reachability

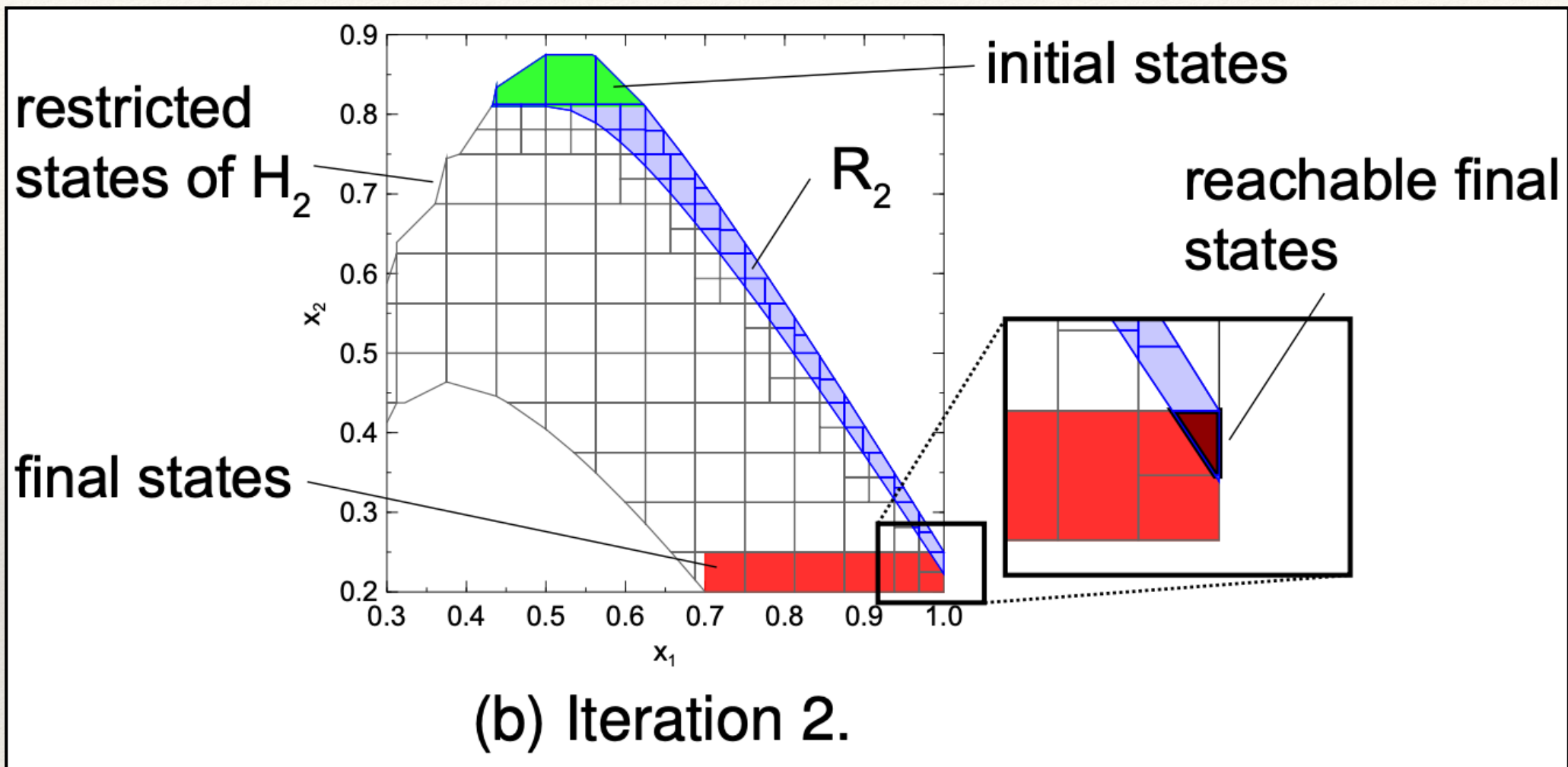


Verifying analog oscillator circuits using forward/backward abstraction refinement.
Goran Frehse, Bruce H. Krogh, and Rob A. Rutenbar. DATE '06

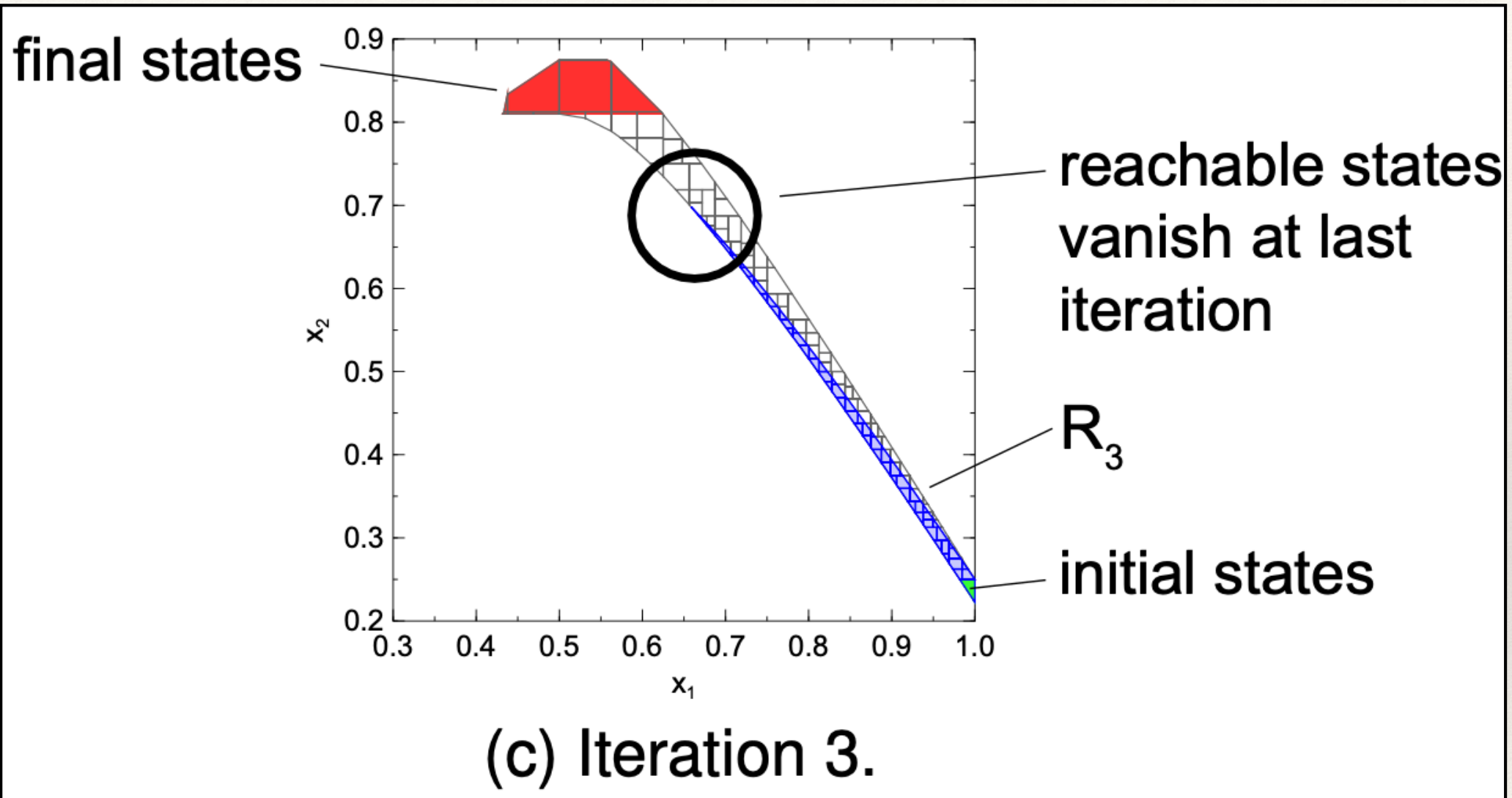
Forward Reachability



(a) Iteration 1.



Verifying analog oscillator circuits using forward/backward abstraction refinement.
Goran Frehse, Bruce H. Krogh, and Rob A. Rutenbar. DATE '06



3/12/26

lecture 12 date was exam

3/24/26

lecture 13

discuss:

- Exam 1 solution posted. PS2 solution posted
- Lab 2 due today (3/24)
- PS3 posted, due 4/3
- No office hours tomorrow

Excerpt from Canvas example

```
automaton sys1
  contr_var: x,y;  // the two variables of the automaton
  synclabs: a;    // don't need to change this, "a" is a label for transitions

  // -----
  // This comment block explains the three lines after the comment. Read carefully!
  //
  // loc locA: while x<=3 & y<=3 wait {x' == 1 & y' == 1}
  //   ... first location is named locA
  //   ... x<=3 & y<=3 is the location invariant for locA
  //   ... while waiting in locA, derivative of x and y are both 1.
  //
  //   when x>= 1 sync a do{x'==0 & y'==0} goto locA;
  //   ... define jump transition from locA with guard condition x>=1
  //   ... if jump occurs, x and y are set to 0, and sys1 goes back to locA
  //   ... NOTE: x' and y' on the jump refer to updates, not derivatives
  //
  //   when x < 1 sync a do{x'==x & y'==y} goto locB;
  //   ... define jump transition from locA with guard condition x<1
  //   ... if jump occurs, vars x and y are unchanged, and sys1 goes to locB
  //   ... NOTE: unchanged vars are set to own values (e.g. x'==x), not omitted
  // -----
  loc locA: while x<=3 & y<=3 wait {x' == 1 & y' == 1};
    when x>= 1 sync a do{x'==0 & y'==0} goto locA;
    when x < 1 sync a do{x'==x & y'==y} goto locB;
  loc locB: while x<=3 & y<=3 wait {x' == 1 & y' == 1};
    when x>=1 sync a do {x'==0 & y'==y} goto locB;
    when y>0   sync a do {x'==x & y'==y} goto locA;
```

- ❖ This is from **ps3.pha** on Canvas
- ❖ Read this model carefully to understand Phaver's syntax.
- ❖ You will use this in ps3

Bouncing Ball (Lab 3 Model)

```
// Hybrid System Example: Bouncing Ball
```

```
g:=1;      // constant for acceleration from gravity
```

```
automaton bball
```

```
  contr_var: x, v;      // x is position, v is velocity
```

```
  synclabs: jump;
```

```
  loc falling: while  $0 \leq x \leq 3$  &  $-3 \leq v \leq 3$  wait { $x'==v$  &  $v'==-g$ }
```

```
    when  $x==0$  &  $v < 0$  sync jump do { $v'==-v*0.5$  &  $x'==x$ } goto falling;
```

```
  initially: falling &  $x==2$  &  $v==0$ ;
```

```
end
```

```
// ----- Start of Analysis -----
```

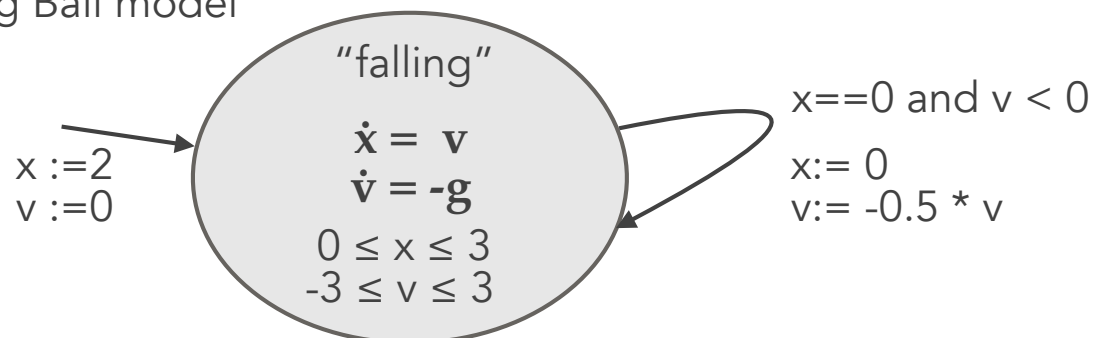
```
bball.add_label(tau);
```

```
pc:=0.2;      // define partition constraint
```

```
bball.set_partition_constraints((x,pc),(v,pc),tau);
```

This model is posted on Canvas: [bouncing_ball.pha](#)

Bouncing Ball model



Reachability using PHAVer (Lab 3)

```
// Hybrid System Example: Bouncing Ball

g:=1;      // constant for acceleration from gravity

automaton bball
  contr_var: x, v;    // x is position, v is velocity
  synclabs: jump;
  loc falling: while 0<=x & x<=3 & -3<=v & v<=3 wait {x'==v & v'==-g}
    when x==0 & v<0 sync jump do {v'==v*0.5 & x'==x} goto falling;
  initially: falling & x==2 & v==0;
end

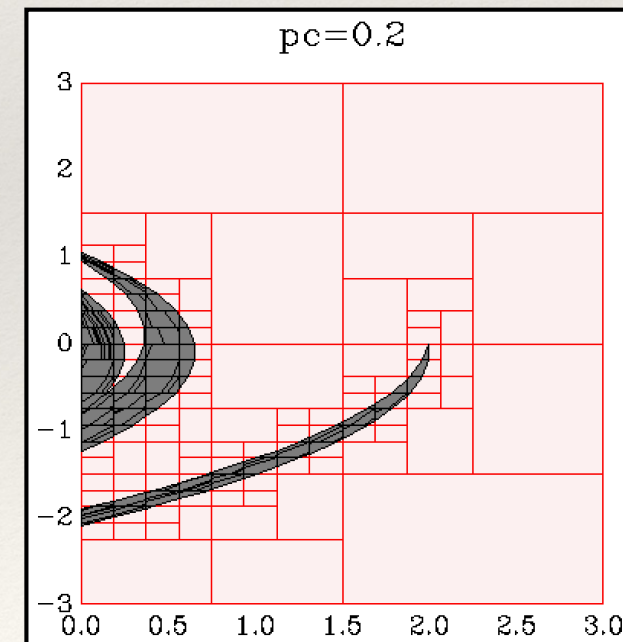
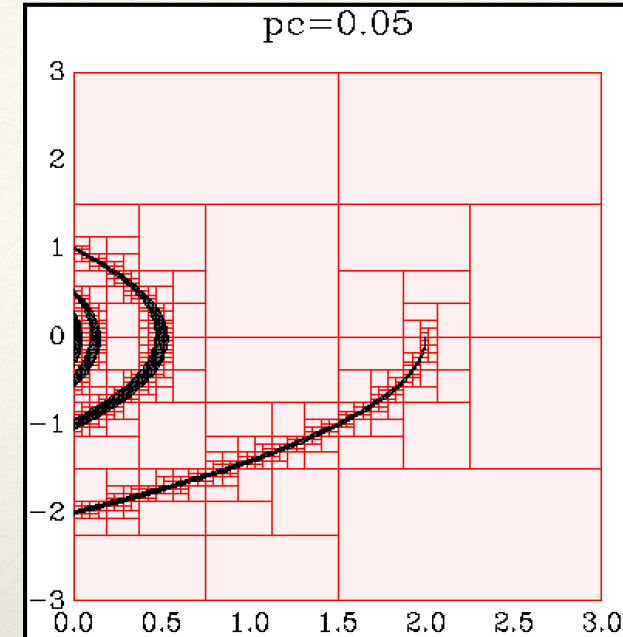
// ----- Start of Analysis -----
bball.add_label(tau);
pc:=0.2;      // define partition constraint
bball.set_partition_constraints((x,pc),(v,pc),tau);

reg=bball.reachable;      // reached part of state space
reg.print("out_reach",2);  // output as a list of vertices

inv=bball.get_invariants;  // grid of partitions explored
inv.print("out_inv",2);

// add commands here to check reachable states

// plots shown are generated with this command ...
// graph -T png -C -B -L "pc=0.2" -q0.05 out_inv -C -B -q0.5 out_reach > comb_0p2.png
```



- ❖ What are hybrid automata and timed automata?
- ❖ Be able to recognize each one
- ❖ Given some behavior, produce a hybrid system that could have generated behavior (and vice versa)
- ❖ Check by hand whether a hybrid system satisfies some property (for simple cases; otherwise computer needed)
- ❖ Understand the implications of exact reachability vs conservative (over-approximated) reachability